

《动手学深度学习》

PyTorch 版学习笔记

内容提炼

2026 年 9 月 1 日

说明

本笔记按 d2l-zh 2.0.0 PyTorch 版原书目录提炼全书。正文保留全部章、节、子节标题；印刷目录与书签只显示到章和主要节。正文以公式陈述定义、成立条件、关键推导与结论；衔接句只连接相邻公式。

目录

说明	i
前言	viii
安装	ix
符号	x
第一章 引言	1
1.1 日常生活中的机器学习	1
1.2 机器学习中的关键组件	1
1.3 各种机器学习问题	2
1.4 起源	3
1.5 深度学习的发展	3
1.6 深度学习的成功案例	3
1.7 特点	3
1.8 小结	3
1.9 练习	4
第二章 预备知识	5
2.1 数据操作	5
2.2 数据预处理	7
2.3 线性代数	8
2.4 微积分	11
2.5 自动微分	13
2.6 概率	16

2.7 查阅文档	18
第三章 线性神经网络	20
3.1 线性回归	20
3.2 线性回归的从零开始实现	23
3.3 线性回归的简洁实现	25
3.4 softmax 回归	26
3.5 图像分类数据集	29
3.6 softmax 回归的从零开始实现	30
3.7 softmax 回归的简洁实现	32
第四章 多层感知机	34
4.1 多层感知机	34
4.2 多层感知机的从零开始实现	36
4.3 多层感知机的简洁实现	37
4.4 模型选择、欠拟合和过拟合	38
4.5 权重衰减	40
4.6 暂退法 (Dropout)	42
4.7 前向传播、反向传播和计算图	44
4.8 数值稳定性和模型初始化	45
4.9 环境和分布偏移	47
4.10 实战 Kaggle 比赛：预测房价	51
第五章 深度学习计算	53
5.1 层和块	53
5.2 参数管理	54
5.3 延后初始化	56
5.4 自定义层	57
5.5 读写文件	58
5.6 GPU	59
第六章 卷积神经网络	61
6.1 从全连接层到卷积	61

6.2	图像卷积	63
6.3	填充和步幅	64
6.4	多输入多输出通道	65
6.5	汇聚层	66
6.6	卷积神经网络 (LeNet)	67
第七章	现代卷积神经网络	69
7.1	深度卷积神经网络 (AlexNet)	69
7.2	使用块的网络 (VGG)	71
7.3	网络中的网络 (NiN)	72
7.4	含并行连结的网络 (GoogLeNet)	73
7.5	批量规范化	74
7.6	残差网络 (ResNet)	75
7.7	稠密连接网络 (DenseNet)	77
第八章	循环神经网络	79
8.1	序列模型	79
8.2	文本预处理	81
8.3	语言模型和数据集	82
8.4	循环神经网络	84
8.5	循环神经网络的从零开始实现	85
8.6	循环神经网络的简洁实现	87
8.7	通过时间反向传播	87
第九章	现代循环神经网络	91
9.1	门控循环单元 (GRU)	91
9.2	长短期记忆网络 (LSTM)	93
9.3	深度循环神经网络	95
9.4	双向循环神经网络	96
9.5	机器翻译与数据集	97
9.6	编码器-解码器架构	98
9.7	序列到序列学习 (seq2seq)	99

9.8 束搜索	101
第十章 注意力机制	103
10.1 注意力提示	103
10.2 注意力汇聚: Nadaraya-Watson 核回归	104
10.3 注意力评分函数	106
10.4 Bahdanau 注意力	107
10.5 多头注意力	108
10.6 自注意力和位置编码	109
10.7 Transformer	110
第十一章 优化算法	113
11.1 优化和深度学习	113
11.2 凸性	114
11.3 梯度下降	117
11.4 随机梯度下降	119
11.5 小批量随机梯度下降	120
11.6 动量法	121
11.7 AdaGrad 算法	124
11.8 RMSProp 算法	125
11.9 Adadelta	126
11.10 Adam 算法	126
11.11 学习率调度器	127
第十二章 计算性能	130
12.1 编译器和解释器	130
12.2 异步计算	131
12.3 自动并行	132
12.4 硬件	133
12.5 多 GPU 训练	135
12.6 多 GPU 的简洁实现	136
12.7 参数服务器	137

第十三章 计算机视觉	139
13.1 图像增广	139
13.2 微调	140
13.3 目标检测和边界框	141
13.4 锚框	141
13.5 多尺度目标检测	143
13.6 目标检测数据集	143
13.7 单发多框检测 (SSD)	144
13.8 区域卷积神经网络 (R-CNN) 系列	146
13.9 语义分割和数据集	147
13.10 转置卷积	148
13.11 全卷积网络	149
13.12 风格迁移	150
13.13 实战 Kaggle 比赛: 图像分类 (CIFAR-10)	151
13.14 实战 Kaggle 比赛: 狗的品种识别 (ImageNet Dogs)	153
第十四章 自然语言处理: 预训练	155
14.1 词嵌入 (word2vec)	155
14.2 近似训练	157
14.3 用于预训练词嵌入的数据集	158
14.4 预训练 word2vec	159
14.5 全局向量的词嵌入 (GloVe)	160
14.6 子词嵌入	161
14.7 词的相似性和类比任务	162
14.8 来自 Transformers 的双向编码器表示 (BERT)	163
14.9 用于预训练 BERT 的数据集	164
14.10 预训练 BERT	165
第十五章 自然语言处理: 应用	166
15.1 情感分析及数据集	166
15.2 情感分析: 使用循环神经网络	167
15.3 情感分析: 使用卷积神经网络	167

15.4 自然语言推断与数据集	169
15.5 自然语言推断：使用注意力	169
15.6 针对序列级和词元级应用微调 BERT	171
15.7 自然语言推断：微调 BERT	172
附录 A 深度学习工具	174
A.1 使用 Jupyter Notebook	174
A.2 使用 Amazon SageMaker	175
A.3 使用 Amazon EC2 实例	176
A.4 选择服务器和 GPU	177
A.5 为本书做贡献	178
A.6 d2l API 文档	179

前言

本书把深度学习写成可运行的对象与映射：用数据构造经验分布，用可微模型给出预测，用标量目标衡量误差，用优化算法更新参数。笔记只保留定义、成立条件、关键推导和结论。

写作方式是同一对象的三种等价表示同时出现：

概念 $\longleftrightarrow$ 公式 $\longleftrightarrow$ 可执行计算.

因此后文以公式为主陈述模型，代码只在需要核验映射时出现。

安装

运行原书代码需要 Python 3 环境、PyTorch 与辅助包 d2l。记依赖集合为 $\mathcal{P}$ ，运行时环境为映射

$(\text{Python}, \mathcal{P}) \mapsto \text{可执行的 Jupyter Notebook}.$

原书推荐用 conda 创建名为 d2l 的隔离环境，再安装 CPU 或 GPU 版 PyTorch。有 NVIDIA GPU 且已配置 CUDA 时，同一模型的前向与反向可在加速器上求值；前几章用 CPU 即可完成。每次运行前激活该环境，使解释器与包版本一致。

符号

沿用原书记号。后文把 $\mathbf{x}$ 写成 $\boldsymbol{x}$, 含义不变。

数字

$x \in \mathbb{R}$	标量,
$\boldsymbol{x} \in \mathbb{R}^n$	向量,
$\boldsymbol{X} \in \mathbb{R}^{a \times b}$	矩阵,
X	高阶张量,
$\boldsymbol{I}$	单位矩阵,
$x_i = [\boldsymbol{x}]_i$	向量第 i 个元素,
$x_{ij} = [\boldsymbol{X}]_{ij}$	矩阵第 i 行第 j 列.

集合

$\mathcal{X}$	集合, $\mathbb{Z}$ 整数, $\mathbb{R}$ 实数,
$\mathbb{R}^n, \mathbb{R}^{a \times b}$	向量与矩阵空间,
$\mathcal{A} \cup \mathcal{B}, \mathcal{A} \cap \mathcal{B}, \mathcal{A} \setminus \mathcal{B}$	并、交、相对补.

函数与算子

$f(\cdot), \log(\cdot), \exp(\cdot), \mathbf{1}_{\mathcal{X}}$	指示函数,
$(\cdot)^\top, \boldsymbol{X}^{-1}, \odot$	转置、逆、Hadamard 积,
$[\cdot, \cdot], \mathcal{X} , \ \cdot\ _p, \ \cdot\ $	连结、基数、 L_p 与 L_2 ,
$\langle \boldsymbol{x}, \boldsymbol{y} \rangle = \boldsymbol{x}^\top \boldsymbol{y}, \quad \sum, \prod, \stackrel{\text{def}}{=}$	.

微积分

$$\frac{\mathrm{d}y}{\mathrm{d}x}, \quad \frac{\partial y}{\partial x}, \quad \nabla_{\boldsymbol{x}} y, \quad \int_a^b f(x) \mathrm{d}x.$$

概率与信息论

$$P(\cdot), \quad z \sim P, \quad \mathbb{E}_{x \sim P}[f(x)], \quad \text{Var}(f(x)), \\ H(X), \quad D_{\text{KL}}(P \| Q).$$

第一章 引言

程序不再只执行固定业务规则，而从数据中估计映射 f 。深度学习把 f 参数化为一串可微变换，用优化更新参数。

1.1 日常生活中的机器学习

日常系统把观测 x 映到决策 a 。语音识别、推荐、视觉检测都是在估计

$$a = \hat{f}(x), \quad \hat{f} \in \mathcal{F},$$

其中 $\mathcal{F}$ 由数据而非手写规则确定。经验增加时， $\hat{f}$ 的风险下降；固定规则程序不自动改进。

1.2 机器学习中的关键组件

学习问题写成四元组 $(\mathcal{D}, \mathcal{F}, L, \mathcal{A})$ ：数据、模型类、目标、优化算法。

1.2.1 数据

训练集 $\mathcal{D} = \{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^n$ 提供经验。样本应尽可能独立同分布地来自未知 $P(\mathbf{x}, y)$ 。特征维度、标签类型和 n 决定 $\mathcal{F}$ 能多复杂。

1.2.2 模型

模型是带参数 θ 的映射

$$\hat{y} = f_{\theta}(\mathbf{x}).$$

线性模型对应 $\mathcal{F} = \{\mathbf{x} \mapsto \mathbf{w}^T \mathbf{x} + b\}$ ；深度模型把若干简单映射复合为 $f_L \circ \cdots \circ f_1$ 。

1.2.3 目标函数

目标（损失）把预测与真值变成可比较的标量：

$$L(\boldsymbol{\theta}) = \frac{1}{n} \sum_{i=1}^n \ell(f_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}), y^{(i)}).$$

回归常用平方损失，分类常用交叉熵。经验风险 $L(\boldsymbol{\theta})$ 是对真实风险 $R(\boldsymbol{\theta}) = \mathbb{E}[\ell(f_{\boldsymbol{\theta}}(\mathbf{x}), y)]$ 的近似。

1.2.4 优化算法

训练求解

$$\boldsymbol{\theta}^* \in \underset{\boldsymbol{\theta}}{\operatorname{argmin}} L(\boldsymbol{\theta}).$$

深度学习中 L 通常非凸且无闭式解，故用梯度迭代

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \widehat{\nabla L}(\boldsymbol{\theta}),$$

其中 $\widehat{\nabla L}$ 常由小批量估计。

1.3 各种机器学习问题

按是否提供标签、是否与环境交互，学习问题分成不同的条件设置。

1.3.1 监督学习

每个 $\mathbf{x}$ 配有 y ，目标是估计 $P(y | \mathbf{x})$ 或 $f(\mathbf{x}) \approx y$ 。回归取 $y \in \mathbb{R}$ ，分类取 $y \in \{1, \dots, K\}$ 。

1.3.2 无监督学习

只有 $\{\mathbf{x}^{(i)}\}$ 。任务是估计结构，例如密度 $p(\mathbf{x})$ 、聚类划分、或低维表示 $\mathbf{z} = g(\mathbf{x})$ 。

1.3.3 与环境互动

数据不再一次性给定，而由策略与环境交互产生轨迹

$$(\mathbf{s}_t, a_t, r_t, \mathbf{s}_{t+1})_{t \geq 0}.$$

分布依赖于当前策略，因此 i.i.d. 假设不自动成立。

1.3.4 强化学习

目标是最大化期望回报

$$J(\pi) = \mathbb{E}_{\pi} \left[\sum_{t=0}^T \gamma^t r_t \right], \quad 0 \leq \gamma \leq 1.$$

监督信号是延迟的标量奖励，而不是每步的正确标签。

1.4 起源

可学习机器的思想来自神经科学中的神经元加权求和、统计学中的回归与决策理论、以及控制论中的反馈。感知机给出

$$\hat{y} = \mathbf{1}\{\mathbf{w}^T \mathbf{x} + b > 0\},$$

多层网络则取消线性可分限制。

1.5 深度学习的发展

关键条件是：可微复合模型、反向传播计算 $\nabla_{\theta} L$ 、大规模数据与并行硬件。深度指复合层数 L 增大，使 $\mathcal{F}$ 更丰富。

1.6 深度学习的成功案例

当 x 具有局部相关结构（图像、语音、文本）且 n 足够大时，端到端训练的 f_{θ} 在分类、检测、翻译等风险上低于手工特征加浅层模型。

1.7 特点

深度学习用端到端可微管道代替分阶段特征工程：

$$x \xrightarrow{f_1} h_1 \xrightarrow{f_2} \cdots \xrightarrow{f_L} \hat{y},$$

各 f_{ℓ} 的参数由同一 $L(\theta)$ 的梯度同时更新。

1.8 小结

机器学习估计 f_{θ} ；深度学习把 f_{θ} 写成深层复合，并用 $L(\theta)$ 的梯度迭代求解。

1.9 练习

核验四元组 $(\mathcal{D}, \mathcal{F}, L, \mathcal{A})$ 在回归与分类中如何特化，以及经验风险与真实风险的差别。

第二章 预备知识

本章把后续计算写成同一套对象与映射：数据是张量，前向是线性映射，训练目标是可微标量，自动微分在计算图上实现链式法则，概率把预测写成分布。正文保留原书标题，内容以公式为主。

2.1 数据操作

记 n 阶张量为 $\mathbf{X} \in \mathbb{R}^{d_1 \times \cdots \times d_n}$ 。 $n = 0, 1, 2$ 分别对应标量、向量、矩阵；更高阶用于批量、通道或时间轴。下面先固定形状，再写运算。

2.1.1 入门

形状与元素总数为

$$\text{shape}(\mathbf{X}) = (d_1, \dots, d_n), \quad \text{numel}(\mathbf{X}) = \prod_{i=1}^n d_i.$$

`reshape` 只改变轴的划分，不改变元素值与存储次序。若

$$\prod_{i=1}^n d_i = \prod_{j=1}^m d'_j,$$

则存在把 $\mathbb{R}^{d_1 \times \cdots \times d_n}$ 重新解释为 $\mathbb{R}^{d'_1 \times \cdots \times d'_m}$ 的双射。某个 $d'_k = -1$ 时，由上式唯一确定该轴长度。

```
x = torch.arange(12).reshape(3, 4)
x.shape, x.numel()
```

2.1.2 运算符

形状相同后，二元运算默认逐元素进行。若 $\mathbf{u}, \mathbf{v} \in \mathbb{R}^d$,

$$(f(\mathbf{u}, \mathbf{v}))_i = f(u_i, v_i), \quad i = 1, \dots, d,$$

其中 f 可取 $+$, $-$, $\odot$, $/$, 或幂。沿轴 k 连接要求除第 k 轴外形状一致:

$$\text{cat}(\mathbf{A}, \mathbf{B}; k) \in \mathbb{R}^{d_1 \times \dots \times (d_k^{(A)} + d_k^{(B)}) \times \dots \times d_n}.$$

比较返回 $\{0, 1\}$ 值张量; 求和是

$$\sum(\mathbf{X}) = \sum_{i_1=1}^{d_1} \cdots \sum_{i_n=1}^{d_n} X_{i_1 \dots i_n}.$$

2.1.3 广播机制

形状不同时, 从末轴向前对齐。第 i 轴可运算当且仅当

$$d_i^{(X)} = d_i^{(Y)} \quad \text{或} \quad d_i^{(X)} = 1 \quad \text{或} \quad d_i^{(Y)} = 1.$$

长度为 1 的轴在逻辑上复制到目标长度。于是

$$\mathbb{R}^{3 \times 1} + \mathbb{R}^{1 \times 2} \longrightarrow \mathbb{R}^{3 \times 2}, \quad (A + B)_{ij} = A_{i1} + B_{1j}.$$

2.1.4 索引和切片

对 $\mathbf{X} \in \mathbb{R}^{m \times n}$, 下标从 0 起算。切片 $[a : b]$ 取 $\{a, a + 1, \dots, b - 1\}$, 即半开区间 $[a, b)$, 且 -1 表示最后一个合法下标。多轴独立作用:

$$\mathbf{X}[i, j] = X_{ij}, \quad \mathbf{X}[a : b, :] \in \mathbb{R}^{(b-a) \times n}.$$

2.1.5 节省内存

记 $\text{id}(\cdot)$ 为对象标识。赋值

$$\mathbf{Y} \leftarrow \mathbf{Y} + \mathbf{X}$$

通常分配新存储后再改引用; 切片写回

$$\mathbf{Y}[:,] \leftarrow \mathbf{Y} + \mathbf{X}$$

保持 $\text{id}(\mathbf{Y})$ 不变。参数规模大或同一映射反复求值时, 减少分配等价于减少临时张量个数。

2.1.6 转换为其他 Python 对象

PyTorch 张量与 NumPy 数组可共享同一块内存, 因此对一侧的原地更新会改变另一侧。标量提取是映射

$$\mathbb{R}^1 \longrightarrow \mathbb{R}, \quad \mathbf{x} \mapsto x = \text{item}(\mathbf{x}).$$

2.1.7 小结

数据操作把 $\mathbf{X}$ 的结构写成 $\text{shape}(\mathbf{X})$ ，把组合写成逐元素映射与广播，把局部访问写成索引，把存储控制写成是否保持 $\text{id}(\mathbf{Y})$ 。

2.1.8 练习

原书练习对应三条约束：由 $\text{numel}(\mathbf{X})$ 反推合法 $(d_1, \dots, d_n)$ 、由广播规则写出结果形状、以及判断共享内存是否蕴含 id 相同。

2.2 数据预处理

原始表通常含缺失与非数值字段，不能直接作为上一节中的 $\mathbf{X}$ 。预处理的目标是构造数值张量 $(\mathbf{X}, \mathbf{y})$ 。

2.2.1 读取数据集

设表有 n 行、 p 列。一行是一个样本，一列是一个属性。划分后

$$\mathbf{x}^{(i)} \in \mathbb{R}^{p-1}, \quad y^{(i)} \in \mathbb{R}, \quad i = 1, \dots, n,$$

缺失在 pandas 中记为 NaN。

2.2.2 处理缺失值

对数值列 j ，令观测指标集 $\mathcal{I}_j = \{i : x_{ij} \text{ 非缺失}\}$ 。样本均值

$$\bar{x}_j = \frac{1}{|\mathcal{I}_j|} \sum_{i \in \mathcal{I}_j} x_{ij}$$

用于填充： $i \notin \mathcal{I}_j$ 时令 $x_{ij} \leftarrow \bar{x}_j$ 。类别列取值于有限集 $\mathcal{C}$ 时，独热映射为

$$c \mapsto \mathbf{e}_c \in \{0, 1\}^{|\mathcal{C}|}, \quad (\mathbf{e}_c)_{c'} = [c = c'].$$

缺失可并入一个额外类别，使编码落在 $\mathbb{R}^{|\mathcal{C}|+1}$ 。

2.2.3 转换为张量格式

全部字段数值化且类型一致后，

$$(\mathbf{x}^{(i)}, y^{(i)})_{i=1}^n \longrightarrow \mathbf{X} \in \mathbb{R}^{n \times d}, \mathbf{y} \in \mathbb{R}^n.$$

实现路径为 $\text{DataFrame} \rightarrow \text{ndarray} \rightarrow \text{Tensor}$ 。

2.2.4 小结

预处理不改变学习目标 y ，只把不完整字段映成模型可计算的 $(\mathbf{X}, y)$ 。

2.2.5 练习

删除缺失最多的列对应投影 $\mathbb{R}^d \rightarrow \mathbb{R}^{d-1}$ ；均值填充则保持维度 d 。二者是不同的缺失处理映射。

2.3 线性代数

张量给出对象；线性代数给出映射。从标量升到矩阵，再由点积与乘法得到层间变换，最后用范数把误差压成标量。

2.3.1 标量

标量 $x \in \mathbb{R}$ 满足域运算

$$x + y, \quad xy, \quad x/y \ (y \neq 0), \quad x^p.$$

2.3.2 向量

n 个标量排成列向量

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} \in \mathbb{R}^n.$$

2.3.2.1 长度、维度和形状

$\mathbf{x} \in \mathbb{R}^n$ 表示分量数（向量维度）为 n ，对应一维张量形状 $(n,)$ 。张量的“维数”还可指轴数 k ，即 $\mathbf{X} \in \mathbb{R}^{d_1 \times \cdots \times d_k}$ 的阶。

2.3.3 矩阵

m 个 $\mathbb{R}^n$ 中的行排成

$$\mathbf{A} = \begin{bmatrix} a_{11} & \cdots & a_{1n} \\ \vdots & \ddots & \vdots \\ a_{m1} & \cdots & a_{mn} \end{bmatrix} \in \mathbb{R}^{m \times n}.$$

转置定义为 $(\mathbf{A}^\top)_{ij} = a_{ji}$, 因而

$$\mathbf{A}^\top \in \mathbb{R}^{n \times m}.$$

若 $\mathbf{A} = \mathbf{A}^\top$, 则 $\mathbf{A}$ 为对称矩阵, 此时必有 $m = n$ 。

2.3.4 张量

向量、矩阵分别是 1 阶、2 阶张量。图像可写为 $\mathbf{X} \in \mathbb{R}^{H \times W \times C}$; 加入批量后为 $\mathbb{R}^{B \times H \times W \times C}$ 。

2.3.5 张量算法的基本性质

同形状逐元素运算保持形状。Hadamard 积

$$(\mathbf{A} \odot \mathbf{B})_{ij} = a_{ij}b_{ij}, \quad \mathbf{A}, \mathbf{B} \in \mathbb{R}^{m \times n}$$

不同于下面的矩阵乘法。标量广播为

$$(\alpha + \mathbf{A})_{ij} = \alpha + a_{ij}, \quad \alpha \in \mathbb{R}.$$

2.3.6 降维

对 $\mathbf{A} \in \mathbb{R}^{m \times n}$, 全求和得到标量

$$\sum_{i=1}^m \sum_{j=1}^n a_{ij}.$$

沿轴归约则降低阶:

$$\left(\sum_{i=1}^m \mathbf{A} \right)_j = \sum_{i=1}^m a_{ij} \in \mathbb{R}^n, \quad \left(\sum_{j=1}^n \mathbf{A} \right)_i = \sum_{j=1}^n a_{ij} \in \mathbb{R}^m.$$

均值为总和除以元素个数:

$$\bar{A} = \frac{1}{mn} \sum_{i=1}^m \sum_{j=1}^n a_{ij}.$$

2.3.6.1 非降维求和

`keepdim=True` 把被求和轴保留为长度 1，从而可广播。行归一化

$$\tilde{a}_{ij} = \frac{a_{ij}}{\sum_{k=1}^n a_{ik}} \quad \left(\sum_k a_{ik} \neq 0 \right)$$

满足 $\sum_j \tilde{a}_{ij} = 1$ 。沿行的累积和保持长度：

$$s_{ij} = \sum_{k=1}^j a_{ik}.$$

2.3.7 点积 (Dot Product)

等长向量的点积是加权求和

$$\mathbf{x}^\top \mathbf{y} = \sum_{i=1}^d x_i y_i = \sum_{i=1}^d (\mathbf{x} \odot \mathbf{y})_i.$$

若 $\|\mathbf{x}\|_2 = \|\mathbf{y}\|_2 = 1$ ，则 $\mathbf{x}^\top \mathbf{y} = \cos \theta$ 。

2.3.8 矩阵-向量积

把 $\mathbf{A}$ 的每一行与 $\mathbf{x}$ 做点积：对 $\mathbf{A} \in \mathbb{R}^{m \times n}$ 、 $\mathbf{x} \in \mathbb{R}^n$ ，

$$\mathbf{Ax} = \begin{bmatrix} \mathbf{a}_1^\top \mathbf{x} \\ \vdots \\ \mathbf{a}_m^\top \mathbf{x} \end{bmatrix} \in \mathbb{R}^m.$$

该映射 $f(\mathbf{x}) = \mathbf{Ax}$ 满足

$$f(\alpha \mathbf{x} + \beta \mathbf{z}) = \alpha f(\mathbf{x}) + \beta f(\mathbf{z}),$$

即 $\mathbb{R}^n \rightarrow \mathbb{R}^m$ 的线性变换。

2.3.9 矩阵-矩阵乘法

中间维相等时，矩阵乘法是多个矩阵-向量积并列。若 $\mathbf{A} \in \mathbb{R}^{n \times k}$ 、 $\mathbf{B} \in \mathbb{R}^{k \times m}$ ，则 $\mathbf{C} = \mathbf{AB} \in \mathbb{R}^{n \times m}$ 且

$$c_{ij} = \mathbf{a}_i^\top \mathbf{b}_j = \sum_{\ell=1}^k a_{i\ell} b_{\ell j}.$$

2.3.10 范数

范数 $f: \mathbb{R}^n \rightarrow \mathbb{R}_{\geq 0}$ 满足

$$\begin{aligned} f(\alpha \mathbf{x}) &= |\alpha| f(\mathbf{x}), \\ f(\mathbf{x} + \mathbf{y}) &\leq f(\mathbf{x}) + f(\mathbf{y}), \\ f(\mathbf{x}) &\geq 0, \\ f(\mathbf{x}) = 0 &\iff \mathbf{x} = \mathbf{0}. \end{aligned}$$

L_p 范数为

$$\|\mathbf{x}\|_p = \left(\sum_{i=1}^n |x_i|^p \right)^{1/p}, \quad \|\mathbf{x}\|_1 = \sum_i |x_i|, \quad \|\mathbf{x}\|_2 = \sqrt{\sum_i x_i^2}.$$

矩阵的 Frobenius 范数是把矩阵当作向量后的 L_2 :

$$\|\mathbf{X}\|_F = \sqrt{\sum_{i=1}^m \sum_{j=1}^n x_{ij}^2}.$$

2.3.10.1 范数和目标

“预测接近真值”写成距离最小化。平方误差使用 L_2 范数平方:

$$\min_{\boldsymbol{\theta}} \|\mathbf{y} - f_{\boldsymbol{\theta}}(\mathbf{X})\|_2^2.$$

2.3.11 关于线性代数的更多信息

后续章节主要用到 $\mathbf{A}\mathbf{x}$ 、 $\mathbf{A}\mathbf{B}$ 与 $\|\cdot\|_p$ 。矩阵分解、低秩与张量分解不在本章推导。

2.3.12 小结

线性代数把数据写成张量，把层写成 $\mathbf{A}\mathbf{x}$ 或 $\mathbf{A}\mathbf{B}$ ，把误差写成 $\|\cdot\|$ 。

2.3.13 练习

需要同时核验恒等式与形状，例如 $(\mathbf{A}\mathbf{B})^T = \mathbf{B}^T \mathbf{A}^T$ 蕴含 $\mathbb{R}^{n \times m}$ 与 $\mathbb{R}^{m \times n}$ 的对应。

2.4 微积分

前向已写成映射 f ；训练要知道参数扰动如何改变标量目标。优化关心 $\min_{\boldsymbol{\theta}} L(\boldsymbol{\theta})$ ，泛化关心所得 $\boldsymbol{\theta}$ 在未见样本上的 L 。训练主要用微分。

2.4.1 导数和微分

若极限存在,

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}.$$

一阶展开为

$$f(x + \Delta x) = f(x) + f'(x)\Delta x + o(\Delta x).$$

等价记号

$$f'(x) = y' = \frac{dy}{dx} = D_x f(x).$$

对 $f(x) = 3x^2 - 4x$,

$$f'(x) = 6x - 4, \quad f'(1) = 2,$$

故 $x = 1$ 附近 $\Delta f \approx 2\Delta x$ 。常用法则:

$$\begin{aligned} (Cf)' &= Cf', \\ (f+g)' &= f' + g', \\ (fg)' &= f'g + fg', \\ \left(\frac{f}{g}\right)' &= \frac{f'g - fg'}{g^2} \quad (g \neq 0). \end{aligned}$$

2.4.2 偏导数

多元函数 $y = f(x_1, \dots, x_n)$ 沿第 i 个坐标的变化率把其余变量视为常数:

$$\frac{\partial y}{\partial x_i} = \lim_{h \rightarrow 0} \frac{f(x_1, \dots, x_i + h, \dots, x_n) - f(x_1, \dots, x_i, \dots, x_n)}{h}.$$

等价记号:

$$\frac{\partial y}{\partial x_i} = \frac{\partial f}{\partial x_i} = f_{x_i} = D_{x_i} f.$$

2.4.3 梯度

把全部偏导排成列向量。若 $f: \mathbb{R}^n \rightarrow \mathbb{R}$,

$$\nabla_{\mathbf{x}} f(\mathbf{x}) = \begin{bmatrix} \partial f / \partial x_1 \\ \vdots \\ \partial f / \partial x_n \end{bmatrix}.$$

一阶展开为

$$f(\mathbf{x} + \Delta \mathbf{x}) = f(\mathbf{x}) + \nabla_{\mathbf{x}} f(\mathbf{x})^\top \Delta \mathbf{x} + o(\|\Delta \mathbf{x}\|).$$

原书常用结果：

$$\begin{aligned}\nabla_{\mathbf{x}}(\mathbf{A}\mathbf{x}) &= \mathbf{A}^\top, \\ \nabla_{\mathbf{x}}(\mathbf{x}^\top \mathbf{A}) &= \mathbf{A}, \\ \nabla_{\mathbf{x}}(\mathbf{x}^\top \mathbf{A}\mathbf{x}) &= (\mathbf{A} + \mathbf{A}^\top)\mathbf{x}, \\ \nabla_{\mathbf{x}}\|\mathbf{x}\|_2^2 &= 2\mathbf{x}, \\ \nabla_{\mathbf{X}}\|\mathbf{X}\|_F^2 &= 2\mathbf{X}.\end{aligned}$$

2.4.4 链式法则

复合 $y = f(u)$ 、 $u = g(x)$ 时

$$\frac{dy}{dx} = \frac{dy}{du} \frac{du}{dx}.$$

若 y 依赖 $u_1, \dots, u_m$ ，而每个 u_j 依赖 x_i ，则路径求和：

$$\frac{\partial y}{\partial x_i} = \sum_{j=1}^m \frac{\partial y}{\partial u_j} \frac{\partial u_j}{\partial x_i}.$$

神经网络是复合 $L = f_L \circ \dots \circ f_1$ ，反向传播即沿计算图反向重复该式。

2.4.5 小结

单变量用 $f'(x)$ ，多变量用 $\partial y / \partial x_i$ ，全部偏导组成 $\nabla_{\mathbf{x}} f$ ，复合函数用链式法则把各层 Jacobian 连乘（或按路径求和）。

2.4.6 练习

几何上 $f'(x)$ 是切线斜率；代数上对给定 f 求 f' ，二者应一致。

2.5 自动微分

链式法则给出规则；自动微分对实际执行的程序求数值导数。前向记录计算图 $\mathcal{G}$ ，反向对每个节点应用

$$\frac{\partial L}{\partial u} = \sum_{v: u \rightarrow v} \frac{\partial L}{\partial v} \frac{\partial v}{\partial u}.$$

2.5.1 一个简单的例子

令

$$y = 2\mathbf{x}^\top \mathbf{x} = 2 \sum_i x_i^2.$$

则

$$\frac{\partial y}{\partial x_i} = 4x_i, \quad \nabla_{\mathbf{x}} y = 4\mathbf{x}.$$

```
x = torch.arange(4.0, requires_grad=True)
y = 2 * torch.dot(x, x)
y.backward()
x.grad
```

默认累积：若连续两次反向且不清零，则

$$\nabla_{\mathbf{x}} \leftarrow \nabla_{\mathbf{x}} + 4\mathbf{x}.$$

2.5.2 非标量变量的反向传播

$\mathbf{y} \in \mathbb{R}^m$ 、 $\mathbf{x} \in \mathbb{R}^n$ 时，完整导数是 Jacobian

$$J = \frac{\partial \mathbf{y}}{\partial \mathbf{x}} \in \mathbb{R}^{m \times n}, \quad J_{ij} = \frac{\partial y_i}{\partial x_j}.$$

训练通常计算向量-Jacobian 积 $\mathbf{v}^\top J$ ，而不是显式形成 J 。若 $\mathbf{y} = \mathbf{x} \odot \mathbf{x}$ 且取 $\mathbf{v} = \mathbf{1}$,

$$\nabla_{\mathbf{x}}(\mathbf{1}^\top \mathbf{y}) = \nabla_{\mathbf{x}} \sum_i x_i^2 = 2\mathbf{x}.$$

故 `y.sum().backward()` 与用 $\mathbf{v} = \mathbf{1}$ 调用反向传播等价。

2.5.3 分离计算

设 $\mathbf{y} = \mathbf{x} \odot \mathbf{x}$ ，令 $\mathbf{u} = \text{detach}(\mathbf{y})$ ，即数值相同但切断边 $\mathbf{x} \rightarrow \mathbf{y}$ 。对

$$z = \mathbf{u}^\top \mathbf{x}$$

把 $\mathbf{u}$ 视为常向量，得

$$\nabla_{\mathbf{x}} z = \mathbf{u} = \mathbf{x} \odot \mathbf{x},$$

而完整复合 $z = (\mathbf{x} \odot \mathbf{x})^\top \mathbf{x} = \sum_i x_i^3$ 的梯度本应为 $3\mathbf{x} \odot \mathbf{x}$ 。

2.5.4 Python 控制流的梯度计算

动态图只沿本次前向实际执行的可微路径求导。若该路径上

$$f(a) = ka \quad (k \text{ 依赖本次分支, 但对 } a \text{ 局部为常数}),$$

则

$$\frac{df}{da} = k = \frac{f(a)}{a}, \quad a \neq 0.$$

2.5.5 小结

流程是：对 $\mathbf{x}$ 启用追踪，计算标量 L ，执行

$$L.backward() \implies \mathbf{x}.grad = \nabla_{\mathbf{x}} L.$$

2.5.6 练习

原书练习对应二阶导数、重复反向、非标量输入与控制流。

2.5.7 练习解答

2.5.7.1 1. 为什么二阶导数的开销更大？

一阶梯度 $\nabla L \in \mathbb{R}^n$ 。海森

$$H_{ij} = \frac{\partial^2 L}{\partial x_i \partial x_j}$$

最多含 n^2 个元素，且需在一阶图上再反传一次，故时间与内存都更大。

2.5.7.2 2. 连续运行两次反向传播

默认第一次反传后释放 $\mathcal{G}$ 。保留图后可再求 ∇L ，但梯度累积：

$$\mathbf{g} \leftarrow \mathbf{g} + \nabla_{\mathbf{x}} L.$$

2.5.7.3 3. 将控制流例子中的输入改成向量或矩阵

若输出 $\mathbf{f}(\mathbf{a})$ 非标量，反向目标改为

$$\nabla_{\mathbf{a}}(\mathbf{1}^\top \mathbf{f}(\mathbf{a})) = J^\top \mathbf{1}.$$

对路径 $\mathbf{f}(\mathbf{a}) = k\mathbf{a}$ ，有 $J = kI$ ，因而每个分量的梯度均为 k 。

2.5.7.4 4. 重新设计控制流梯度例子

分段

$$g(x) = \begin{cases} x^2, & x > 0, \\ -x^3, & x \leq 0, \end{cases} \quad g'(x) = \begin{cases} 2x, & x > 0, \\ -3x^2, & x < 0. \end{cases}$$

自动微分对选中分支求导，不对 $[x > 0]$ 这一离散选择求导。

2.6 概率

点估计给出 $\hat{y}$ ；概率把不确定性写成 $P(\cdot \mid \text{观测})$ 。

2.6.1 基本概率论

独立重复抽样下，事件 $\mathcal{A}$ 的经验频率

$$\hat{P}_N(\mathcal{A}) = \frac{1}{N} \sum_{i=1}^N \mathbf{1}\{X_i \in \mathcal{A}\}$$

满足 $\hat{P}_N(\mathcal{A}) \rightarrow P(\mathcal{A})$ （大数定律）。

2.6.1.1 概率论公理

样本空间为 $\mathcal{S}$ ，事件为 $\mathcal{A} \subseteq \mathcal{S}$ 。公理：

$$\begin{aligned} P(\mathcal{A}) &\geq 0, \\ P(\mathcal{S}) &= 1, \\ P\left(\bigcup_{i=1}^{\infty} \mathcal{A}_i\right) &= \sum_{i=1}^{\infty} P(\mathcal{A}_i) \quad (\mathcal{A}_i \text{ 两两不交}). \end{aligned}$$

取 $\mathcal{A}_i = \emptyset$ 得 $P(\emptyset) = 0$ ；互补事件满足 $P(\mathcal{A}^c) = 1 - P(\mathcal{A})$ 。

2.6.1.2 随机变量

随机变量是 $X: \mathcal{S} \rightarrow \mathbb{R}$ 。离散时直接用 $P(X = x)$ ；连续时用密度 p ，

$$P(a \leq X \leq b) = \int_a^b p(x) \mathrm{d}x,$$

因而单点可以有 $P(X = x) = 0$ 。

2.6.2 处理多个随机变量

成对变量由联合分布 $P(A, B)$ 描述。条件化限制到 $\{A = a\}$ ，边际化消去 A 。

2.6.2.1 联合概率

$$P(A = a, B = b) = P(\{A = a\} \cap \{B = b\}) \leq P(A = a).$$

2.6.2.2 条件概率

当 $P(A = a) > 0$,

$$P(B = b | A = a) = \frac{P(A = a, B = b)}{P(A = a)}.$$

乘法法则即 $P(A, B) = P(B | A)P(A)$ 。

2.6.2.3 贝叶斯定理

由 $P(B | A)P(A) = P(A | B)P(B)$ 交换条件方向:

$$P(A | B) = \frac{P(B | A)P(A)}{P(B)}, \quad P(B) > 0.$$

其中 $P(A)$ 为先验, $P(B | A)$ 为似然, $P(A | B)$ 为后验。

2.6.2.4 边际化

分母由全概率公式得到。A 离散时

$$P(B) = \sum_A P(A, B) = \sum_A P(B | A)P(A);$$

连续时把求和换成 $\int p(a, b) da$ 。

2.6.2.5 独立性

$$A \perp B \iff P(A, B) = P(A)P(B) \iff P(A | B) = P(A) \quad (P(B) > 0).$$

条件独立:

$$A \perp B | C \iff P(A, B | C) = P(A | C)P(B | C).$$

2.6.2.6 应用

令 $H \in \{0, 1\}$ 表示感染, $D_1 \in \{0, 1\}$ 表示检测阳性。已知

$$P(H = 1) = 0.0015, \quad P(D_1 = 1 | H = 0) = 0.01, \quad P(D_1 = 1 | H = 1) = 1.$$

边际

$$P(D_1 = 1) = 0.01(1 - 0.0015) + 1 \cdot 0.0015 = 0.011485,$$

后验

$$P(H = 1 | D_1 = 1) = \frac{1 \cdot 0.0015}{0.011485} \approx 0.1306.$$

因此高灵敏度并不蕴含高后验: 先验 $P(H = 1)$ 过小时, 一次阳性仍不足。

2.6.3 期望和方差

离散期望

$$\mathbb{E}[X] = \sum_x xP(X=x), \quad \mathbb{E}_{x \sim P}[f(x)] = \sum_x f(x)P(x).$$

方差

$$\text{Var}(X) = \mathbb{E}[(X - \mathbb{E}[X])^2] = \mathbb{E}[X^2] - \mathbb{E}[X]^2,$$

标准差 $\sqrt{\text{Var}(X)}$ 与 X 同量纲。函数值的离散程度为

$$\text{Var}[f(x)] = \mathbb{E}[(f(x) - \mathbb{E}[f(x)])^2].$$

2.6.4 小结

公理给出 P ，联合/条件/边际给出多变量关系，贝叶斯给出 $P(A | B) \propto P(B | A)P(A)$ ，期望与方差概括位置与分散。

2.6.5 练习

对应 $\hat{P}_N$ 、条件化、贝叶斯更新、 $A \perp B$ 以及 $\mathbb{E}, \text{Var}$ 。

2.7 查阅文档

接口随版本变化，核验对象本身，而不是记忆映射表。

2.7.1 查找模块中的所有函数和类

`dir(module)` 列出属性。名称 `__.` 属语言机制，`.` 通常为内部实现。

2.7.2 查找特定函数和类的用法

`help(object)` 给出文档字符串。Jupyter 中 `?` 查看帮助，`??` 在可用时给出源码。最终以当前安装版本的官方签名为准。

2.7.3 小结

先 `dir` 得到候选集合，再 `help` 读取参数与返回值。

2.7.4 练习

对选定张量函数核对其数学定义与实现签名是否一致。

第三章 线性神经网络

线性模型把输入 $\mathbf{x}$ 映成标量或类别分数，再用平方损失或交叉熵得到可微目标。训练在小批量上用梯度更新参数；分类时在线性输出后再接 softmax。下面按原书顺序写出对象、损失、最优条件和实现所对应的映射。

3.1 线性回归

回归估计 $y \in \mathbb{R}$ 。给定特征 $\mathbf{x} \in \mathbb{R}^d$ ，模型输出一个实数预测 $\hat{y}$ 。

3.1.1 线性回归的基本元素

线性回归假设 $\hat{y}$ 对 $\mathbf{x}$ 仿射，噪声为加性高斯。训练集

$$\mathcal{D} = \{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^n$$

用来确定权重 $\mathbf{w}$ 与偏置 b 。

3.1.1.1 线性模型

房价例子把面积与房龄写成加权和

$$\text{price} = w_{\text{area}} \cdot \text{area} + w_{\text{age}} \cdot \text{age} + b.$$

一般地， d 个特征给出

$$\hat{y} = w_1 x_1 + \cdots + w_d x_d + b = \mathbf{w}^\top \mathbf{x} + b.$$

把 n 个样本排成 $\mathbf{X} \in \mathbb{R}^{n \times d}$ 后，向量形式为

$$\hat{\mathbf{y}} = \mathbf{X}\mathbf{w} + b.$$

3.1.1.2 损失函数

第 i 个样本的平方损失为

$$l^{(i)}(\mathbf{w}, b) = \frac{1}{2}(\hat{y}^{(i)} - y^{(i)})^2.$$

经验风险是样本平均

$$\begin{aligned} L(\mathbf{w}, b) &= \frac{1}{n} \sum_{i=1}^n l^{(i)}(\mathbf{w}, b) \\ &= \frac{1}{n} \sum_{i=1}^n \frac{1}{2} (\mathbf{w}^\top \mathbf{x}^{(i)} + b - y^{(i)})^2. \end{aligned}$$

参数估计定义为

$$\mathbf{w}^*, b^* = \underset{\mathbf{w}, b}{\operatorname{argmin}} L(\mathbf{w}, b).$$

3.1.1.3 解析解

把偏置并入 $\mathbf{X}$ 的全 1 列后，正规方程给出

$$\mathbf{w}^* = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y},$$

成立条件是 $\mathbf{X}^\top \mathbf{X}$ 可逆，即特征线性无关且 n 足够大。

3.1.1.4 随机梯度下降

小批量 $\mathcal{B}$ 上的更新为

$$(\mathbf{w}, b) \leftarrow (\mathbf{w}, b) - \frac{\eta}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \partial_{(\mathbf{w}, b)} l^{(i)}(\mathbf{w}, b).$$

平方损失的显式梯度是

$$\begin{aligned} \mathbf{w} &\leftarrow \mathbf{w} - \frac{\eta}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \mathbf{x}^{(i)} (\mathbf{w}^\top \mathbf{x}^{(i)} + b - y^{(i)}), \\ b &\leftarrow b - \frac{\eta}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} (\mathbf{w}^\top \mathbf{x}^{(i)} + b - y^{(i)}). \end{aligned}$$

3.1.1.5 用模型进行预测

学得 $(\hat{\mathbf{w}}, \hat{b})$ 后，新样本的预测是仿射映射

$$\hat{y} = \hat{\mathbf{w}}^\top \mathbf{x} + \hat{b}.$$

3.1.2 矢量化加速

对 $\mathbf{a}, \mathbf{b} \in \mathbb{R}^m$, 逐元素求和应写成一次向量加法

$$\mathbf{c} = \mathbf{a} + \mathbf{b}, \quad c_i = a_i + b_i,$$

而不是 m 次标量循环。小批量上的 $\mathbf{X}\mathbf{w}$ 同样一次完成, 复杂度由线性代数库承担。

3.1.3 正态分布与平方损失

高斯密度为

$$p(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{1}{2\sigma^2}(x - \mu)^2\right).$$

观测模型

$$y = \mathbf{w}^\top \mathbf{x} + b + \epsilon$$

在 $\epsilon \sim \mathcal{N}(0, \sigma^2)$ 下给出

$$P(y | \mathbf{x}) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{1}{2\sigma^2}(y - \mathbf{w}^\top \mathbf{x} - b)^2\right).$$

独立样本的似然是

$$P(\mathbf{y} | \mathbf{X}) = \prod_{i=1}^n p(y^{(i)} | \mathbf{x}^{(i)}),$$

负对数似然为

$$\begin{aligned} -\log P(\mathbf{y} | \mathbf{X}) &= \sum_{i=1}^n \frac{1}{2} \log(2\pi\sigma^2) \\ &\quad + \frac{1}{2\sigma^2} (y^{(i)} - \mathbf{w}^\top \mathbf{x}^{(i)} - b)^2. \end{aligned}$$

σ 固定时, 极大似然与最小化 $L(\mathbf{w}, b)$ 等价。

3.1.4 从线性回归到深度网络

同一仿射映射可看成只有一层计算单元的网络: 输入给定, 输出由 $\mathbf{w}, b$ 计算。

3.1.4.1 神经网络图

输入层提供 $x_1, \dots, x_d$, 输出层一个单元

$$o_1 = \mathbf{w}^\top \mathbf{x} + b.$$

每个输入都连到该单元, 参数个数为 $d + 1$ 。

3.1.4.2 生物学

树突接收 x_i ，突触权重为 w_i ，胞体做加权和后再经激活

$$o = \phi(\mathbf{w}^\top \mathbf{x} + b).$$

线性回归对应 ϕ 为恒等映射。

3.1.5 小结

对象是仿射预测 $\hat{y} = \mathbf{w}^\top \mathbf{x} + b$ ，目标是 $L(\mathbf{w}, b)$ ，高斯噪声下它等于负对数似然；可逆时用正规方程，否则用小批量梯度。

3.1.6 练习

核验 $\sum_i (x_i - b)^2$ 对 b 的驻点、把 b 并入 $\mathbf{X}$ 后正规方程是否仍成立，以及 $\mathbf{X}^\top \mathbf{X}$ 奇异时梯度法与解析解的关系。

3.2 线性回归的从零开始实现

实现对应四个映射：采样数据、按批读取、仿射预测、平方损失上的小批量更新。

3.2.1 生成数据集

真值为

$$\mathbf{y} = \mathbf{X}\mathbf{w} + b + \boldsymbol{\epsilon},$$

其中 $\boldsymbol{\epsilon}$ 的分量独立同分布。合成数据时 $\mathbf{w}, b$ 已知，用来对照估计。

3.2.2 读取数据集

一次迭代取出小批量

$$(\mathbf{X}_B, \mathbf{y}_B), \quad |B| = B.$$

遍历前打乱下标，使各批近似无偏。

3.2.3 初始化模型参数

$$\mathbf{w} \sim \mathcal{N}(\mathbf{0}, 0.01^2 \mathbf{I}), \quad b = 0,$$

并对其后每次损失关于 $(\mathbf{w}, b)$ 求梯度。

3.2.4 定义模型

前向是矩阵-向量积加偏置广播

$$\hat{\mathbf{y}} = \mathbf{X}\mathbf{w} + b.$$

3.2.5 定义损失函数

批量平方损失与上一节一致，实现时把 $\mathbf{y}$ 的形状对齐到 $\hat{\mathbf{y}}$ ：

$$l = \frac{1}{2} \|\hat{\mathbf{y}} - \mathbf{y}\|_2^2 \quad (\text{再按约定取平均或求和}).$$

3.2.6 定义优化算法

一步小批量梯度下降

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \frac{\eta}{|\mathcal{B}|} \nabla_{\boldsymbol{\theta}} \sum_{i \in \mathcal{B}} l^{(i)}, \quad \boldsymbol{\theta} = (\mathbf{w}, b).$$

用 $|\mathcal{B}|$ 规范化后，步长不随批量大小线性膨胀。

3.2.7 训练

每个周期对全部小批量执行

$$\begin{aligned} \hat{\mathbf{y}} &\leftarrow \mathbf{X}_{\mathcal{B}}\mathbf{w} + b, \\ g &\leftarrow \partial_{(\mathbf{w}, b)} \frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} l^{(i)}, \\ (\mathbf{w}, b) &\leftarrow (\mathbf{w}, b) - \eta g. \end{aligned}$$

3.2.8 小结

从零实现把训练写成张量上的 $\hat{\mathbf{y}} = \mathbf{X}\mathbf{w} + b$ 与 $(\mathbf{w}, b) \leftarrow (\mathbf{w}, b) - \eta g$ ，不引入层对象。

3.2.9 练习

核验 $\mathbf{w} = \mathbf{0}$ 时梯度是否仍指向残差方向、二阶导数在自动微分中的形状，以及 n 不能整除 $|\mathcal{B}|$ 时最后一批的基数。

3.3 线性回归的简洁实现

同一映射改由层、损失类与优化器组合，前向与更新规则不变。

3.3.1 生成数据集

仍用 $\mathbf{y} = \mathbf{X}\mathbf{w} + b + \epsilon$ 生成 $(\mathbf{X}, \mathbf{y})$ 。

3.3.2 读取数据集

数据迭代器实现

$$(\mathbf{X}, \mathbf{y}) \mapsto \{(\mathbf{X}_B, \mathbf{y}_B)\}.$$

训练模式下每个周期打乱样本次序。

3.3.3 定义模型

全连接层

$$f(\mathbf{X}) = \mathbf{X}\mathbf{W} + \mathbf{b}, \quad \mathbf{W} \in \mathbb{R}^{d \times 1}$$

作为唯一计算层；顺序容器只含这一层。

3.3.4 初始化模型参数

$$W_{ij} \sim \mathcal{N}(0, 0.01^2), \quad b = 0.$$

3.3.5 定义损失函数

均方误差是平方 L_2 的样本平均

$$L = \frac{1}{n} \sum_{i=1}^n \frac{1}{2} (\hat{y}^{(i)} - y^{(i)})^2.$$

3.3.6 定义优化算法

优化器在参数集合上执行

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \nabla_{\boldsymbol{\theta}} L_B.$$

3.3.7 训练

每个小批量依次：前向 L_B 、反向 $\nabla_{\theta} L_B$ 、参数更新。周期结束时用 L 监控拟合。

3.3.8 小结

简洁实现把 $\mathbf{X} \mapsto \mathbf{XW} + \mathbf{b}$ 、平方损失与小批量梯度封装成可组合对象，数学映射与从零实现相同。

3.3.9 练习

核验分段损失

$$l(y, y') = \begin{cases} |y - y'| - \frac{\sigma}{2}, & |y - y'| > \sigma, \\ \frac{1}{2\sigma}(y - y')^2, & \text{其它情况} \end{cases}$$

在 $|y - y'| = \sigma$ 处的连续性与可导性。

3.4 softmax 回归

分类不估计“多少”，而估计“哪一类”。线性层输出 q 个分数，再用 softmax 变成单纯形上的概率。

3.4.1 分类问题

$q = 3$ 时标签用独热向量

$$\mathbf{y} \in \{(1, 0, 0), (0, 1, 0), (0, 0, 1)\}.$$

一般 $y \in \{1, \dots, q\}$ 对应 $\mathbf{y} = \mathbf{e}_y \in \{0, 1\}^q$ 。

3.4.2 网络架构

四个特征、三个类别时，线性分数为

$$o_1 = x_1 w_{11} + x_2 w_{12} + x_3 w_{13} + x_4 w_{14} + b_1,$$

$$o_2 = x_1 w_{21} + x_2 w_{22} + x_3 w_{23} + x_4 w_{24} + b_2,$$

$$o_3 = x_1 w_{31} + x_2 w_{32} + x_3 w_{33} + x_4 w_{34} + b_3.$$

矩阵形式 $\mathbf{o} = \mathbf{W}\mathbf{x} + \mathbf{b}$, $\mathbf{W} \in \mathbb{R}^{q \times d}$ 。

3.4.3 全连接层的参数开销

d 入 q 出的全连接层参数个数为

$$\#(\mathbf{W}, \mathbf{b}) = dq + q = \mathcal{O}(dq).$$

3.4.4 softmax 运算

把分数映到概率单纯形

$$\hat{\mathbf{y}} = \text{softmax}(\mathbf{o}), \quad \hat{y}_j = \frac{\exp(o_j)}{\sum_k \exp(o_k)}.$$

指数严格递增，故

$$\operatorname{argmax}_j \hat{y}_j = \operatorname{argmax}_j o_j.$$

3.4.5 小批量样本的矢量化

$\mathbf{X} \in \mathbb{R}^{n \times d}$ 时

$$\begin{aligned} \mathbf{O} &= \mathbf{XW} + \mathbf{b}, \\ \hat{\mathbf{Y}} &= \text{softmax}(\mathbf{O}), \end{aligned}$$

其中 softmax 沿类别轴作用。

3.4.6 损失函数

仍由最大似然导出：独立样本下最小化 $-\log P(\mathbf{Y} \mid \mathbf{X})$ 。

3.4.6.1 对数似然

$$P(\mathbf{Y} \mid \mathbf{X}) = \prod_{i=1}^n P(\mathbf{y}^{(i)} \mid \mathbf{x}^{(i)}).$$

负对数似然分解为交叉熵之和

$$-\log P(\mathbf{Y} \mid \mathbf{X}) = \sum_{i=1}^n -\log P(\mathbf{y}^{(i)} \mid \mathbf{x}^{(i)}) = \sum_{i=1}^n l(\mathbf{y}^{(i)}, \hat{\mathbf{y}}^{(i)}),$$

其中

$$l(\mathbf{y}, \hat{\mathbf{y}}) = -\sum_{j=1}^q y_j \log \hat{y}_j.$$

独热标签时 $l = -\log \hat{y}_{y^*}$ 。

3.4.6.2 softmax 及其导数

把 $\hat{y}_j$ 代入交叉熵:

$$\begin{aligned} l(\mathbf{y}, \hat{\mathbf{y}}) &= - \sum_{j=1}^q y_j \log \frac{\exp(o_j)}{\sum_{k=1}^q \exp(o_k)} \\ &= \sum_{j=1}^q y_j \log \sum_{k=1}^q \exp(o_k) - \sum_{j=1}^q y_j o_j \\ &= \log \sum_{k=1}^q \exp(o_k) - \sum_{j=1}^q y_j o_j. \end{aligned}$$

对分数求导得到

$$\begin{aligned} \partial_{o_j} l(\mathbf{y}, \hat{\mathbf{y}}) &= \frac{\exp(o_j)}{\sum_{k=1}^q \exp(o_k)} - y_j \\ &= \text{softmax}(\mathbf{o})_j - y_j. \end{aligned}$$

因此反向传播中, $\nabla_{\mathbf{o}} l = \hat{\mathbf{y}} - \mathbf{y}$ 。

3.4.6.3 交叉熵损失

若 $\mathbf{y}$ 本身已是分布而非独热, 同一式

$$l(\mathbf{y}, \hat{\mathbf{y}}) = - \sum_{j=1}^q y_j \log \hat{y}_j$$

仍是交叉熵, 等于在标签分布下对 $-\log \hat{y}_j$ 的期望。

3.4.7 信息论基础

交叉熵同时度量编码长度与似然。

3.4.7.1 熵

分布 P 的熵

$$H[P] = \sum_j -P(j) \log P(j)$$

是按 P 编码时的最小期望码长。

3.4.7.2 信息量

事件 j 的自信息为

$$I(j) = -\log P(j) = \log \frac{1}{P(j)}.$$

$P(j)$ 越小, $I(j)$ 越大。熵是自信息的期望。

3.4.7.3 重新审视交叉熵

从 P 到 Q 的交叉熵

$$H(P, Q) = \sum_j -P(j) \log Q(j) = H(P) + D_{\text{KL}}(P \| Q)$$

在 $Q = P$ 时取最小 $H(P)$ 。分类目标等价于极大似然，也等价于降低用 $\hat{\mathbf{y}}$ 编码真实标签所需的惊异。

3.4.8 模型预测和评估

预测类别

$$\hat{y} = \operatorname{argmax}_j \hat{y}_j.$$

精度是正确个数与总数之比

$$\text{acc} = \frac{1}{n} \sum_{i=1}^n \mathbf{1}\{\hat{y}^{(i)} = y^{(i)}\}.$$

精度不可导，训练仍最小化交叉熵。

3.4.9 小结

softmax 把 $\mathbf{o}$ 映到概率， $\nabla_{\mathbf{o}} l = \hat{\mathbf{y}} - \mathbf{y}$ 连接交叉熵与线性层；精度在单纯形顶点上评估 argmax 。

3.4.10 练习

核验交叉熵对 $\mathbf{o}$ 的 Hessian 与 $\operatorname{softmax}(\mathbf{o})$ 的协方差是否一致，以及均匀分布 $(\frac{1}{3}, \frac{1}{3}, \frac{1}{3})$ 下二元码的冗余。

3.5 图像分类数据集

Fashion-MNIST 提供 $q = 10$ 的服装灰度图，后续 softmax 实验都在该 $\mathcal{D}$ 上评估。

3.5.1 读取数据集

训练集 $n_{\text{train}} = 60000$ ，测试集 $n_{\text{test}} = 10000$ ，每类分别 6000 与 1000 张。单张图

$$\mathbf{X} \in \mathbb{R}^{28 \times 28}, \quad C = 1,$$

形状记为 $h \times w = 28 \times 28$ 。测试集不参与参数更新。

3.5.2 读取小批量

迭代器输出

$$\mathbf{X} \in \mathbb{R}^{|\mathcal{B}| \times 1 \times 28 \times 28}, \quad \mathbf{y} \in \{1, \dots, 10\}^{|\mathcal{B}|}.$$

训练时打乱，测试时按序。

3.5.3 整合所有组件

读取映射把 (批量大小, 可选边长) 映到训练与测试迭代器。可选边长把 28×28 双线性映到指定 (h', w') 。

3.5.4 小结

图像对象是 $h \times w$ 灰度阵，批量读取把它们变成可并行的 $(\mathbf{X}, \mathbf{y})$ 。

3.5.5 练习

核验 $|\mathcal{B}| \rightarrow 1$ 时 I/O 相对计算的占比，以及迭代器是否成为训练的时间瓶颈。

3.6 softmax 回归的从零开始实现

把 28×28 展平为 $\mathbb{R}^{784}$ ，线性层输出 10 维，再接 softmax 与交叉熵。

3.6.1 初始化模型参数

$$\mathbf{W} \in \mathbb{R}^{784 \times 10}, \quad \mathbf{b} \in \mathbb{R}^{1 \times 10},$$

$W_{ij} \sim \mathcal{N}(0, \sigma^2)$, $b_j = 0$ 。展平是

$$\mathbb{R}^{28 \times 28} \longrightarrow \mathbb{R}^{784}.$$

3.6.2 定义 softmax 操作

对矩阵按行归一化

$$\text{softmax}(\mathbf{X})_{ij} = \frac{\exp(X_{ij})}{\sum_k \exp(X_{ik})}.$$

3.6.3 定义模型

$$\hat{\mathbf{Y}} = \text{softmax}(\mathbf{X}\mathbf{W} + \mathbf{b}),$$

其中 $\mathbf{X}$ 已展平为 $|\mathcal{B}| \times 784$ 。

3.6.4 定义损失函数

独热选取使

$$l(\mathbf{y}, \hat{\mathbf{Y}}) = - \sum_{i=1}^{|\mathcal{B}|} \log \hat{Y}_{i, y^{(i)}}.$$

3.6.5 分类精度

$$\text{acc}(\hat{\mathbf{Y}}, \mathbf{y}) = \frac{1}{|\mathcal{B}|} \sum_i \mathbf{1}\{\arg\max_j \hat{Y}_{ij} = y^{(i)}\}.$$

3.6.6 训练

与线性回归同一循环：前向 $\hat{\mathbf{Y}}$ 、交叉熵、反向、小批量更新。每个周期在测试集上报告 acc。

3.6.7 预测

对新图取

$$\hat{y} = \arg\max_j \text{softmax}(\mathbf{x}^\top \mathbf{W} + \mathbf{b})_j.$$

3.6.8 小结

从零实现的核心是 $\hat{\mathbf{Y}} = \text{softmax}(\mathbf{X}\mathbf{W} + \mathbf{b})$ 与 $l = - \sum \log \hat{y}_y$ ，训练循环与线性回归同构。

3.6.9 练习

核验 $\exp(o_j)$ 在 o_j 很大时的溢出、 $\log \hat{y}_j$ 在 $\hat{y}_j \rightarrow 0$ 时的定义域，以及 $\arg\max$ 是否总是决策最优。

3.7 softmax 回归的简洁实现

同一分类映射改由全连接层加交叉熵实现；数值上把 softmax 与 log 合并。

3.7.1 初始化模型参数

顺序模型中一层 $\mathbb{R}^{784} \rightarrow \mathbb{R}^{10}$,

$$W_{ij} \sim \mathcal{N}(0, 0.01^2), \quad b_j = 0.$$

3.7.2 重新审视 Softmax 的实现

平移不变性给出数值稳定形式

$$\begin{aligned} \hat{y}_j &= \frac{\exp(o_j - \max_k o_k) \exp(\max_k o_k)}{\sum_k \exp(o_k - \max_k o_k) \exp(\max_k o_k)} \\ &= \frac{\exp(o_j - \max_k o_k)}{\sum_k \exp(o_k - \max_k o_k)}. \end{aligned}$$

对数概率为

$$\begin{aligned} \log \hat{y}_j &= \log \frac{\exp(o_j - \max_k o_k)}{\sum_k \exp(o_k - \max_k o_k)} \\ &= o_j - \max_k o_k - \log \sum_k \exp(o_k - \max_k o_k). \end{aligned}$$

交叉熵直接用 $\log \hat{y}_{y^*}$ ，避免先指数再取对数。

3.7.3 优化算法

小批量随机梯度

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \nabla_{\boldsymbol{\theta}} L_{\mathcal{B}}, \quad \eta = 0.1.$$

3.7.4 训练

调用与从零实现相同的周期循环；框架在交叉熵内部完成稳定的 $\log \text{softmax}$ 。

3.7.5 小结

简洁路径仍是 $\mathbf{O} = \mathbf{XW} + \mathbf{b}$ 与交叉熵，差别在于用 $\log \hat{y}_j = o_j - \max o - \text{LSE}(\mathbf{o} - \max o)$ 保证数值稳定。

3.7.6 练习

核验增大周期后测试精度下降是否对应过拟合，以及 $|\mathcal{B}|, \eta$ 对 $\nabla L_{\mathcal{B}}$ 方差的影响。

第四章 多层感知机

线性层只能表示仿射关系。在层间插入非线性 σ 后，复合映射可以逼近更一般的 f 。训练仍最小化可微损失，但深度带来梯度消失或爆炸，并用权重衰减、暂退法控制容量。分布一旦偏离训练 P ，经验风险不再自动逼近真实风险。

4.1 多层感知机

softmax 回归是一次仿射再归一化。多层感知机在仿射之间加入隐藏层与逐元素非线性。

4.1.1 隐藏层

仿射 $\mathbf{x} \mapsto \mathbf{W}\mathbf{x} + \mathbf{b}$ 对每个特征是单调的。隐藏层用来表示特征之间的交互。

4.1.1.1 线性模型可能会出错

权重为正时， x_j 增大则线性分数增大。许多关系非单调，例如体温与风险在正常值两侧反向变化，单次仿射无法同时拟合两支。

4.1.1.2 在网络中加入隐藏层

$L - 1$ 个隐藏层给出表示，最后一层做线性预测。一层隐藏、宽度 h 的 MLP 为

$$\begin{aligned}\mathbf{h} &= \sigma(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)}), \\ \mathbf{o} &= \mathbf{W}^{(2)}\mathbf{h} + \mathbf{b}^{(2)},\end{aligned}$$

两层都全连接。输入层不计计算，故该网络层数为 2。

4.1.1.3 从线性到非线性

若隐层不用 σ ，则

$$\mathbf{H} = \mathbf{X}\mathbf{W}^{(1)} + \mathbf{b}^{(1)},$$

$$\mathbf{O} = \mathbf{H}\mathbf{W}^{(2)} + \mathbf{b}^{(2)},$$

复合后仍是仿射

$$\begin{aligned}\mathbf{O} &= (\mathbf{X}\mathbf{W}^{(1)} + \mathbf{b}^{(1)})\mathbf{W}^{(2)} + \mathbf{b}^{(2)} \\ &= \mathbf{X}\mathbf{W}^{(1)}\mathbf{W}^{(2)} + \mathbf{b}^{(1)}\mathbf{W}^{(2)} + \mathbf{b}^{(2)} \\ &= \mathbf{X}\mathbf{W} + \mathbf{b}.\end{aligned}$$

插入逐元素 σ 后

$$\begin{aligned}\mathbf{H} &= \sigma(\mathbf{X}\mathbf{W}^{(1)} + \mathbf{b}^{(1)}), \\ \mathbf{O} &= \mathbf{H}\mathbf{W}^{(2)} + \mathbf{b}^{(2)}.\end{aligned}$$

此时一般不能再合并为单次仿射。

4.1.1.4 通用近似定理

单隐层、宽度足够且 σ 非多项式时，MLP 可在紧集上一致逼近连续函数。可表示不等于可学习；更深的复合往往用更少的单元逼近同一类 f 。

4.1.2 激活函数

激活 $\sigma: \mathbb{R} \rightarrow \mathbb{R}$ 是可微（或几乎处处可微）的逐元素非线性，作用在仿射输出上。

4.1.2.1 ReLU 函数

$$\text{ReLU}(x) = \max(x, 0).$$

其亚梯度在 $x \neq 0$ 为 $\mathbf{1}\{x > 0\}$ 。带泄露参数 α 的变体

$$\text{pReLU}(x) = \max(0, x) + \alpha \min(0, x).$$

4.1.2.2 sigmoid 函数

$$\text{sigmoid}(x) = \frac{1}{1 + \exp(-x)}.$$

导数为

$$\begin{aligned}\frac{\mathrm{d}}{\mathrm{d}x} \text{sigmoid}(x) &= \frac{\exp(-x)}{(1 + \exp(-x))^2} \\ &= \text{sigmoid}(x)(1 - \text{sigmoid}(x)).\end{aligned}$$

$|x|$ 大时导数趋于 0。

4.1.2.3 tanh 函数

$$\tanh(x) = \frac{1 - \exp(-2x)}{1 + \exp(-2x)}.$$

导数

$$\frac{d}{dx} \tanh(x) = 1 - \tanh^2(x).$$

奇函数且值域 $(-1, 1)$ 。恒等式 $\tanh(x) + 1 = 2 \operatorname{sigmoid}(2x)$ 。

4.1.3 小结

MLP 的定义是 $\mathbf{H} = \sigma(\mathbf{X}\mathbf{W}^{(1)} + \mathbf{b}^{(1)})$ 再接线性输出；无 σ 则退回 $\mathbf{O} = \mathbf{X}\mathbf{W} + \mathbf{b}$ 。常用 σ 为 ReLU、sigmoid、tanh。

4.1.4 练习

核验 pReLU 的导数、仅用 ReLU 的网络是否为连续分段线性，以及 $\tanh(x) + 1 = 2 \operatorname{sigmoid}(2x)$ 。

4.2 多层感知机的从零开始实现

在 Fashion-MNIST 上实现单隐层 MLP：展平后 $784 \rightarrow 256 \rightarrow 10$ 。

4.2.1 初始化模型参数

$$\mathbf{W}^{(1)} \in \mathbb{R}^{784 \times 256}, \quad \mathbf{b}^{(1)} \in \mathbb{R}^{256}, \quad \mathbf{W}^{(2)} \in \mathbb{R}^{256 \times 10}, \quad \mathbf{b}^{(2)} \in \mathbb{R}^{10}.$$

每层单独存储，以便 $\partial J / \partial \mathbf{W}^{(\ell)}$ 写回。

4.2.2 激活函数

逐元素

$$\sigma(\mathbf{Z}) = \operatorname{ReLU}(\mathbf{Z}) = \max(\mathbf{Z}, \mathbf{0}).$$

4.2.3 模型

展平后

$$\begin{aligned} \mathbf{H} &= \text{ReLU}(\mathbf{X}\mathbf{W}^{(1)} + \mathbf{b}^{(1)}), \\ \mathbf{O} &= \mathbf{H}\mathbf{W}^{(2)} + \mathbf{b}^{(2)}. \end{aligned}$$

4.2.4 损失函数

对 $\mathbf{O}$ 用交叉熵（内含稳定 softmax）

$$L = \frac{1}{|\mathcal{B}|} \sum_i l(\mathbf{y}^{(i)}, \text{softmax}(\mathbf{o}^{(i)})).$$

4.2.5 训练

与 softmax 回归同一循环，只是 f_{θ} 换成上面的两层映射；典型设置周期 10、 $\eta = 0.1$ 。

4.2.6 小结

从零实现把 MLP 写成两组 $(\mathbf{W}^{(\ell)}, \mathbf{b}^{(\ell)})$ 与一次 ReLU；层数增多时参数命名与梯度缓冲会线性增加。

4.2.7 练习

核验隐藏宽度、层数与 η 对训练损失和测试精度的联合影响，以及超参数空间为何使网格搜索代价上升。

4.3 多层感知机的简洁实现

用顺序容器堆叠线性层与 ReLU，训练循环不变。

4.3.1 模型

$$\mathbf{X} \xrightarrow{\text{Linear}(784,256)} \text{ReLU} \xrightarrow{\text{Linear}(256,10)} \mathbf{O}.$$

损失与优化器与 softmax 简洁实现相同。

4.3.2 小结

简洁实现与 softmax 回归的差别只是中间多了 $\mathbf{H} = \text{ReLU}(\mathbf{X}\mathbf{W}^{(1)} + \mathbf{b}^{(1)})$ 。

4.3.3 练习

核验隐层数、 σ 的选取以及 $\mathbf{W}$ 的初值分布对同一分类损失的影响。

4.4 模型选择、欠拟合和过拟合

目标是降低来自同一 P 的新样本上的风险，而不是只把训练集误差打到零。

4.4.1 训练误差和泛化误差

训练误差 R_{emp} 在 $\mathcal{D}_{\text{train}}$ 上计算；泛化误差是

$$R(f) = \mathbb{E}_{(\mathbf{x}, y) \sim P} [\ell(f(\mathbf{x}), y)],$$

只能用独立测试集估计。

4.4.1.1 统计学习理论

监督学习默认独立同分布： $\mathcal{D}_{\text{train}}$ 与测试样本均来自同一 P ，且样本间无记忆。在该条件下，经验风险以可控制的速率逼近 $R(f)$ ；函数类越丰富，逼近越慢。

4.4.1.2 模型复杂性

参数多、取值范围大或需要更多更新步的模型更复杂。简单模型加大数据时 $R_{\text{emp}} \approx R$ ；复杂模型加小样本时 R_{emp} 下降而 R 上升。

4.4.2 模型选择

在候选 $\mathcal{F}_1, \dots, \mathcal{F}_m$ （含超参数）中选一个，依据不能是测试集。

4.4.2.1 验证集

数据划分为训练、验证、测试。超参数只在验证集上比较；测试集保留给最终估计 R 。反复使用验证集等于间接拟合验证分布。

4.4.2.2 K 折交叉验证

把训练集分成 K 个不交子集 $\mathcal{I}_1, \dots, \mathcal{I}_K$ 。第 k 折在 $\bigcup_{j \neq k} \mathcal{I}_j$ 上训练，在 $\mathcal{I}_k$ 上验证，再平均 K 次误差。

4.4.3 欠拟合还是过拟合？

训练与验证误差都高且接近，是欠拟合： $\mathcal{F}$ 不足以表示目标。训练误差明显低于验证误差，是过拟合：容量相对 n 过大。

4.4.3.1 模型复杂性

多项式

$$\hat{y} = \sum_{i=0}^d x^i w_i$$

的阶数 d 直接控制容量： d 过小欠拟合， d 过大过拟合。

4.4.3.2 数据集大小

n 越小越容易过拟合。固定 P 时，增大 n 通常降低 R ；数据足够才支撑更复杂的 $\mathcal{F}$ 。

4.4.4 多项式回归

用已知阶数的多项式生成数据，再比较不同 d 的拟合。

4.4.4.1 生成数据集

$$y = 5 + 1.2x - 3.4 \frac{x^2}{2!} + 5.6 \frac{x^3}{3!} + \epsilon, \quad \epsilon \sim \mathcal{N}(0, 0.1^2).$$

特征常取 $x^i/i!$ 以平衡各阶尺度。

4.4.4.2 对模型进行训练和测试

在训练集上最小化平方损失，在测试集上估计 R 。同一优化循环，只是 $\mathbf{x}$ 换成多项式特征。

4.4.4.3 三阶多项式函数拟合 (正常)

取 $d = 3$ 与数据生成同阶。训练损失与测试损失同时下降，估计接近

$$\mathbf{w} \approx (5, 1.2, -3.4, 5.6).$$

4.4.4.4 线性函数拟合 (欠拟合)

$d = 1$ 只能表示仿射。非线性真值下训练损失停在高位，验证损失同样高。

4.4.4.5 高阶多项式函数拟合 (过拟合)

d 过大时高阶系数缺乏约束，拟合噪声。训练损失可很低，测试损失仍高。

4.4.5 小结

比较的是 R_{emp} 与验证估计的 R ：二者都高为欠拟合，差距大为过拟合。用验证集或 K 折选择 d 一类复杂度，而不是把训练损失当作泛化。

4.4.6 练习

核验多项式正规方程是否有闭式解、训练损失随 d 降到 0 所需阶数，以及去掉 $1/i!$ 缩放后各列条件数的变化。

4.5 权重衰减

在不能增加 n 时，用参数范数惩罚降低有效容量。

4.5.1 范数与权重衰减

无惩罚的平方损失为

$$L(\mathbf{w}, b) = \frac{1}{n} \sum_{i=1}^n \frac{1}{2} (\mathbf{w}^\top \mathbf{x}^{(i)} + b - y^{(i)})^2.$$

L_2 正则目标

$$L(\mathbf{w}, b) + \frac{\lambda}{2} \|\mathbf{w}\|^2$$

对应高斯先验。梯度步把惩罚写成衰减

$$\mathbf{w} \leftarrow (1 - \eta\lambda)\mathbf{w} - \frac{\eta}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \mathbf{x}^{(i)} (\mathbf{w}^\top \mathbf{x}^{(i)} + b - y^{(i)}).$$

$\lambda = 0$ 退回普通 SGD； b 通常不惩罚。

4.5.2 高维线性回归

合成

$$y = 0.05 + \sum_{i=1}^d 0.01 x_i + \epsilon, \quad \epsilon \sim \mathcal{N}(0, 0.01^2),$$

取 d 大于 n 时，无正则的最小二乘会过拟合。

4.5.3 从零开始实现

把 $\frac{\lambda}{2} \|\mathbf{w}\|_2^2$ 加进批量损失，再用原有线性模型与平方损失。

4.5.3.1 初始化模型参数

与线性回归相同： $\mathbf{w}$ 小高斯， $b = 0$ 。

4.5.3.2 定义 L_2 范数惩罚

$$\frac{\lambda}{2} \|\mathbf{w}\|_2^2 = \frac{\lambda}{2} \sum_j w_j^2.$$

4.5.3.3 定义训练代码实现

批量目标

$$\frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \frac{1}{2} (\hat{y}^{(i)} - y^{(i)})^2 + \frac{\lambda}{2} \|\mathbf{w}\|_2^2,$$

梯度同时含数据项与 $\lambda \mathbf{w}$ 。

4.5.3.4 忽略正则化直接训练

$\lambda = 0$ 时训练误差下降、测试误差不降，对应过拟合。

4.5.3.5 使用权重衰减

$\lambda > 0$ 时训练误差上升、测试误差下降，有效范数 $\|\mathbf{w}\|$ 被压小。

4.5.4 简洁实现

优化器在每步对选定参数做

$$\mathbf{w} \leftarrow (1 - \eta\lambda)\mathbf{w} - \eta \nabla_{\mathbf{w}} L_{\mathcal{B}},$$

可只衰减 $\mathbf{w}$ 而不衰减 b 。与把惩罚写入损失在一阶上等价。

4.5.5 小结

权重衰减把目标写成 $L + \frac{\lambda}{2}\|\mathbf{w}\|^2$ ，更新中出现因子 $(1 - \eta\lambda)$ 。

4.5.6 练习

核验测试误差随 λ 的曲线、把惩罚改成 $\sum_j |w_j|$ 后的近端更新，以及 $\|\mathbf{w}\|_2^2 = \mathbf{w}^\top \mathbf{w}$ 在矩阵参数上对应的 Frobenius 式 $\|\mathbf{W}\|_F^2$ 。

4.6 暂退法 (Dropout)

暂退法在训练时随机将隐藏活性置零并重标定，降低对单一隐藏单元的依赖。

4.6.1 重新审视过拟合

线性模型偏差高、方差低；深度网可表示特征交互，方差高。特征多而 n 小时过拟合更严重。暂退法在不删特征的前提下注入噪声。

4.6.2 扰动的稳健性

对活性 h ，以丢弃概率 p 定义

$$h' = \begin{cases} 0, & \text{概率为 } p, \\ \frac{h}{1-p}, & \text{其他情况.} \end{cases}$$

于是 $\mathbb{E}[h'] = h$ 。测试时不用暂退，直接用 h 。

4.6.3 实践中的暂退法

暂退作用在激活之后。被置零的单元前向为 0，反向梯度亦为 0。靠近输入的层常用较小的 p 。推断默认 $p = 0$ 。

4.6.4 从零开始实现

掩码 $M_j \sim \text{Bernoulli}(1 - p)$,

$$h'_j = \frac{M_j}{1 - p} h_j.$$

$p = 0$ 为恒等, $p = 1$ 输出全零。

4.6.4.1 定义模型参数

两隐层 MLP, $784 \rightarrow 256 \rightarrow 256 \rightarrow 10$, 各层 $(\mathbf{W}^{(\ell)}, \mathbf{b}^{(\ell)})$ 。

4.6.4.2 定义模型

训练时

$$\begin{aligned}\mathbf{H}_1 &= \text{dropout}_{p_1}(\text{ReLU}(\mathbf{X}\mathbf{W}^{(1)} + \mathbf{b}^{(1)})), \\ \mathbf{H}_2 &= \text{dropout}_{p_2}(\text{ReLU}(\mathbf{H}_1\mathbf{W}^{(2)} + \mathbf{b}^{(2)})), \\ \mathbf{O} &= \mathbf{H}_2\mathbf{W}^{(3)} + \mathbf{b}^{(3)},\end{aligned}$$

常见 $p_1 = 0.2$ 、 $p_2 = 0.5$; 评估时两处暂退关闭。

4.6.4.3 训练和测试

训练循环与普通 MLP 相同; 测试前向是无掩码的确定性映射。

4.6.5 简洁实现

在每个全连接与激活之后插入暂退层, 训练随机丢弃, 测试恒等通过。

4.6.6 小结

暂退把 h 换成期望仍为 h 的 h' , 只在训练使用; 它与范数惩罚从不同方向限制共适应。

4.6.7 练习

核验交换两层 p 对 $\text{Var}(\mathbf{H}_\ell)$ 的影响、测试期保持暂退是否改变 $\mathbb{E}[\hat{\mathbf{y}}]$, 以及同时使用 $\lambda\|\mathbf{w}\|^2$ 与暂退时验证误差是否可加。

4.7 前向传播、反向传播和计算图

以带 L_2 的单隐层 MLP 为例，把梯度写成计算图上的反向复合。

4.7.1 前向传播

省略偏置后

$$\begin{aligned} \mathbf{z} &= \mathbf{W}^{(1)}\mathbf{x}, \\ \mathbf{h} &= \phi(\mathbf{z}), \\ \mathbf{o} &= \mathbf{W}^{(2)}\mathbf{h}, \\ L &= l(\mathbf{o}, y), \\ s &= \frac{\lambda}{2} (\|\mathbf{W}^{(1)}\|_F^2 + \|\mathbf{W}^{(2)}\|_F^2), \\ J &= L + s. \end{aligned}$$

前向按依赖顺序计算并缓存 $\mathbf{z}, \mathbf{h}, \mathbf{o}, L, s, J$ 。

4.7.2 前向传播计算图

结点是变量与算子。数据从 $\mathbf{x}$ 流向 J : $\mathbf{x}, \mathbf{W}^{(1)} \rightarrow \mathbf{z} \rightarrow \mathbf{h}$, 再与 $\mathbf{W}^{(2)}$ 得 $\mathbf{o}$, $(\mathbf{o}, y)$ 得 L , $(\mathbf{W}^{(1)}, \mathbf{W}^{(2)})$ 得 s , 最后 $J = L + s$ 。

4.7.3 反向传播

张量复合用

$$\frac{\partial Z}{\partial \mathbf{X}} = \text{prod}\left(\frac{\partial Z}{\partial \mathbf{Y}}, \frac{\partial \mathbf{Y}}{\partial \mathbf{X}}\right).$$

目标对两分支

$$\frac{\partial J}{\partial L} = 1, \quad \frac{\partial J}{\partial s} = 1.$$

对输出

$$\frac{\partial J}{\partial \mathbf{o}} = \text{prod}\left(\frac{\partial J}{\partial L}, \frac{\partial L}{\partial \mathbf{o}}\right) = \frac{\partial L}{\partial \mathbf{o}} \in \mathbb{R}^q.$$

正则项

$$\frac{\partial s}{\partial \mathbf{W}^{(1)}} = \lambda \mathbf{W}^{(1)}, \quad \frac{\partial s}{\partial \mathbf{W}^{(2)}} = \lambda \mathbf{W}^{(2)}.$$

第二层权重

$$\begin{aligned} \frac{\partial J}{\partial \mathbf{W}^{(2)}} &= \text{prod}\left(\frac{\partial J}{\partial \mathbf{o}}, \frac{\partial \mathbf{o}}{\partial \mathbf{W}^{(2)}}\right) + \text{prod}\left(\frac{\partial J}{\partial s}, \frac{\partial s}{\partial \mathbf{W}^{(2)}}\right) \\ &= \frac{\partial J}{\partial \mathbf{o}} \mathbf{h}^\top + \lambda \mathbf{W}^{(2)}. \end{aligned}$$

隐层

$$\frac{\partial J}{\partial \mathbf{h}} = \text{prod}\left(\frac{\partial J}{\partial \mathbf{o}}, \frac{\partial \mathbf{o}}{\partial \mathbf{h}}\right) = \mathbf{W}^{(2)\top} \frac{\partial J}{\partial \mathbf{o}}.$$

预激活

$$\frac{\partial J}{\partial \mathbf{z}} = \text{prod}\left(\frac{\partial J}{\partial \mathbf{h}}, \frac{\partial \mathbf{h}}{\partial \mathbf{z}}\right) = \frac{\partial J}{\partial \mathbf{h}} \odot \phi'(\mathbf{z}).$$

第一层权重

$$\begin{aligned} \frac{\partial J}{\partial \mathbf{W}^{(1)}} &= \text{prod}\left(\frac{\partial J}{\partial \mathbf{z}}, \frac{\partial \mathbf{z}}{\partial \mathbf{W}^{(1)}}\right) + \text{prod}\left(\frac{\partial J}{\partial s}, \frac{\partial s}{\partial \mathbf{W}^{(1)}}\right) \\ &= \frac{\partial J}{\partial \mathbf{z}} \mathbf{x}^\top + \lambda \mathbf{W}^{(1)}. \end{aligned}$$

4.7.4 训练神经网络

每步先前向缓存中间量，再反向用这些值组梯度，最后

$$\mathbf{W}^{(\ell)} \leftarrow \mathbf{W}^{(\ell)} - \eta \frac{\partial J}{\partial \mathbf{W}^{(\ell)}}.$$

反向依赖前向的 $\mathbf{h}, \mathbf{z}$ ；下一次前向又依赖更新后的 $\mathbf{W}$ 。训练需保存中间激活，内存大于纯预测。

4.7.5 小结

前向按图计算 $J = L + s$ ，反向按 $\partial J / \partial \mathbf{W}^{(2)} = \frac{\partial J}{\partial \mathbf{o}} \mathbf{h}^\top + \lambda \mathbf{W}^{(2)}$ 与 $\partial J / \partial \mathbf{W}^{(1)} = \frac{\partial J}{\partial \mathbf{z}} \mathbf{x}^\top + \lambda \mathbf{W}^{(1)}$ 写回参数梯度。

4.7.6 练习

核验 $f(\mathbf{X})$ 对 $\mathbf{X} \in \mathbb{R}^{n \times m}$ 的梯度形状、给隐层加 $\mathbf{b}$ 后的额外反向式，以及二阶导数如何加厚计算图。

4.8 数值稳定性和模型初始化

深度复合中每层的尺度进入梯度乘积。初始化决定起步时梯度是否可用，并打破隐藏单元对称。

4.8.1 梯度消失和梯度爆炸

L 层网络

$$\mathbf{h}^{(l)} = f_l(\mathbf{h}^{(l-1)}), \quad \mathbf{o} = f_L \circ \cdots \circ f_1(\mathbf{x}).$$

对第 l 层权重

$$\partial_{\mathbf{W}^{(l)}} \mathbf{o} = \underbrace{\partial_{\mathbf{h}^{(L-1)}} \mathbf{h}^{(L)}}_{\mathbf{M}^{(L)}} \cdots \underbrace{\partial_{\mathbf{h}^{(l)}} \mathbf{h}^{(l+1)}}_{\mathbf{M}^{(l+1)}} \underbrace{\partial_{\mathbf{W}^{(l)}} \mathbf{h}^{(l)}}_{\mathbf{v}^{(l)}}.$$

各 $\mathbf{M}$ 的奇异值大于 1 则爆炸，小于 1 则消失。

4.8.1.1 梯度消失

sigmoid 的导数最大为 $1/4$ ，且 $|x|$ 大时接近 0。多层连乘后 $\partial J / \partial \mathbf{W}^{(1)}$ 可数值为零。ReLU 在正半轴导数为 1，减轻该问题。

4.8.1.2 梯度爆炸

若每层 Jacobian 的尺度约 1 但略大，或初始化方差过大，则 $\prod_{\ell} \mathbf{M}^{(\ell)}$ 指数增长，一步更新即可使参数溢出。

4.8.1.3 打破对称性

隐单元置换不改变可表示函数。若 $\mathbf{W}^{(1)}$ 各行相同，则各隐单元激活相同、梯度相同，对称无法被梯度打破。随机独立初始化使各单元轨迹分离。

4.8.2 参数初始化

用适当方差的随机初始化控制前向与反向的尺度。

4.8.2.1 默认初始化

不指定时框架用小方差随机分布；中等深度往往可用，但不能保证任意深度。

4.8.2.2 Xavier 初始化

一层

$$o_i = \sum_{j=1}^{n_{\text{in}}} w_{ij} x_j.$$

设 $E[w_{ij}] = E[x_j] = 0$ 且独立， $E[x_j^2] = \gamma^2$ ， $E[w_{ij}^2] = \sigma^2$ ，则

$$\begin{aligned} E[o_i] &= \sum_{j=1}^{n_{\text{in}}} E[w_{ij}] E[x_j] = 0, \\ \text{Var}[o_i] &= E[o_i^2] - (E[o_i])^2 \end{aligned}$$

$$\begin{aligned}
&= \sum_{j=1}^{n_{\text{in}}} E[w_{ij}^2] E[x_j^2] \\
&= n_{\text{in}} \sigma^2 \gamma^2.
\end{aligned}$$

希望 $\text{Var}[o_i]$ 不随 n_{in} 爆炸，同时反向不随 n_{out} 爆炸，折中

$$\begin{aligned}
\frac{1}{2}(n_{\text{in}} + n_{\text{out}})\sigma^2 &= 1, \\
\sigma &= \sqrt{\frac{2}{n_{\text{in}} + n_{\text{out}}}}.
\end{aligned}$$

对应均匀分布

$$U\left(-\sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}, \sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}\right).$$

4.8.2.3 额外阅读

Xavier 只覆盖独立、零均值、方差匹配的情形。共享参数、残差或序列模型需要各自的尺度启发式。

4.8.3 小结

梯度是一层层 $\mathbf{M}^{(\ell)}$ 的乘积；Xavier 令 $\sigma = \sqrt{2/(n_{\text{in}} + n_{\text{out}})}$ ，使输出方差与梯度方差同时可控，随机性用来破对称。

4.8.4 练习

核验线性或 softmax 回归把全部 w_{ij} 设成同一常数是否仍能学，以及矩阵乘积特征值界对 $\prod \mathbf{M}^{(\ell)}$ 尺度的含义。

4.9 环境和分布偏移

部署时样本来自 p_T ，训练来自 p_S 。二者不同则经验风险最小解不必在测试上最优。

4.9.1 分布偏移的类型

无结构假设时， $p_S(\mathbf{x}) = p_T(\mathbf{x})$ 仍可有 $p_S(y | \mathbf{x}) = 1 - p_T(y | \mathbf{x})$ ，不可学。因此要对 $p(y | \mathbf{x})$ 或 $p(\mathbf{x} | y)$ 施加不变性。

4.9.1.1 协变量偏移

假设条件标签不变、特征边缘变：

$$p(y | \mathbf{x}) = q(y | \mathbf{x}), \quad p(\mathbf{x}) \neq q(\mathbf{x}).$$

在认为 $\mathbf{x}$ 导致 y 时自然成立。

4.9.1.2 标签偏移

假设类条件特征不变、标签边缘变：

$$p(\mathbf{x} | y) = q(\mathbf{x} | y), \quad p(y) \neq q(y).$$

在认为 y 导致 $\mathbf{x}$ （如疾病导致症状）时自然成立。

4.9.1.3 概念偏移

$p(y | \mathbf{x})$ 本身随时间或地点改变，即同一 $\mathbf{x}$ 对应的标签定义漂移。

4.9.2 分布偏移示例

下列情形中 p_S 与 p_T 的差异往往被精度数字掩盖。

4.9.2.1 医学诊断

用医院患者与校园献血者构造对照，则 $p(\mathbf{x})$ 混入年龄等与疾病无关的差异，即使 $p(y | \mathbf{x})$ 在目标人群中不同，训练集精度也不能迁移。

4.9.2.2 自动驾驶汽车

合成渲染的路沿纹理恒定，检测器学到该伪特征。真实图像的 $p(\mathbf{x})$ 不含这一捷径，部署失败。

4.9.2.3 非平稳分布

P 缓变而模型不更新：广告中的新设备、垃圾邮件的新模板、节后过时的推荐，都属于 $p_T \neq p_S$ 。

4.9.2.4 更多轶事

训练缺特写人脸、英美查询词分布不同、类别在网上均匀而现实长尾，都使经验风险偏离真实风险。

4.9.3 分布偏移纠正

在可检验的不变性下，用重要性权重改写风险积分。

4.9.3.1 经验风险与实际风险

经验风险最小化

$$\underset{f}{\text{minimize}} \quad \frac{1}{n} \sum_{i=1}^n l(f(\mathbf{x}_i), y_i)$$

逼近真实风险

$$\mathbb{E}_{p(\mathbf{x}, y)}[l(f(\mathbf{x}), y)] = \iint l(f(\mathbf{x}), y) p(\mathbf{x}, y) d\mathbf{x} dy.$$

4.9.3.2 协变量偏移纠正

$p(y | \mathbf{x}) = q(y | \mathbf{x})$ 时

$$\begin{aligned} & \iint l(f(\mathbf{x}), y) p(y | \mathbf{x}) p(\mathbf{x}) d\mathbf{x} dy \\ &= \iint l(f(\mathbf{x}), y) q(y | \mathbf{x}) q(\mathbf{x}) \frac{p(\mathbf{x})}{q(\mathbf{x})} d\mathbf{x} dy. \end{aligned}$$

样本权重

$$\beta_i \stackrel{\text{def}}{=} \frac{p(\mathbf{x}_i)}{q(\mathbf{x}_i)}$$

给出加权经验风险

$$\underset{f}{\text{minimize}} \quad \frac{1}{n} \sum_{i=1}^n \beta_i l(f(\mathbf{x}_i), y_i).$$

用分类器区分源/目标域，

$$P(z = 1 | \mathbf{x}) = \frac{p(\mathbf{x})}{p(\mathbf{x}) + q(\mathbf{x})}, \quad \frac{P(z = 1 | \mathbf{x})}{P(z = -1 | \mathbf{x})} = \frac{p(\mathbf{x})}{q(\mathbf{x})}.$$

若 $h(\mathbf{x})$ 为对数几率，则

$$\begin{aligned} \beta_i &= \frac{1/(1 + \exp(-h(\mathbf{x}_i)))}{\exp(-h(\mathbf{x}_i))/(1 + \exp(-h(\mathbf{x}_i)))} \\ &= \exp(h(\mathbf{x}_i)). \end{aligned}$$

4.9.3.3 标签偏移纠正

$p(\mathbf{x} | y) = q(\mathbf{x} | y)$ 时

$$\iint l(f(\mathbf{x}), y) p(\mathbf{x} | y) p(y) d\mathbf{x} dy$$

$$= \iint l(f(\mathbf{x}), y) q(\mathbf{x} | y) q(y) \frac{p(y)}{q(y)} d\mathbf{x} dy,$$

权重

$$\beta_i \stackrel{\text{def}}{=} \frac{p(y_i)}{q(y_i)}.$$

混淆关系

$$\mathbf{C} p(\mathbf{y}) = \mu(\hat{\mathbf{y}})$$

用源域混淆矩阵 $\mathbf{C}$ 与目标域预测边缘 $\mu(\hat{\mathbf{y}})$ 反解 $p(\mathbf{y})$ 。

4.9.3.4 概念偏移纠正

$p(y | \mathbf{x})$ 改变时重要性加权不再成立，需新标签。缓慢漂移可用新数据继续更新已有 θ 。

4.9.4 学习问题的分类法

按数据何时到达、环境是否记得行动，把学习写成不同的信息约束。

4.9.4.1 批量学习

一次给定 $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$ ，学得 f 后在同一 P 上部署，不再更新。

4.9.4.2 在线学习

$$f_t \longrightarrow \mathbf{x}_t \longrightarrow f_t(\mathbf{x}_t) \longrightarrow y_t \longrightarrow l(y_t, f_t(\mathbf{x}_t)) \longrightarrow f_{t+1}.$$

4.9.4.3 老虎机

行动集有限，每步选一只臂并观测损失。比连续参数化的 f 有更强的后悔界。

4.9.4.4 控制

环境状态依赖历史行动，例如锅炉温度依赖是否加热。策略必须考虑动力学，而不仅是 i.i.d. 样本。

4.9.4.5 强化学习

在可记忆、甚至对抗的环境中选行动以最大化期望回报。监督批学习是其无交互特例。

4.9.4.6 考虑到环境

若行动改变未来的 P ，静止环境上的最优 f 部署后可能失效。变化慢可限制参数步长；变化稀但剧烈则需检测后再适应。

4.9.5 机器学习中的公平、责任和透明度

部署把 $\hat{y}$ 变成影响人的决策。精度不是唯一风险：不同子群上 $p(y | \mathbf{x})$ 不同、各类错误代价不对称时，应用代价敏感的损失而不是单纯 acc。

4.9.6 小结

真实风险是 $\mathbb{E}_p[l]$ ，经验风险是其样本平均。协变量偏移用 $\beta_i = p(\mathbf{x}_i)/q(\mathbf{x}_i)$ 纠偏，标签偏移用 $\beta_i = p(y_i)/q(y_i)$ ；概念偏移则要求更新标签定义。

4.9.7 练习

核验用域分类器估计 β_i 是否恢复 $p(\mathbf{x})/q(\mathbf{x})$ ，以及除分布偏移外还有哪些因素使 R_{emp} 偏离 R 。

4.10 实战 Kaggle 比赛：预测房价

把前面的预处理、线性或 MLP、权重衰减与 K 折选择用到带缺失的表格回归。

4.10.1 下载和缓存数据集

名称映到 (url, sha1)。本地缓存命中且校验一致则不再下载。

4.10.2 Kaggle

比赛提供训练表（含房价）与测试表（无房价）。提交的是测试集上的 $\hat{y}$ 向量。

4.10.3 访问和读取数据集

每行一座房屋，特征含数值与类别，缺失记为 NA。价格 y 只在训练集出现。划分训练集得到验证折；官方测试误差只在提交后可见。

4.10.4 数据预处理

数值列标准化

$$x \leftarrow \frac{x - \mu}{\sigma},$$

缺失用列均值填充。类别列独热。训练集与测试集的编码与 μ, σ 必须同源统计量。

4.10.5 训练

比赛指标是对数均方根误差

$$\sqrt{\frac{1}{n} \sum_{i=1}^n (\log y_i - \log \hat{y}_i)^2},$$

对应相对误差而非绝对美元误差。模型可以是线性层或带权重衰减的 MLP。

4.10.6 K 折交叉验证

第 i 折取切片 $\mathcal{I}_i$ 为验证、其余为训练。 K 次后报告平均训练误差与平均验证误差。

4.10.7 模型选择

在折平均验证误差上选层宽、 λ 、暂退 p 与 η 。训练误差远低于折验证误差表示过拟合。

4.10.8 提交 Kaggle 预测

用选定超参数在全部训练样本上重拟合，再映射测试特征到 $\hat{y}$ ，写成提交表。

4.10.9 小结

表格映射是：标准化与独热得到 $\mathbf{X}$ ，用 $\log$ RMSE 训练，用 K 折选超参数后再全量拟合。

4.10.10 练习

核验直接优化 $\log y$ 与优化 y 再取对数的差别、均值填充在非随机缺失下的偏差，以及不标准化连续列时梯度尺度。

第五章 深度学习计算

层把输入映成输出并携带参数；块把层或块再复合。训练需要访问、初始化与共享这些参数，必要时延后到首次前向才确定形状。自定义层给出新的映射，读写把参数与张量固化到存储，计算设备决定张量所在的空间。

5.1 层和块

单输出线性模型是映射 $f: \mathbb{R}^d \rightarrow \mathbb{R}$ ，带参数 θ 。一层接受向量、产生向量、并由可训练参数描述；整个多层感知机仍是同一类对象。块（block）是可嵌套的计算单元：输入进前向映射，输出可改形状，梯度由反向传播给出。

5.1.1 自定义块

块必须实现前向 $f(\mathbf{x})$ ，并持有参数以便求导与初始化。以单隐层感知机为例，设 $\mathbf{x} \in \mathbb{R}^{20}$ ，

$$\begin{aligned} \mathbf{h} &= \text{ReLU}(\mathbf{W}_1 \mathbf{x} + \mathbf{b}_1), & \mathbf{W}_1 &\in \mathbb{R}^{256 \times 20}, \\ \mathbf{o} &= \mathbf{W}_2 \mathbf{h} + \mathbf{b}_2, & \mathbf{W}_2 &\in \mathbb{R}^{10 \times 256}. \end{aligned}$$

即 $f = \ell_2 \circ \text{ReLU} \circ \ell_1$ 。形状允许 $f(\mathbf{x}) \in \mathbb{R}^{10}$ 与 $\mathbf{x} \in \mathbb{R}^{20}$ 不同。反向传播给出 $\nabla_{\mathbf{W}_1} L$ 、 $\nabla_{\mathbf{W}_2} L$ 及对输入的 $\nabla_{\mathbf{x}} L$ 。

5.1.2 顺序块

顺序复合定义为

$$(\text{Seq}(f_1, \dots, f_n))(\mathbf{x}) = f_n \circ f_{n-1} \circ \dots \circ f_1(\mathbf{x}).$$

追加操作把 f_{n+1} 接到链条末端。参数集是各子块参数的不交并

$$\text{params}(\text{Seq}(f_1, \dots, f_n)) = \bigsqcup_{i=1}^n \text{params}(f_i),$$

这要求每个 f_i 登记为子模块，而不能只放入普通列表。

5.1.3 在前向传播函数中执行代码

并非所有架构都是 $f_n \circ \dots \circ f_1$ 。前向中可插入控制流或与参数无关的常数。设 c 为常数参数 (constant parameter)，层

$$f(\mathbf{x}, \mathbf{w}) = c \cdot \mathbf{w}^\top \mathbf{x}$$

在 $\mathbf{w}$ 上可训练， c 不进入 $\text{params}(f)$ 。亦允许按范数分支，例如

$$\mathbf{h} \leftarrow \begin{cases} \mathbf{h}/\|\mathbf{h}\|_2, & \|\mathbf{h}\|_2 > 1, \\ \mathbf{h}, & \text{否则}, \end{cases}$$

再进入后续线性层。只要这些运算可微或次可微，反向传播仍给出 $\nabla_{\theta} L$ 。

5.1.4 效率

前向若含大量宿主解释与字典查找，加速器上的核必须等待。总时间可写成

$$T = T_{\text{host}} + T_{\text{device}} + T_{\text{copy}}.$$

当 T_{host} 因全局解释器锁而串行时， T_{device} 被阻塞。把可向量化的映射尽量放进一次设备核，可减小 T_{host}/T 。

5.1.5 小结

块是可嵌套映射 f ，顺序块实现 $f_n \circ \dots \circ f_1$ ，并负责参数登记与反向传播。

5.1.6 练习

核验列表存储是否仍使 params 递归可收集，以及并联 $[f_1(\mathbf{x}), f_2(\mathbf{x})]$ 与重复实例 $f \circ \dots \circ f$ 的形状。

5.2 参数管理

架构与超参数固定后，训练求解

$$\boldsymbol{\theta}^* \in \underset{\boldsymbol{\theta}}{\operatorname{argmin}} L(\boldsymbol{\theta}).$$

随后需要读出 $\boldsymbol{\theta}$ 以便复用、保存或检查。下面把访问、初始化与共享写成对 $\boldsymbol{\theta}$ 的运算。

5.2.1 参数访问

顺序模型中第 ℓ 层参数是命名张量，通常含权重 $\mathbf{W}^{(\ell)}$ 与偏置 $\mathbf{b}^{(\ell)}$ 。名称在整网中唯一，即使层数达数百。

5.2.1.1 目标参数

参数对象同时携带值、梯度与状态。未反向传播时 $\nabla_{\theta} L$ 处于初始（通常为零）状态；一次 `L.backward` 之后

$$\frac{\partial L}{\partial \mathbf{W}^{(\ell)}}, \quad \frac{\partial L}{\partial \mathbf{b}^{(\ell)}}$$

才有定义。对参数的数值运算必须先取出底层张量。

5.2.1.2 一次性访问所有参数

嵌套块的参数树为

$$\text{params}(B) = \text{local}(B) \cup \bigcup_{B' \in \text{children}(B)} \text{params}(B').$$

一次性遍历给出全部 $(\text{name}, \theta_{\text{name}})$ ，而不必按层手工索引。

5.2.1.3 从嵌套块收集参数

若 $B = \text{Seq}(B_1, B_2, \dots)$ 且每个 B_i 自身又是顺序块，则名称按层级拼接。访问“第一主块、第二子块、第一层”的偏置，对应路径索引下的 $\mathbf{b}$ 。分层嵌套与嵌套列表索引同构。

5.2.2 参数初始化

默认把 $\mathbf{W}$ 、 $\mathbf{b}$ 抽自由输入输出维决定的均匀区间。自定义规则是从另一分布 P 抽样

$$\mathbf{W} \sim P.$$

5.2.2.1 内置初始化

高斯初始化取

$$w_{ij} \sim \mathcal{N}(0, 0.01^2), \quad b_i = 0.$$

亦可令全部元素为常数，例如 $\mathbf{W} = \mathbf{1}$ 。不同块可使用不同规则：第一层 Xavier，第三层常数 42。Xavier 对 $\mathbf{W} \in \mathbb{R}^{n_{\text{out}} \times n_{\text{in}}}$ 满足

$$\text{Var}(w) = \frac{2}{n_{\text{in}} + n_{\text{out}}} \quad (\text{或均匀区间按该方差标定}).$$

5.2.2.2 自定义初始化

按元素独立抽样

$$w \sim \begin{cases} U(5, 10), & \text{概率 } 1/4, \\ 0, & \text{概率 } 1/2, \\ U(-10, -5), & \text{概率 } 1/4. \end{cases}$$

该规则仍是合法的 P ，只是把质量放在远离零的两侧与原点。

5.2.3 参数绑定

共享指定 $\mathbf{W}^{(3)} = \mathbf{W}^{(5)} = \mathbf{W}$ 为同一张量，而非值相等的两份副本。于是 $d\mathbf{W}^{(3)} = d\mathbf{W}^{(5)}$ 。反向传播时梯度相加：

$$\frac{\partial L}{\partial \mathbf{W}} = \frac{\partial L}{\partial \mathbf{W}^{(3)}} + \frac{\partial L}{\partial \mathbf{W}^{(5)}}.$$

前向两次使用同一 $\mathbf{W}$ ，更新一步同时改变两处表观层。

5.2.4 小结

参数是带梯度的命名张量；初始化给定 P ，绑定使若干层共享同一 $\mathbf{W}$ 并把梯度相加。

5.2.5 练习

在含共享层的感知机上核验 $\mathbf{W}^{(i)} = \mathbf{W}^{(j)}$ 是否为同一存储，以及 $\nabla_{\mathbf{W}} L$ 是否等于两条路径梯度之和。

5.3 延后初始化

定义网络时可以暂不给出输入维 d 。此时第一层权重 $\mathbf{W}^{(1)} \in \mathbb{R}^{h \times d}$ 中的 d 未知，无法分配。延后初始化（deferred initialization）把形状推断推迟到数据第一次通过模型。

5.3.1 实例化网络

实例化后、首次前向之前，params 为空。设首个批量 $\mathbf{X} \in \mathbb{R}^{n \times d}$ 到达，由 d 确定

$$\mathbf{W}^{(1)} \in \mathbb{R}^{h \times d}, \quad \mathbf{W}^{(2)} \in \mathbb{R}^{q \times h},$$

再依层顺序分配并抽样。仅第一层真正依赖未知输入维，但框架仍按拓扑顺序完成全部形状。

5.3.2 小结

首次前向把未知维代入，使

$$\text{shape}(\mathbf{W}^{(\ell)})$$

全部成为已知后再初始化。

5.3.3 练习

核验：只给出第一层输入维时哪些层立即分配；维不匹配是否使乘法 $\mathbf{W}\mathbf{x}$ 无定义；输入维改变时共享参数如何约束形状。

5.4 自定义层

层是块的特例。当预置层不包含所需映射时，按同一接口定义新层：前向 f 加上可选的 $\text{params}(f)$ 。

5.4.1 不带参数的层

中心化层是无参数映射

$$f(\mathbf{x}) = \mathbf{x} - \bar{x}\mathbf{1}, \quad \bar{x} = \frac{1}{n} \sum_{i=1}^n x_i.$$

对批量矩阵按约定的轴取均值后广播相减。浮点下 $\sum_i f(\mathbf{x})_i$ 只是接近 0。该层可嵌入任意 Seq。

5.4.2 带参数的层

自定义全连接层需要 $\mathbf{W} \in \mathbb{R}^{q \times d}$ 、 $\mathbf{b} \in \mathbb{R}^q$,

$$f(\mathbf{x}) = \text{ReLU}(\mathbf{W}\mathbf{x} + \mathbf{b}).$$

用框架的参数构造器登记 $\mathbf{W}, \mathbf{b}$ ，则访问、初始化、共享、序列化都沿用统一接口，而不必为该层单独写读写程序。

5.4.3 小结

自定义层是映射 f ，可无参数或带 $\mathbf{W}, \mathbf{b}$ ；登记后即可出现在任意架构中。

5.4.4 练习

核验二次型层

$$y_k = \sum_{i,j} W_{ijk} x_i x_j$$

的形状 $k \mapsto y_k$ ，以及把输入映到傅里叶系数前半部分的线性映射。

5.5 读写文件

训练得到 θ 后，需要把张量或整份参数字典写入存储，以便部署或断点续训。

5.5.1 加载和保存张量

单个张量的读写是一对互逆映射

$$\text{save}(\mathbf{X}, p), \quad \mathbf{X}' = \text{load}(p),$$

在格式一致时 $\mathbf{X}' = \mathbf{X}$ 。列表 $(\mathbf{X}_1, \dots, \mathbf{X}_m)$ 与字典 $k \mapsto \mathbf{X}_k$ 同样可往返，后者直接对应“全部权重名到张量”。

5.5.2 加载和保存模型参数

整网保存的是参数字典而非可执行架构，因为架构含任意代码，难以序列化。恢复过程为

$$f \leftarrow \text{arch}(), \quad \theta(f) \leftarrow \text{load}(p).$$

必须先用代码重建与保存时相同的 f ，再填入 θ 。随机初始化被文件中的值覆盖。

5.5.3 小结

`save/load` 作用于张量；整网只固化 θ ，架构仍由代码给出。

5.5.4 练习

核验只加载前两层 $\theta_{1:2}$ 并入新架构是否形状相容，以及同时保存架构时必须对 f 施加何种可序列化限制。

5.6 GPU

张量带有设备 (device)。默认在主机内存由 CPU 计算；GPU 把同一线性代数核放到高吞吐器件上。记设备集合 $\{\text{cpu}, \text{cuda:0}, \dots, \text{cuda:k} - 1\}$ 。

5.6.1 计算设备

`cpu` 表示全部主机核与内存；`cuda:i` 表示第 i 块卡及其显存 (i 从 0 起)。`cuda:0` 与 `cuda` 等价。可用卡数 k 查询后，第 i 块记为 `cuda:i`。

5.6.2 张量与 GPU

每个张量 $\mathbf{X}$ 有位置 $\text{loc}(\mathbf{X})$ 。二元运算 $\mathbf{X} + \mathbf{Y}$ 有定义当且仅当

$$\text{loc}(\mathbf{X}) = \text{loc}(\mathbf{Y}),$$

否则结果的存储位置与计算位置均不确定。

5.6.2.1 存储在 GPU 上

创建时可指定 $\text{loc}(\mathbf{X}) = \text{cuda:0}$ 。该张量只占用该卡显存，容量受显存上界约束。第二块卡上的张量满足 $\text{loc}(\mathbf{Y}) = \text{cuda:1}$ 。

5.6.2.2 复制

若 $\text{loc}(\mathbf{X}) \neq \text{loc}(\mathbf{Y})$ ，应先做设备间复制

$$\mathbf{Z} = \text{copy}_{\text{loc}(\mathbf{Y})}(\mathbf{X}),$$

再计算 $\mathbf{Z} + \mathbf{Y}$ 。若 $\mathbf{Z}$ 已在目标设备，再次请求同一设备不分配新存储，即 `id` 不变。

5.6.2.3 旁注

器件间传输带宽远低于器件内计算。多次小复制的开销通常大于一次大复制。打印或转为主机数组会把数据拉回内存，并可能触发全局解释器锁，从而阻塞全部 GPU 核。

5.6.3 神经网络与 GPU

模型参数同样带位置。把 θ 放到设备 d 后，输入必须满足

$$\text{loc}(\mathbf{X}) = d,$$

前向才在 d 上计算 $f_{\theta}(\mathbf{X})$ 。数据和参数同设备是有效训练的成立条件。

5.6.4 小结

运算要求全部输入同设备；默认 $\text{loc} = \text{cpu}$ 。不必要的 copy 会支配 T 。

5.6.5 练习

比较大矩阵乘法上 CPU 与 GPU 的 T ，并核验逐次把 Frobenius 范数拉回主机是否显著增加 T_{copy} ；两卡并行与单卡顺序的时间比应接近线性。

第六章 卷积神经网络

全连接层把图像展平后不再区分空间邻近。卷积用平移不变与局部性约束核，使同一组权重在所有位置复用。下面从全连接出发得到互相关，再加入填充、步幅、通道与汇聚，最后组成 LeNet。

6.1 从全连接层到卷积

表格数据中特征无预定空间结构，多层感知机合适。对高维图像，若像素数为 10^6 、隐单元为 10^3 ，单层参数已达

$$10^6 \times 10^3 = 10^9.$$

需要把平移不变性与局部性写进映射，以减少自由参数。

6.1.1 不变性

检测物体的响应不应随物体平移而改变。同一滤波器在所有空间位置上扫描，等价于要求隐藏表示随输入平移而平移。

6.1.2 多层感知机的限制

把图像 $\mathbf{X} \in \mathbb{R}^{n_h \times n_w}$ 与四维核 $\mathbf{W}$ 的全连接写成

$$\begin{aligned} [\mathbf{H}]_{i,j} &= [\mathbf{U}]_{i,j} + \sum_k \sum_l [\mathbf{W}]_{i,j,k,l} [\mathbf{X}]_{k,l} \\ &= [\mathbf{U}]_{i,j} + \sum_a \sum_b [\mathbf{V}]_{i,j,a,b} [\mathbf{X}]_{i+a,j+b}. \end{aligned}$$

第二式只是把求和指标改成相对位移 (a, b) ，尚未减少参数。

6.1.2.1 平移不变性

令核与偏置都不依赖输出位置 (i, j) ，即 $[V]_{i,j,a,b} = [V]_{a,b}$ 、 $[U]_{i,j} = u$ ，则

$$[H]_{i,j} = u + \sum_a \sum_b [V]_{a,b} [X]_{i+a,j+b}.$$

同一 V 在所有位置复用。

6.1.2.2 局部性

再限制位移只在窗口 $|a|, |b| \leq \Delta$ 内，

$$[H]_{i,j} = u + \sum_{a=-\Delta}^{\Delta} \sum_{b=-\Delta}^{\Delta} [V]_{a,b} [X]_{i+a,j+b}.$$

远处像素不进入该处隐藏表示。 $\Delta = 0$ 时退化为逐像素通道混合。

6.1.3 卷积

连续卷积为

$$(f * g)(\mathbf{x}) = \int f(\mathbf{z}) g(\mathbf{x} - \mathbf{z}) d\mathbf{z}.$$

一维、二维离散形式为

$$\begin{aligned} (f * g)(i) &= \sum_a f(a) g(i - a), \\ (f * g)(i, j) &= \sum_a \sum_b f(a, b) g(i - a, j - b). \end{aligned}$$

与上一小节的互相关相比，卷积在核上多了翻转 $(a, b) \mapsto (-a, -b)$ 。

6.1.4 “沃尔多在哪里” 回顾

用滤波器 V 在图上滑动窗口并加权，得到目标出现的响应图。学习 V 使响应在目标位置取大值。

6.1.4.1 通道

输入 X 与输出 H 可有通道。四维核 V 把输入通道 c 混到输出通道 d ：

$$[H]_{i,j,d} = \sum_{a=-\Delta}^{\Delta} \sum_{b=-\Delta}^{\Delta} \sum_c [V]_{a,b,c,d} [X]_{i+a,j+b,c}.$$

6.1.5 小结

平移不变给出与位置无关的 $\mathbf{V}$ ，局部性把求和限制在 Δ 窗口；多通道时用 $\mathbf{V}$ 混合通道。

6.1.6 练习

当 $\Delta = 0$ 时核验上式是否在每个空间位置上退化为通道上的全连接；并说明二维离散卷积 $f * g = g * f$ 由核翻转后的交换性得出。

6.2 图像卷积

上节给出原理；本节把二维互相关写成可学习层，并用于边缘检测。

6.2.1 互相关运算

输入 $\mathbf{X} \in \mathbb{R}^{n_h \times n_w}$ 、核 $\mathbf{K} \in \mathbb{R}^{k_h \times k_w}$ 的互相关（cross-correlation）为

$$[\mathbf{Y}]_{ij} = \sum_{a=0}^{k_h-1} \sum_{b=0}^{k_w-1} [\mathbf{X}]_{i+a, j+b} [\mathbf{K}]_{ab}.$$

对 3×3 输入与 2×2 核，四个输出为

$$0 \cdot 0 + 1 \cdot 1 + 3 \cdot 2 + 4 \cdot 3 = 19,$$

$$1 \cdot 0 + 2 \cdot 1 + 4 \cdot 2 + 5 \cdot 3 = 25,$$

$$3 \cdot 0 + 4 \cdot 1 + 6 \cdot 2 + 7 \cdot 3 = 37,$$

$$4 \cdot 0 + 5 \cdot 1 + 7 \cdot 2 + 8 \cdot 3 = 43.$$

无填充、步幅为 1 时输出形状

$$(n_h - k_h + 1) \times (n_w - k_w + 1).$$

6.2.2 卷积层

卷积层在互相关之后加标量偏置：

$$\mathbf{Y} = \mathbf{X} \star \mathbf{K} + b.$$

可训练参数是 $\mathbf{K}$ 与 b 。高宽为 h, w 的核称为 $h \times w$ 卷积核，对应层称为 $h \times w$ 卷积层。

6.2.3 图像中目标的边缘检测

核 $\mathbf{K} = [1, -1]$ 在水平相邻像素上计算差分。相邻相等时输出 0，否则非零：由白到黑得 1，由黑到白得 -1。该 $\mathbf{K}$ 检测垂直边缘；转置输入后垂直边缘响应消失，因其不匹配水平结构。

6.2.4 学习卷积核

给定配对 $(\mathbf{X}, \mathbf{Y})$ ，学习

$$\min_{\mathbf{K}} \|\mathbf{X} \star \mathbf{K} - \mathbf{Y}\|_F^2.$$

梯度更新 $\mathbf{K}$ 若干步后，所得核接近手工差分核 $[1, -1]$ 。

6.2.5 互相关和卷积

严格卷积要求核翻转后再做互相关。若层执行互相关并学到 $\mathbf{K}$ ，则执行严格卷积时学到翻转后的 $\mathbf{K}'$ 。二者前向差一个核翻转；核从数据学习时，可表示的函数类相同，输出不受本质影响。

6.2.6 特征映射和感受野

卷积输出称为特征映射（feature map）。某元素 x 的感受野（receptive field）是前向中能影响 x 的所有先前元素。 2×2 核下，输出中对应 19 的元素感受野为输入的四个覆盖像素。叠层后感受野增长，可超过原始输入尺寸（若计入填充）。

6.2.7 小结

二维卷积层的核心是 $\mathbf{Y} = \mathbf{X} \star \mathbf{K} + b$ ；核可手工设计或由 $\|\mathbf{X} \star \mathbf{K} - \mathbf{Y}\|_F^2$ 学习。

6.2.8 练习

核验把 $\mathbf{X}$ 、 $\mathbf{K}$ 适当展开后 $\star$ 可写成一次矩阵乘法，以及对角边缘图像上 $\mathbf{K}$ 与 $\mathbf{K}^\top$ 的响应如何改变。

6.3 填充和步幅

无填充时输出为 $(n_h - k_h + 1) \times (n_w - k_w + 1)$ ，连续卷积会快速缩小空间尺寸。填充（padding）与步幅（stride）是另外两个控制输出形状的量。

6.3.1 填充

在输入四周补 p_h 行、 p_w 列（通常上下分摊 p_h 、左右分摊 p_w ）后，输出形状为

$$(n_h - k_h + p_h + 1) \times (n_w - k_w + p_w + 1).$$

当 k_h 为奇数且 $p_h = k_h - 1$ （步幅 1）时，

$$n_h - k_h + (k_h - 1) + 1 = n_h,$$

高宽保持不变。 3×3 核常用每边填充 1。

6.3.2 步幅

窗口每次在高、宽上分别滑动 s_h 、 s_w 。输出形状为

$$\left\lfloor \frac{n_h - k_h + p_h + s_h}{s_h} \right\rfloor \times \left\lfloor \frac{n_w - k_w + p_w + s_w}{s_w} \right\rfloor.$$

$s_h = s_w = 2$ 时空间尺寸大约减半。步幅大于 1 减少后续层的位置数，从而减少计算。

6.3.3 小结

填充增大输出高宽，常用于保持与输入相同；步幅按上式减小高宽。二者共同决定输出维。

6.3.4 练习

把给定的 (n_h, n_w, k, p, s) 代入上式，核验是否与层输出形状一致；并解释音频上 $s = 2$ 对应时间轴下采样。

6.4 多输入多输出通道

彩色输入形状为 $c_i \times h \times w$ ，通道维长度为 c_i （例如 RGB 中 $c_i = 3$ ）。输入、核与输出都升为至少三维张量。

6.4.1 多输入通道

输入通道数为 c_i 时，核也必须有 c_i 个二维切片，形状 $c_i \times k_h \times k_w$ 。对各通道做二维互相关再求和：

$$\mathbf{Y} = \sum_{c=1}^{c_i} \mathbf{X}^{(c)} \star \mathbf{K}^{(c)}.$$

$\mathbf{Y}$ 仍是单通道二维图。 $c_i = 1$ 时退化为上一节。

6.4.2 多输出通道

设输出通道数为 c_o 。每个输出通道配备一套形状为 $c_i \times k_h \times k_w$ 的核，总核形状为

$$c_o \times c_i \times k_h \times k_w.$$

第 d 个输出通道为

$$\mathbf{Y}^{(d)} = \sum_{c=1}^{c_i} \mathbf{X}^{(c)} \star \mathbf{K}^{(d,c)}, \quad d = 1, \dots, c_o.$$

加深网络时通常减小空间分辨率、增大 c_o ，使每处位置携带更多特征。

6.4.3 1×1 卷积层

$k_h = k_w = 1$ 时窗口内无空间邻域交互，计算只发生在通道上。在每个空间位置 (i, j) ,

$$[\mathbf{Y}]_{i,j,:} = \mathbf{W} [\mathbf{X}]_{i,j,:}, \quad \mathbf{W} \in \mathbb{R}^{c_o \times c_i},$$

即逐像素的全连接。高宽不变， 1×1 卷积用来改通道数并控制复杂度。

6.4.4 小结

多通道把互相关推广为对 c_i 求和、对 c_o 并列； 1×1 卷积是每像素上的 $\mathbf{W}\mathbf{x}$ 。

6.4.5 练习

核验无非线性时两个核串联可否写成一次卷积，以及前向乘法次数 $c_o c_i k_h k_w n'_h n'_w$ 随 c_i, c_o 加倍如何变化。

6.5 汇聚层

需要逐渐降低空间分辨率、增大感受野，使顶层对整图敏感。汇聚（pooling）在窗口上聚合，并减轻卷积响应对精确位置的敏感。

6.5.1 最大汇聚层和平均汇聚层

窗口 $\mathcal{W}_{ij}$ 上，

$$[\mathbf{H}]_{ij}^{\max} = \max_{(a,b) \in \mathcal{W}_{ij}} [\mathbf{X}]_{ab},$$

$$[\mathbf{H}]_{ij}^{\text{avg}} = \frac{1}{|\mathcal{W}_{ij}|} \sum_{(a,b) \in \mathcal{W}_{ij}} [\mathbf{X}]_{ab}.$$

对与卷积示例相同的 2×2 窗口，最大汇聚给出

$$\max(0, 1, 3, 4) = 4,$$

$$\max(1, 2, 4, 5) = 5,$$

$$\max(3, 4, 6, 7) = 7,$$

$$\max(4, 5, 7, 8) = 8.$$

6.5.2 填充和步幅

汇聚的输出形状与卷积相同，窗口 $k_h \times k_w$ 扮演核的角色：

$$\left\lfloor \frac{n_h - k_h + p_h + s_h}{s_h} \right\rfloor \times \left\lfloor \frac{n_w - k_w + p_w + s_w}{s_w} \right\rfloor.$$

默认步幅常取与窗口相等，例如 3×3 窗口默认 $s_h = s_w = 3$ 。窗口、填充、步幅均可取矩形且高宽不同。

6.5.3 多个通道

汇聚在每个输入通道上独立进行，不做通道间求和，因此

$$c_o = c_i.$$

通道维连结后的输入，汇聚后通道数保持不变。

6.5.4 小结

最大汇聚取窗口 $\max$ ，平均汇聚取均值；填充与步幅改变形状，通道数不变。

6.5.5 练习

把平均汇聚写成元素全为 $1/(p_h p_w)$ 的卷积核；并比较最大、平均与 softmax 加权聚合在窗口上的不同。

6.6 卷积神经网络 (LeNet)

不再把 28×28 图像展成 $\mathbb{R}^{784}$ ，而在空间上保留二维结构。卷积替代全连接可大幅减少参数。LeNet 是早期完整卷积网络。

6.6.1 LeNet

LeNet-5 分为卷积编码器与全连接块。每个卷积单元是 5×5 卷积、sigmoid 与 2×2 平均汇聚（步幅 2）：

$$\begin{aligned} C_1 &= \sigma(X \star K_1 + b_1) \in \mathbb{R}^{6 \times 24 \times 24}, \\ S_2 &= \text{AvgPool}_{2 \times 2}(C_1) \in \mathbb{R}^{6 \times 12 \times 12}, \\ C_3 &= \sigma(S_2 \star K_3 + b_3) \in \mathbb{R}^{16 \times 8 \times 8}, \\ S_4 &= \text{AvgPool}_{2 \times 2}(C_3) \in \mathbb{R}^{16 \times 4 \times 4}. \end{aligned}$$

展平后 $z \in \mathbb{R}^{256}$ ，再

$$h_5 = \sigma(W_5 z + b_5) \in \mathbb{R}^{120}, \quad h_6 = \sigma(W_6 h_5 + b_6) \in \mathbb{R}^{84}, \quad o = W_o h_6 + b_o \in \mathbb{R}^{10}.$$

池化使空间维数每次约减为 $1/4$ ，同时通道由 6 增至 16。

6.6.2 模型训练

训练映射与多层感知机相同：小批量上计算损失、反向、更新 θ 。每个参数参与的乘法次数多于同等参数量的全连接网，故常用 GPU。批量 X 与 θ 须满足 $\text{loc}(X) = \text{loc}(\theta)$ 。

6.6.3 小结

LeNet 把 $\sigma \circ (\cdot \star K)$ 与平均汇聚叠成编码器，再接全连接得到 $o \in \mathbb{R}^{10}$ 。

6.6.4 练习

核验把平均汇聚换成最大汇聚、把 σ 换成 ReLU、或改变 k 与 c_o 时，各层形状是否仍由填充-步幅公式给出。

第七章 现代卷积神经网络

LeNet 之后，更深的卷积网依靠更多数据与 GPU 才超过手工特征。AlexNet 证明学习特征可行；VGG 用重复块，NiN 用 1×1 卷积，GoogLeNet 用并行 Inception，BatchNorm 稳定深层分布，ResNet 用 $\boldsymbol{x} + F(\boldsymbol{x})$ ，DenseNet 用通道连结。

7.1 深度卷积神经网络（AlexNet）

小数据上卷积早已有效，但在更大真实图像上，神经网络曾长期不敌支持向量机等。传统视觉流水线用手工特征再接线性分类器；卷积则直接从像素学习 f 。

7.1.1 学习表征

2012 年前主导的是 SIFT、SURF、HOG 与词袋等固定特征。另一路线主张特征本身由数据学习，并把若干层叠成深度 $f_L \circ \cdots \circ f_1$ 。

7.1.1.1 缺少的成分：数据

深度模型参数多，需要大量有标签样本才优于凸方法。早期公开数据往往只有数百至数千张低分辨率图。ImageNet 提供约 10^6 张图、 10^3 类，使过参数化卷积网的经验风险可被数据约束。

7.1.1.2 缺少的成分：硬件

每轮训练要多次穿过线性代数层。GPU 原本优化 4×4 矩阵-向量积，与卷积中的局部乘加同类。相对 CPU，GPU 以高吞吐并行乘加降低 T_{device} ，使更深 f 可训练。

7.1.2 AlexNet

AlexNet 为八层：五层卷积、两层全连接隐层、一层全连接输出，并在 ImageNet 上显著超过手工特征。与 LeNet 同构但更深更宽，且用 ReLU 替代 sigmoid。

7.1.2.1 模型设计

输入按 224×224 计。第一层核 11×11 （匹配更大目标），随后 5×5 与 3×3 。在第一、第二、第五卷积后接 3×3 、步幅 2 的最大汇聚。通道数约为 LeNet 的十倍。末两层全连接各 4096 维，参数量很大。原实现曾把 θ 拆到两块 GPU；显存充足时可放在单设备上。

7.1.2.2 激活函数

$\text{ReLU}(u) = \max(0, u)$ 无需指数。sigmoid 在饱和区 $\sigma'(u) \approx 0$ ，反向传播无法更新对应参数；ReLU 在正半轴导数恒为 1，对初始化更不敏感。

7.1.2.3 容量控制和预处理

全连接层用暂退法约束复杂度；训练输入经翻转、裁切、变色等变换 T ，即在增广分布上最小化

$$\mathbb{E}_T[\ell(f_{\theta}(T(\mathbf{X})), y)].$$

用 224×224 单通道张量穿过各层，可核验与架构表一致的 shape。

7.1.3 读取数据集

原论文在 ImageNet 上训练。为缩短演示，把 Fashion-MNIST 的 28×28 升到 224×224 ，以便套用同一套核与步幅，尽管这并非该数据上的最优预处理。

7.1.4 训练 AlexNet

与 LeNet 相比，深度、宽度与分辨率都更大，故采用更小学习率 η ，否则 $\theta - \eta \nabla L$ 容易跨过合适区域。

7.1.5 小结

AlexNet 把 LeNet 式的 $\text{Conv} \circ \text{ReLU} \circ \text{Pool}$ 加深加密，并用暂退与增广控制 $\mathcal{F}$ 的有效容量。

7.1.6 练习

比较增加轮数后与 LeNet 的 L 下降；核验主要显存占用是否在 4096 维全连接的 $\mathbf{W}$ ，主要乘加是否在前几层大分辨率卷积。

7.2 使用块的网络（VGG）

AlexNet 证明深度有效，但未给出可复用的设计单元。VGG 把“卷积-激活-汇聚”抽象成块，用重复块堆出不同深度。

7.2.1 VGG 块

经典序列是：填充以保持分辨率的卷积、非线性、汇聚。VGG 块由若干 3×3 、填充 1 的卷积（各接 ReLU）再接 2×2 、步幅 2 的最大汇聚组成。填充 1 时

$$n - 3 + 2 \cdot 1 + 1 = n,$$

故块内空间尺寸不变，块末汇聚使高宽减半。记卷积层数为 n 、输出通道为 c ，则一块是映射

$$\text{VGG-block}(\mathbf{X}; n, c) = \text{MaxPool}_{2,2}((\text{ReLU} \circ \text{Conv}_{3 \times 3})^n(\mathbf{X})).$$

7.2.2 VGG 网络

网络仍分卷积段与全连接段。超参数 `conv_arch` 给出每块的 (n, c) 。原始 VGG-11 含五块：前两块各一层卷积，后三块各两层，通道 64, 128, 256, 512, 512，故八层卷积加三层全连接。后续块将 c 翻倍直至 512。

7.2.3 训练模型

完整 VGG-11 计算量大于 AlexNet。演示中减小各块通道数，仍在 Fashion-MNIST 上按同一损失训练，学习率可略高。

7.2.4 小结

VGG-11 由可复用块 $\text{VGG-block}(\cdot; n, c)$ 串联；更深更窄的 3×3 栈优于更浅更宽的大核。

7.2.5 练习

打印形状时卷积层计数为 8 而非 11，核验其余三层是否为全连接；并把输入从 224 改为 96 后代入步幅公式，看每块输出高宽如何减半。

7.3 网络中的网络 (NiN)

LeNet、AlexNet、VGG 都是卷积提取空间特征再接全连接。NiN 在每个空间位置的通道上再放多层感知机，即 1×1 卷积，而不在早期展平。

7.3.1 NiN 块

四维张量轴为样本、通道、高、宽。把每个像素当作样本、通道当作特征，则逐位置全连接即 1×1 卷积。NiN 块为

$$\text{NiN-block} = (\text{ReLU} \circ \text{Conv}_{1 \times 1})^2 \circ (\text{ReLU} \circ \text{Conv}_{k \times k}),$$

先用 $k \times k$ 提取局部，再用两个 1×1 增加逐像素非线性。

7.3.2 NiN 模型

卷积窗口取 11×11 、 5×5 、 3×3 ，通道设置平行于 AlexNet。每块后接 3×3 、步幅 2 的最大汇聚。取消末端全连接：最后一块输出通道数等于类别数 q ，再全局平均汇聚

$$o_c = \frac{1}{hw} \sum_{i=1}^h \sum_{j=1}^w [\mathbf{H}]_{c,ij}, \quad c = 1, \dots, q,$$

得到 logits。参数量显著小于大全连接，但有时增加达到相同训练误差的时间。

7.3.3 训练模型

在 Fashion-MNIST 上的目标与 AlexNet、VGG 相同，只是 f_θ 换成 NiN 块加全局平均汇聚。

7.3.4 小结

NiN 用 $k \times k$ 后接 1×1 的块提供逐像素 MLP，并用全局平均汇聚代替全连接输出。

7.3.5 练习

去掉块中一个 1×1 层，核验逐像素非线性深度如何改变；并估计参数量主要来自各 Conv_k 的 $c_o c_i k^2$ 。

7.4 含并行连结的网络（GoogLeNet）

不同任务上“最佳”核宽并不唯一。Inception 在同一层并行使用多种窗口，再沿通道连结。

7.4.1 Inception 块

四条路径并行，输出在通道维连结：

$$\begin{aligned} P_1 &= \text{Conv}_{1 \times 1}(X), \\ P_2 &= \text{Conv}_{3 \times 3}(\text{Conv}_{1 \times 1}(X)), \\ P_3 &= \text{Conv}_{5 \times 5}(\text{Conv}_{1 \times 1}(X)), \\ P_4 &= \text{Conv}_{1 \times 1}(\text{MaxPool}_{3 \times 3}(X)), \\ H &= [P_1, P_2, P_3, P_4]. \end{aligned}$$

中间路径的 1×1 先把通道数降为 c' ，再做 3×3 或 5×5 ，以降低 $c_o c_i k^2 n'_h n'_w$ 。各路径输出高宽相同（靠填充），通道数相加。

7.4.2 GoogLeNet 模型

九个 Inception 块与其间的最大汇聚堆叠，末端用全局平均汇聚而非大全连接。第一模块：64 通道、 7×7 卷积。第二模块： 1×1 扩到 64 通道再 3×3 把通道乘三，对应 Inception 第二条路径。其后模块串联 Inception 块，块间汇聚降维。通道分配比由实验选定。

7.4.3 训练模型

输入先变成 96×96 ，再在 Fashion-MNIST 上最小化同一分类损失。

7.4.4 小结

Inception 把多尺度 $\text{Conv}_{1,3,5}$ 与汇聚并行后连结，并用 1×1 降维；GoogLeNet 将其重复九次。

7.4.5 练习

比较删去部分路径后的 Inception 与残差块的关系；并核验通道分配改变时 H 的通道维 $c_1 + c_2 + c_3 + c_4$ 。

7.5 批量规范化

深层网难以在短时间内收敛。批量规范化 (batch normalization) 用小批量统计量校正中间表示, 并可与残差块一起支持很深的 f 。

7.5.1 训练深层网络

对小批量 $\mathcal{B}$ 中的向量,

$$\begin{aligned}\hat{\boldsymbol{\mu}}_{\mathcal{B}} &= \frac{1}{|\mathcal{B}|} \sum_{\mathbf{x} \in \mathcal{B}} \mathbf{x}, \\ \hat{\boldsymbol{\sigma}}_{\mathcal{B}}^2 &= \frac{1}{|\mathcal{B}|} \sum_{\mathbf{x} \in \mathcal{B}} (\mathbf{x} - \hat{\boldsymbol{\mu}}_{\mathcal{B}})^2 + \epsilon, \\ \text{BN}(\mathbf{x}) &= \boldsymbol{\gamma} \odot \frac{\mathbf{x} - \hat{\boldsymbol{\mu}}_{\mathcal{B}}}{\hat{\boldsymbol{\sigma}}_{\mathcal{B}}} + \boldsymbol{\beta}.\end{aligned}$$

$\boldsymbol{\gamma}, \boldsymbol{\beta}$ 可学习, 使该层仍能恢复任意仿射变换; $\epsilon > 0$ 避免除零。

7.5.2 批量规范化层

BN 依赖整个小批量, 不能在公式中把批量大小当作 1 而忽略。全连接与卷积上的归约轴不同。

7.5.2.1 全连接层

放在仿射与激活之间:

$$\mathbf{h} = \phi(\text{BN}(\mathbf{W}\mathbf{x} + \mathbf{b})).$$

均值与方差在批量维上对特征计算, $\boldsymbol{\gamma}, \boldsymbol{\beta}$ 与特征同维。

7.5.2.2 卷积层

同样放在卷积之后、 ϕ 之前。每个输出通道一套标量 γ_c, β_c 。若批量大小为 m 、空间为 $p \times q$, 则第 c 通道在 $m \cdot p \cdot q$ 个元素上共用同一 $\hat{\mu}_c, \hat{\sigma}_c$ 。

7.5.2.3 预测过程中的批量规范化

预测时不再使用当前微批的噪声统计, 且可能 $|\mathcal{B}| = 1$ 。用训练期移动平均 $\boldsymbol{\mu}, \boldsymbol{\sigma}$ 代替 $\hat{\boldsymbol{\mu}}_{\mathcal{B}}, \hat{\boldsymbol{\sigma}}_{\mathcal{B}}$, 使 BN 成为确定映射。因此训练与预测的计算公式不同。

7.5.3 从零实现

实现对应上面的 $\text{BN}_{\gamma, \beta}$ ：训练时用批量矩并更新移动平均，预测时用移动平均。 γ, β 是参数；特征数在从零实现中需预先给出。

7.5.4 使用批量规范化层的 LeNet

在每个卷积或全连接之后、激活之前插入 BN。训练目标不变，但中间表示被校正，通常允许更大的 η 。第一层 BN 学到的 γ, β 可直接读出。

7.5.5 简明实现

框架中的 BN 与从零公式同一映射，只是核在编译后端上计算。同一超参数下应得到同构的 f_{θ} 。

7.5.6 争议

BN 常使优化更平滑，并有正则化副作用。将其解释为减少“内部协变量偏移”并不严谨：该说法与标准协变量偏移不是同一对象，也未构成对泛化的完整证明。

7.5.7 小结

训练时

$$\text{BN}(\mathbf{x}) = \gamma \odot (\mathbf{x} - \hat{\boldsymbol{\mu}}_B) / \hat{\boldsymbol{\sigma}}_B + \beta;$$

预测改用全局移动矩。全连接与卷积的归约轴不同。

7.5.8 练习

核验仿射层在 BN 前是否仍需要偏置 $\mathbf{b}$ (BN 已含 β)；并比较有无 BN 时可用的 η 上界。

7.6 残差网络 (ResNet)

加深时必须保证新层不会把已有函数类变差。残差连接把恒等映射作为默认分量。

7.6.1 函数类

在函数类 $\mathcal{F}$ 中选

$$f_{\mathcal{F}}^* := \operatorname{argmin}_{f \in \mathcal{F}} L(\mathbf{X}, \mathbf{y}, f).$$

若 $\mathcal{F}_1 \subset \mathcal{F}_2$, 则

$$\min_{f \in \mathcal{F}_2} L \leq \min_{f \in \mathcal{F}_1} L.$$

嵌套类使更深模型至少能表示更浅模型；构造上需让恒等容易实现。

7.6.2 残差块

设理想映射为 $f(\mathbf{x})$ 。直接拟合 $f(\mathbf{x})$ 改为拟合残差 $F(\mathbf{x}) := f(\mathbf{x}) - \mathbf{x}$, 则

$$f(\mathbf{x}) = \mathbf{x} + F(\mathbf{x}).$$

若最优为恒等 $f(\mathbf{x}) = \mathbf{x}$, 只需 $F \approx \mathbf{0}$ 。形状相同用恒等捷径；通道或步幅改变时用 1×1 投影 $\mathbf{W}_s$:

$$\mathbf{y} = \phi(\mathbf{x} + F(\mathbf{x})) \quad \text{或} \quad \phi(\mathbf{W}_s \mathbf{x} + F(\mathbf{x})).$$

块内 F 通常为两层 $\text{Conv} \circ \text{BN} \circ \text{ReLU}$ 。

7.6.3 ResNet 模型

前两层同 GoogLeNet 风格：64 通道、 7×7 卷积（步幅 2），再 3×3 最大汇聚（步幅 2），每卷积后接 BN。其后四个模块由残差块组成：第一模块不降空间尺寸，之后每模块在首块把通道翻倍、高宽减半。演示中每模块两个残差块。末端全局平均汇聚加全连接。

7.6.4 训练模型

在 Fashion-MNIST 上最小化同一分类损失；前向为嵌套的 $\mathbf{x} \mapsto \mathbf{x} + F(\mathbf{x})$ 。

7.6.5 小结

残差块实现 $f(\mathbf{x}) = \mathbf{x} + F(\mathbf{x})$, 使更深 $\mathcal{F}$ 容易包含恒等，从而可训练很深的网。

7.6.6 练习

比较 Inception 多路径连结与 $\mathbf{x} + F(\mathbf{x})$ 相加的差别；并说明即使 $\mathcal{F}$ 嵌套，仍要限制 F 的复杂度以免过拟合。

7.7 稠密连接网络 (DenseNet)

DenseNet 把残差的加法换成通道维连结，使第 ℓ 层看见之前所有层的特征。

7.7.1 从 ResNet 到 DenseNet

泰勒展开

$$f(x) = f(0) + f'(0)x + \frac{f''(0)}{2!}x^2 + \frac{f'''(0)}{3!}x^3 + \dots$$

把 f 拆成不同阶项。ResNet 的一阶切分是

$$f(\mathbf{x}) = \mathbf{x} + g(\mathbf{x}).$$

DenseNet 把“加上 $g(\mathbf{x})$ ”换成与此前所有映射的连结

$$\mathbf{x} \longrightarrow [\mathbf{x}, f_1(\mathbf{x}), f_2([\mathbf{x}, f_1(\mathbf{x})]), f_3([\mathbf{x}, f_1(\mathbf{x}), f_2([\mathbf{x}, f_1(\mathbf{x})])]), \dots].$$

信息以拼接而非求和跨层传递。

7.7.2 稠密块体

块内使用 $\text{BN} \circ \text{ReLU} \circ \text{Conv}$ 。第 i 个卷积块输出 k 个通道（增长率 growth rate），并与输入在通道维连结。若输入通道为 c_0 、块内 m 个卷积块，则输出通道

$$c_0 + mk.$$

例如 $c_0 = 3$ 、 $m = 2$ 、 $k = 10$ 时输出 $3 + 20 = 23$ 通道。

7.7.3 过渡层

稠密块使通道单调增加。过渡层用 1×1 卷积把通道压到 c' ，再用步幅 2 的平均汇聚减半高宽：

$$\mathbf{X} \mapsto \text{AvgPool}_{2,2}(\text{Conv}_{1 \times 1}(\mathbf{X})).$$

从而控制模型复杂度。

7.7.4 DenseNet 模型

茎部与 ResNet 相同：卷积加最大汇聚。随后四个稠密块，演示中每块 4 层、增长率 32，故每块增加 128 通道。块间用过渡层减半空间并减半通道。末端全局汇聚加全连接。

7.7.5 训练模型

网络较深时把输入高宽从 224 降到 96 以减小 chw 带来的显存，再在 Fashion-MNIST 上训练。

7.7.6 小结

ResNet 用 $\mathbf{x} + F(\mathbf{x})$ ；DenseNet 用通道连结 $[\mathbf{x}, f_1(\mathbf{x}), f_2([\mathbf{x}, f_1(\mathbf{x})]), \dots]$ ，并由过渡层控制维数。

7.7.7 练习

解释参数量相对 ResNet 更小（核看到已拼接特征，无需再学一份副本）；并核验输入改为 224×224 时连结张量的显存如何随层数线性增长。

第八章 循环神经网络

序列把观测写成按时间排列的随机变量。语言模型把联合分布因式分解为条件概率 $P(x_1, \dots, x_T)$ ，循环神经网络用共享参数的隐状态压缩历史，并通过时间反向传播计算梯度。正文保留原书标题，内容以公式为主。

8.1 序列模型

记观测序列为 $x_1, \dots, x_T$ 。每个时间步的合法预测只能依赖已经发生的历史，而不能依赖未来。下面先固定条件分布，再写成可训练的特征-标签对。

8.1.1 统计工具

对任意 t ，当前项的生成规律是条件分布

$$x_t \sim P(x_t \mid x_{t-1}, \dots, x_1).$$

联合分布由这些条件依次相乘得到。不同的条件独立性假设对应不同的模型类。

8.1.1.1 自回归模型

链规则给出精确分解

$$P(x_1, \dots, x_T) = \prod_{t=1}^T P(x_t \mid x_{t-1}, \dots, x_1).$$

自回归模型直接估计右侧每个条件分布，输入是过去观测本身。隐变量自回归则另设 $h_t = f(x_t, h_{t-1})$ ，再用 $P(x_t \mid h_{t-1})$ 近似条件。

8.1.1.2 马尔可夫模型

若只依赖最近一项，则得到一阶马尔可夫模型

$$P(x_1, \dots, x_T) = \prod_{t=1}^T P(x_t \mid x_{t-1}), \quad P(x_1 \mid x_0) = P(x_1).$$

此时隔一步的条件可由全概率公式消去中间变量：

$$\begin{aligned} P(x_{t+1} | x_{t-1}) &= \frac{\sum_{x_t} P(x_{t+1}, x_t, x_{t-1})}{P(x_{t-1})} \\ &= \sum_{x_t} P(x_{t+1} | x_t) P(x_t | x_{t-1}). \end{aligned}$$

更高阶只是把条件窗口从 1 换成 τ 。

8.1.1.3 因果关系

时间向前时，用过去解释现在。若强行反过来因式分解，

$$P(x_1, \dots, x_T) = \prod_{t=T}^1 P(x_t | x_{t+1}, \dots, x_T),$$

则估计对象变成“由未来推断过去”。因果序列上正向条件通常比反向更容易稳定估计。

8.1.2 训练

用正弦加噪声生成 $x_t = \sin t + \epsilon_t$, $t = 1, \dots, 1000$ 。嵌入窗口长度 τ 后，监督对为

$$y_t = x_t, \quad \mathbf{x}_t = (x_{t-\tau}, \dots, x_{t-1}).$$

前 τ 项没有完整历史，故丢弃；训练只用前 600 个对。于是回归映射是

$$\hat{x}_t = f(\mathbf{x}_t).$$

8.1.3 预测

单步预测把真实历史送入 f 。多步则把已有预测再送回去。以 $\tau = 4$ 为例，

$$\hat{x}_{605} = f(x_{601}, x_{602}, x_{603}, x_{604}),$$

$$\hat{x}_{606} = f(x_{602}, x_{603}, x_{604}, \hat{x}_{605}),$$

$$\hat{x}_{607} = f(x_{603}, x_{604}, \hat{x}_{605}, \hat{x}_{606}).$$

当输入窗口全部换成 $\hat{x}$ 后，误差沿时间累积，这就是 k 步预测。

8.1.4 小结

序列模型的核心对象是

$$P(x_1, \dots, x_T) = \prod_{t=1}^T P(x_t | x_{t-1}, \dots, x_1),$$

训练必须尊重时间顺序； k 步预测把估计误差迭代送入同一映射 f 。

8.1.5 练习

核验对象是窗口长度 τ 、噪声为零时正弦的最小充分历史，以及把已有 $\hat{x}$ 反馈后 k 步误差如何放大。

8.2 文本预处理

文本是离散序列。预处理把它映成整数下标序列，供后面的语言模型计算 $P(x_1, \dots, x_T)$ 。

8.2.1 读取数据集

语料是字符串行的列表。清洗后每行是小写字母与空格组成的序列，标点与大小写差异被抹掉，得到原始字符流。

8.2.2 词元化

词元化是映射

$$\text{tokenize} : \text{字符串} \longrightarrow (z_1, \dots, z_L), \quad z_\ell \in \mathcal{Z},$$

其中 $\mathcal{Z}$ 是词元集合，可取单词或字符。输出是词元列表的列表。

8.2.3 词表

词表是单射

$$\text{Vocab} : \mathcal{Z} \cup \{\text{<unk>}\} \longrightarrow \{0, 1, \dots, |\mathcal{V}| - 1\}.$$

先统计语料频率 $n(z)$ ，丢掉 $n(z) < m$ 的低频项，剩余按频次分配下标。未见或已删词元映到 <unk> 。可选地加入填充 <pad> 、开始 <bos> 、结束 <eos> 。

8.2.4 整合所有功能

把读入、词元化与查表合成

$$\text{文本} \longmapsto (\text{corpus}, \text{vocab}),$$

其中 $\text{corpus} = (x_1, \dots, x_T)$ 且 $x_t \in \{0, \dots, |\mathcal{V}| - 1\}$ 。后续字符级语言模型把整本语料当作一条长序列，而不是按行切成多条。

8.2.5 小结

预处理的终点是整数序列 $(x_1, \dots, x_T)$ 与词表 Vocab，后面所有概率都定义在这套下标上。

8.2.6 练习

核验词元粒度（字符或词）以及最低频次 m 如何改变 $|\mathcal{V}|$ 。

8.3 语言模型和数据集

语言模型的对象是词元序列的联合分布

$$P(x_1, x_2, \dots, x_T).$$

估计该分布后，就能给任意句子打分，也能逐项生成后续词元。

8.3.1 学习语言模型

链规则仍然成立：

$$P(x_1, x_2, \dots, x_T) = \prod_{t=1}^T P(x_t \mid x_1, \dots, x_{t-1}).$$

例如

$$\begin{aligned} &P(\text{deep, learning, is, fun}) \\ &= P(\text{deep}) P(\text{learning} \mid \text{deep}) P(\text{is} \mid \text{deep, learning}) \\ &\quad \cdot P(\text{fun} \mid \text{deep, learning, is}). \end{aligned}$$

计数估计是

$$\hat{P}(\text{learning} \mid \text{deep}) = \frac{n(\text{deep, learning})}{n(\text{deep})}.$$

低频组合用拉普拉斯平滑：

$$\begin{aligned} \hat{P}(x) &= \frac{n(x) + \epsilon_1/m}{n + \epsilon_1}, \\ \hat{P}(x' \mid x) &= \frac{n(x, x') + \epsilon_2 \hat{P}(x')}{n(x) + \epsilon_2}, \\ \hat{P}(x'' \mid x, x') &= \frac{n(x, x', x'') + \epsilon_3 \hat{P}(x'')}{n(x, x') + \epsilon_3}. \end{aligned}$$

8.3.2 马尔可夫模型与 n 元语法

截断条件窗口得到 n 元语法。对长度为 4 的序列，

$$\begin{aligned} P(x_1, x_2, x_3, x_4) &= P(x_1)P(x_2)P(x_3)P(x_4), \\ P(x_1, x_2, x_3, x_4) &= P(x_1)P(x_2 | x_1)P(x_3 | x_2)P(x_4 | x_3), \\ P(x_1, x_2, x_3, x_4) &= P(x_1)P(x_2 | x_1)P(x_3 | x_1, x_2)P(x_4 | x_2, x_3). \end{aligned}$$

三式分别是一元、二元、三元语法。

8.3.3 自然语言统计

词频近似服从齐普夫定律。按降序排列后

$$n_i \propto \frac{1}{i^\alpha}, \quad \log n_i = -\alpha \log i + c.$$

该幂律不仅出现在一元语法，也出现在更高阶 n 元计数上，因此长片段的经验频率会迅速变得极稀。

8.3.4 读取长序列数据

语料长度 T 通常远大于一次能送入网络的步数 n 。把长序列切成长度为 n 的子序列，标签是整体左移一项后的序列，即用 $(x_t, \dots, x_{t+n-1})$ 预测 $(x_{t+1}, \dots, x_{t+n})$ 。

8.3.4.1 随机采样

每个小批量在长序列上独立抽取起点。相邻两个批量的子序列在原文中不必相邻。长度为 35、步数 5 时可切出

$$\lfloor (35 - 1)/5 \rfloor = 6$$

个特征-标签对；批量大小为 2 时得到 3 个批量。

8.3.4.2 顺序分区

相邻批量的第 i 条子序列在原文中也相邻。隐状态可以跨批量延续；随机采样则每个批量必须重新初始化隐状态。两种读法都输出形状为 (批量大小, n) 的 $(\mathbf{X}, \mathbf{Y})$ 。

8.3.5 小结

语言模型估计 $P(x_1, \dots, x_T)$ ； n 元语法用马尔可夫截断使计数可行，长序列则按随机采样或顺序分区切成固定长度窗口。

8.3.6 练习

核验四元语法在词表大小 10^5 时要存储的计数规模，以及随机偏移是否让窗口起点在文档上接近均匀。

8.4 循环神经网络

用一个隐状态压缩全部历史，把变长条件改成固定形状的递推：

$$P(x_t | x_{t-1}, \dots, x_1) \approx P(x_t | h_{t-1}), \quad h_t = f(x_t, h_{t-1}).$$

参数不随 t 增加。

8.4.1 无隐状态的神经网络

把一个时间步的批量输入 $\mathbf{X} \in \mathbb{R}^{n \times d}$ 送入普通隐层

$$\begin{aligned} \mathbf{H} &= \phi(\mathbf{X}\mathbf{W}_{xh} + \mathbf{b}_h), \\ \mathbf{O} &= \mathbf{H}\mathbf{W}_{hq} + \mathbf{b}_q. \end{aligned}$$

不同时间步的 $\mathbf{H}$ 互不传递，因此无法携带 t 之前的信息。

8.4.2 有隐状态的循环神经网络

加入跨时间的循环项 $\mathbf{W}_{hh}$ ：

$$\begin{aligned} \mathbf{H}_t &= \phi(\mathbf{X}_t\mathbf{W}_{xh} + \mathbf{H}_{t-1}\mathbf{W}_{hh} + \mathbf{b}_h), \\ \mathbf{O}_t &= \mathbf{H}_t\mathbf{W}_{hq} + \mathbf{b}_q. \end{aligned}$$

其中 $\mathbf{X}_t \in \mathbb{R}^{n \times d}$ ， $\mathbf{H}_t \in \mathbb{R}^{n \times h}$ ， $\mathbf{O}_t \in \mathbb{R}^{n \times q}$ 。同一套 $(\mathbf{W}_{xh}, \mathbf{W}_{hh}, \mathbf{W}_{hq})$ 在所有 t 上共享。

8.4.3 基于循环神经网络的字符级语言模型

词元取字符。批量大小为 1 时，序列如 “machine” 的每个字符先映成向量，再按时间送入上式。时刻 t 的标签是下一个字符，输出 $\mathbf{O}_t$ 经 softmax 得到 $P(x_{t+1} | h_t)$ 。训练目标仍是

$$P(x_1, \dots, x_T) = \prod_{t=1}^T P(x_t | h_{t-1}).$$

8.4.4 困惑度 (Perplexity)

平均负对数似然

$$\frac{1}{n} \sum_{t=1}^n -\log P(x_t | x_{t-1}, \dots, x_1)$$

再指数化，得到困惑度

$$\exp\left(-\frac{1}{n} \sum_{t=1}^n \log P(x_t | x_{t-1}, \dots, x_1)\right).$$

它是模型在每个位置上有效分支数的几何平均：均匀乱猜约为 $|\mathcal{V}|$ ，完美预测为 1。

8.4.5 小结

循环计算

$$\mathbf{H}_t = \phi(\mathbf{X}_t \mathbf{W}_{xh} + \mathbf{H}_{t-1} \mathbf{W}_{hh} + \mathbf{b}_h)$$

使 h_t 携带到当前为止的历史，并用困惑度衡量 $P(x_t | h_{t-1})$ 的质量。

8.4.6 练习

核验下一字符的输出维必须等于 $|\mathcal{V}|$ ，以及沿很长的 t 反传时乘积 $\prod_j \partial h_j / \partial h_{j-1}$ 的尺度。

8.5 循环神经网络的从零开始实现

把上一节的映射在字符语料上展开：独热输入、共享循环核、裁剪后的梯度更新，以及用困惑度衡量的生成。

8.5.1 独热编码

词表大小为 N 时，下标 i 映到单位向量 $\mathbf{e}_i \in \{0, 1\}^N$ ， $(\mathbf{e}_i)_j = [i = j]$ 。小批量时间步 $\mathbf{X}_t$ 的每一行都是某个 $\mathbf{e}_x$ ，因而 $\mathbf{X}_t \in \mathbb{R}^{n \times N}$ ，且 $\mathbf{X}_t \mathbf{W}_{xh}$ 选出第 x 行。

8.5.2 初始化模型参数

隐藏宽度 h 与词表大小 N 决定

$$\mathbf{W}_{xh} \in \mathbb{R}^{N \times h}, \mathbf{W}_{hh} \in \mathbb{R}^{h \times h}, \mathbf{W}_{hq} \in \mathbb{R}^{h \times N}$$

及偏置。输入层与输出层共用同一词表，故两边维度都是 N 。

8.5.3 循环神经网络模型

初始隐状态 $\mathbf{H}_0 = \mathbf{0} \in \mathbb{R}^{n \times h}$ 。一个时间步是

$$\begin{aligned}\mathbf{H}_t &= \tanh(\mathbf{X}_t \mathbf{W}_{xh} + \mathbf{H}_{t-1} \mathbf{W}_{hh} + \mathbf{b}_h), \\ \mathbf{O}_t &= \mathbf{H}_t \mathbf{W}_{hq} + \mathbf{b}_q.\end{aligned}$$

最外层按 $t = 1, \dots, n_{\text{steps}}$ 循环，把全部 $\mathbf{O}_t$ 拼成输出。

8.5.4 预测

给定前缀。预热期只更新 h_t 不采样；之后每步取

$$\hat{x}_{t+1} = \operatorname{argmax}_x P(x \mid h_t)$$

或按该分布抽样，再把 $\hat{x}_{t+1}$ 作为下一输入。

8.5.5 梯度裁剪

若 f 是 L 利普希茨的，则

$$|f(\mathbf{x}) - f(\mathbf{y})| \leq L \|\mathbf{x} - \mathbf{y}\|.$$

沿梯度走一步后

$$|f(\mathbf{x}) - f(\mathbf{x} - \eta \mathbf{g})| \leq L\eta \|\mathbf{g}\|.$$

阈值 θ 下的裁剪

$$\mathbf{g} \leftarrow \min\left(1, \frac{\theta}{\|\mathbf{g}\|}\right) \mathbf{g}$$

把更新幅度限制在 θ 以内，用来抑制爆炸，但不修复消失。

8.5.6 训练

一个迭代周期内：随机采样则每批量重置 h ；顺序分区则延续 h 但截断梯度。更新前执行上式裁剪，并用困惑度汇总不同长度序列的交叉熵。

8.5.7 小结

从零实现把

$$(\mathbf{e}_{x_t}, h_{t-1}) \mapsto (h_t, \mathbf{O}_t)$$

写成可训练循环核，预热提供可用的 h ，裁剪保证 $\|\mathbf{g}\|$ 有上界。

8.5.8 练习

核验独热与嵌入层第一层等价、去掉裁剪后 $\|g\|$ 是否发散，以及用 $P(x)^\alpha$ 抽样如何改变生成分布。

8.6 循环神经网络的简洁实现

同一套映射改由框架中的循环层一次性求值，输出层仍需单独写出。

8.6.1 定义模型

循环层接收 $\mathbf{X}_{1:T}$ 与 $\mathbf{H}_0$ ，返回

$$(\mathbf{H}_{1:T}, \mathbf{H}_T) = \text{RNN}(\mathbf{X}_{1:T}, \mathbf{H}_0).$$

这里的 $\mathbf{H}_{1:T}$ 还不是词表上的 logits；完整模型是

$$\mathbf{O}_t = \mathbf{H}_t \mathbf{W}_{hq} + \mathbf{b}_q.$$

多层时，上一层的 $\mathbf{H}_t^{(\ell-1)}$ 作为下一层输入。

8.6.2 训练与预测

随机初始化时前缀续写接近均匀乱猜。训练目标、裁剪与困惑度与从零实现相同，只是 RNN 的逐步递推由编译后的算子完成。

8.6.3 小结

简洁实现不改变

$$\mathbf{H}_t = \phi(\mathbf{X}_t \mathbf{W}_{xh} + \mathbf{H}_{t-1} \mathbf{W}_{hh} + \mathbf{b}_h),$$

只把该递包装成层；词表上的 $\mathbf{O}_t$ 仍由单独的线性映射给出。

8.6.4 练习

核验增加隐藏层数是否仍收敛，以及用同一循环核拟合序列节中的自回归窗口 f 。

8.7 通过时间反向传播

损失对共享参数 w_h 的梯度必须沿 $h_T \rightarrow h_{T-1} \rightarrow \cdots \rightarrow h_1$ 反传，这称为通过时间反向传播（BPTT）。

8.7.1 循环神经网络的梯度分析

一步映射与逐步损失为

$$h_t = f(x_t, h_{t-1}, w_h),$$

$$o_t = g(h_t, w_o),$$

$$L = \frac{1}{T} \sum_{t=1}^T l(y_t, o_t).$$

对 w_h 求偏导：

$$\frac{\partial L}{\partial w_h} = \frac{1}{T} \sum_{t=1}^T \frac{\partial l(y_t, o_t)}{\partial o_t} \frac{\partial g(h_t, w_o)}{\partial h_t} \frac{\partial h_t}{\partial w_h}.$$

隐状态本身对参数的依赖是递推

$$\frac{\partial h_t}{\partial w_h} = \frac{\partial f(x_t, h_{t-1}, w_h)}{\partial w_h} + \frac{\partial f(x_t, h_{t-1}, w_h)}{\partial h_{t-1}} \frac{\partial h_{t-1}}{\partial w_h}.$$

令

$$\begin{aligned} a_t &= \frac{\partial h_t}{\partial w_h}, & b_t &= \frac{\partial f(x_t, h_{t-1}, w_h)}{\partial w_h}, \\ c_t &= \frac{\partial f(x_t, h_{t-1}, w_h)}{\partial h_{t-1}}, \end{aligned}$$

则 $a_t = b_t + c_t a_{t-1}$ ，展开为

$$a_t = b_t + \sum_{i=1}^{t-1} \left(\prod_{j=i+1}^t c_j \right) b_i.$$

代回原变量，

$$\begin{aligned} \frac{\partial h_t}{\partial w_h} &= \frac{\partial f(x_t, h_{t-1}, w_h)}{\partial w_h} \\ &+ \sum_{i=1}^{t-1} \left(\prod_{j=i+1}^t \frac{\partial f(x_j, h_{j-1}, w_h)}{\partial h_{j-1}} \right) \frac{\partial f(x_i, h_{i-1}, w_h)}{\partial w_h}. \end{aligned}$$

乘积项决定梯度爆炸或消失。

8.7.1.1 完全计算

把上式中 $i = 1$ 到 $t-1$ 的和全部保留，得到精确 BPTT。计算量随 t 增长，且 $\prod c_j$ 对初值极度敏感，实践中几乎不用。

8.7.1.2 截断时间步

在 τ 步后切断求和，即把 $\partial h_{t-\tau}/\partial w_h$ 视为常数。这给出真实梯度的近似，使模型更关注短程依赖，并对应实现里对隐状态的分离。

8.7.1.3 随机截断

用伯努利变量 ξ_t 随机决定是否把梯度传向 $t-1$ ：

$$z_t = \frac{\partial f(x_t, h_{t-1}, w_h)}{\partial w_h} + \xi_t \frac{\partial f(x_t, h_{t-1}, w_h)}{\partial h_{t-1}} \frac{\partial h_{t-1}}{\partial w_h}.$$

$\mathbb{E}[z_t]$ 可与完全梯度一致，但方差更大。

8.7.1.4 比较策略

随机截断把序列切成不等长片段；规则截断切成等长窗口；完全反传保留整条链。规则截断在偏差、方差与计算之间最常用。

8.7.2 通过时间反向传播的细节

去掉激活后的线性循环为

$$\mathbf{h}_t = \mathbf{W}_{hx} \mathbf{x}_t + \mathbf{W}_{hh} \mathbf{h}_{t-1},$$

$$\mathbf{o}_t = \mathbf{W}_{qh} \mathbf{h}_t,$$

$$L = \frac{1}{T} \sum_{t=1}^T l(\mathbf{o}_t, y_t).$$

输出梯度

$$\frac{\partial L}{\partial \mathbf{o}_t} = \frac{\partial l(\mathbf{o}_t, y_t)}{T \cdot \partial \mathbf{o}_t} \in \mathbb{R}^q.$$

输出权重

$$\frac{\partial L}{\partial \mathbf{W}_{qh}} = \sum_{t=1}^T \frac{\partial L}{\partial \mathbf{o}_t} \mathbf{h}_t^\top.$$

终端隐状态

$$\frac{\partial L}{\partial \mathbf{h}_T} = \mathbf{W}_{qh}^\top \frac{\partial L}{\partial \mathbf{o}_T}.$$

对 $t < T$ 反向递推

$$\frac{\partial L}{\partial \mathbf{h}_t} = \mathbf{W}_{hh}^\top \frac{\partial L}{\partial \mathbf{h}_{t+1}} + \mathbf{W}_{qh}^\top \frac{\partial L}{\partial \mathbf{o}_t}.$$

展开后

$$\frac{\partial L}{\partial \mathbf{h}_t} = \sum_{i=t}^T (\mathbf{W}_{hh}^\top)^{T-i} \mathbf{W}_{qh}^\top \frac{\partial L}{\partial \mathbf{o}_{T+t-i}}.$$

于是

$$\begin{aligned} \frac{\partial L}{\partial \mathbf{W}_{hx}} &= \sum_{t=1}^T \frac{\partial L}{\partial \mathbf{h}_t} \mathbf{x}_t^\top, \\ \frac{\partial L}{\partial \mathbf{W}_{hh}} &= \sum_{t=1}^T \frac{\partial L}{\partial \mathbf{h}_t} \mathbf{h}_{t-1}^\top. \end{aligned}$$

$\mathbf{W}_{hh}^\top$ 的高次幂使特征值 $|\lambda| > 1$ 时爆炸、 $|\lambda| < 1$ 时消失。

8.7.3 小结

BPTT 的核心是

$$\frac{\partial h_t}{\partial w_h} = b_t + \sum_{i < t} \left(\prod_{j=i+1}^t c_j \right) b_i,$$

截断把乘积长度限制在 τ ； $\mathbf{W}_{hh}$ 的谱半径决定梯度尺度。

8.7.4 练习

核验对称矩阵 $\mathbf{M}$ 满足 $\mathbf{M}^k$ 的特征值为 λ_i^k ，以及 $\mathbf{W}_{hh}^{T-t}$ 与主导特征向量对齐如何对应爆炸方向。

第九章 现代循环神经网络

门控让隐状态学会何时写入、何时遗忘。GRU 与 LSTM 用可学习的门缓解长程梯度问题，再堆成深层或双向结构。编码器-解码器把变长输入映成变长输出，束搜索在词表上近似最大似然序列。

9.1 门控循环单元（GRU）

普通循环中 $\prod_j \partial h_j / \partial h_{j-1}$ 容易消失或爆炸。若早期词元必须影响很晚的预测，就需要能够长期保存 h 的机制；若中间词元无关，则应学会跳过更新。GRU 用重置门与更新门实现这两种行为。

9.1.1 门控隐状态

门的取值在 $(0, 1)$ 中，由当前输入与上一隐状态共同决定，因此何时重置、何时拷贝都可以学习。

9.1.1.1 重置门和更新门

对批量输入 $\mathbf{X}_t$ 与上一隐状态 $\mathbf{H}_{t-1}$ ，

$$\mathbf{R}_t = \sigma(\mathbf{X}_t \mathbf{W}_{xr} + \mathbf{H}_{t-1} \mathbf{W}_{hr} + \mathbf{b}_r),$$

$$\mathbf{Z}_t = \sigma(\mathbf{X}_t \mathbf{W}_{xz} + \mathbf{H}_{t-1} \mathbf{W}_{hz} + \mathbf{b}_z).$$

$\mathbf{R}_t$ 靠近 0 时清空历史， $\mathbf{Z}_t$ 靠近 1 时倾向原样保留 $\mathbf{H}_{t-1}$ 。

9.1.1.2 候选隐状态

重置后的历史再与当前输入混合：

$$\tilde{\mathbf{H}}_t = \tanh(\mathbf{X}_t \mathbf{W}_{xh} + (\mathbf{R}_t \odot \mathbf{H}_{t-1}) \mathbf{W}_{hh} + \mathbf{b}_h).$$

$\mathbf{R}_t = 1$ 时退化为普通候选隐状态； $\mathbf{R}_t = 0$ 时候选只看 $\mathbf{X}_t$ 。

9.1.1.3 隐状态

更新门在旧状态与候选之间插值：

$$\mathbf{H}_t = \mathbf{Z}_t \odot \mathbf{H}_{t-1} + (1 - \mathbf{Z}_t) \odot \tilde{\mathbf{H}}_t.$$

$\mathbf{Z}_t = 1$ 时 $\mathbf{H}_t = \mathbf{H}_{t-1}$ ，梯度可沿时间原路回传； $\mathbf{Z}_t = 0$ 时完全改写。

9.1.2 从零开始实现

门控递推与上一章字符语言模型相同，只是 h_t 的更新换成上面三式，输出层仍是 $\mathbf{O}_t = \mathbf{H}_t \mathbf{W}_{hq} + \mathbf{b}_q$ 。

9.1.2.1 初始化模型参数

高斯初始化权重、零偏置。需要实例化 $(\mathbf{W}_{xr}, \mathbf{W}_{hr}, \mathbf{b}_r)$ 、 $(\mathbf{W}_{xz}, \mathbf{W}_{hz}, \mathbf{b}_z)$ 、 $(\mathbf{W}_{xh}, \mathbf{W}_{hh}, \mathbf{b}_h)$ 以及输出层 $(\mathbf{W}_{hq}, \mathbf{b}_q)$ 。隐藏宽度仍由 h 给出。

9.1.2.2 定义模型

$\mathbf{H}_0 = \mathbf{0}$ 。逐步计算 $(\mathbf{R}_t, \mathbf{Z}_t, \tilde{\mathbf{H}}_t, \mathbf{H}_t, \mathbf{O}_t)$ ，参数在所有 t 上共享。映射与定义式逐项相同。

9.1.2.3 训练与预测

损失、裁剪、困惑度与前缀续写均沿用循环神经网络从零实现中的流程。训练后比较训练集困惑度以及给定前缀时的续写序列。

9.1.3 简洁实现

框架中的 GRU 层实现同一组门。前向仍是

$$(\mathbf{H}_{1:T}, \mathbf{H}_T) = \text{GRU}(\mathbf{X}_{1:T}, \mathbf{H}_0),$$

再接输出层；数值上应与从零实现一致，只是算子被编译。

9.1.4 小结

GRU 的状态更新

$$\mathbf{H}_t = \mathbf{Z}_t \odot \mathbf{H}_{t-1} + (1 - \mathbf{Z}_t) \odot \tilde{\mathbf{H}}_t$$

用 $\mathbf{Z}_t$ 保持长程信息、用 $\mathbf{R}_t$ 控制短程候选；两门全开或全关时分别覆盖拷贝与普通 RNN。

9.1.5 练习

核验仅用 $x_{t'}$ 预测更晚输出时 $\mathbf{R}, \mathbf{Z}$ 的理想取值，以及只保留一扇门时递推如何退化。

9.2 长短期记忆网络 (LSTM)

LSTM 在隐状态之外再设记忆元 $\mathbf{C}_t$ ，用三扇门分别控制写入、遗忘与读出。

9.2.1 门控记忆元

记忆元与 $\mathbf{H}_t$ 同形，但不直接进入输出层。输入门决定吸收候选，遗忘门决定丢弃旧记忆，输出门决定暴露多少记忆给 h_t 。

9.2.1.1 输入门、忘记门和输出门

$$\begin{aligned} \mathbf{I}_t &= \sigma(\mathbf{X}_t \mathbf{W}_{xi} + \mathbf{H}_{t-1} \mathbf{W}_{hi} + \mathbf{b}_i), \\ \mathbf{F}_t &= \sigma(\mathbf{X}_t \mathbf{W}_{xf} + \mathbf{H}_{t-1} \mathbf{W}_{hf} + \mathbf{b}_f), \\ \mathbf{O}_t &= \sigma(\mathbf{X}_t \mathbf{W}_{xo} + \mathbf{H}_{t-1} \mathbf{W}_{ho} + \mathbf{b}_o). \end{aligned}$$

三者均在 $(0, 1)$ 中逐元素取值。

9.2.1.2 候选记忆元

$$\tilde{\mathbf{C}}_t = \tanh(\mathbf{X}_t \mathbf{W}_{xc} + \mathbf{H}_{t-1} \mathbf{W}_{hc} + \mathbf{b}_c).$$

$\tanh$ 把候选限制在 $(-1, 1)$ 。

9.2.1.3 记忆元

$$\mathbf{C}_t = \mathbf{F}_t \odot \mathbf{C}_{t-1} + \mathbf{I}_t \odot \tilde{\mathbf{C}}_t.$$

$\mathbf{F}_t = \mathbf{1}, \mathbf{I}_t = \mathbf{0}$ 时记忆原样传递，梯度可沿 $\mathbf{C}$ 传播。

9.2.1.4 隐状态

$$\mathbf{H}_t = \mathbf{O}_t \odot \tanh(\mathbf{C}_t).$$

只有 $\mathbf{H}_t$ 进入 $\mathbf{O}_t^{\text{out}} = \mathbf{H}_t \mathbf{W}_{hq} + \mathbf{b}_q$; $\mathbf{C}_t$ 是内部状态。

9.2.2 从零开始实现

隐状态现在是二元组 $(\mathbf{H}_t, \mathbf{C}_t)$, 其余训练回路不变。

9.2.2.1 初始化模型参数

除三扇门与候选记忆的权重偏置外, 输出层维度仍等于词表大小。权重取标准差 0.01 的高斯, 偏置为零。

9.2.2.2 定义模型

初始化 $(\mathbf{H}_0, \mathbf{C}_0) = (\mathbf{0}, \mathbf{0})$ 。逐步计算 $(\mathbf{I}_t, \mathbf{F}_t, \mathbf{O}_t, \tilde{\mathbf{C}}_t, \mathbf{C}_t, \mathbf{H}_t)$, 仅把 $\mathbf{H}_t$ 映到词表 logits。

9.2.2.3 训练和预测

与 GRU 相同的困惑度目标与前缀生成。记忆元在预测时随 $\mathbf{H}$ 一起递推, 但不直接解码。

9.2.3 简洁实现

$$((\mathbf{H}_{1:T}, \mathbf{C}_T), \mathbf{H}_T) = \text{LSTM}(\mathbf{X}_{1:T}, (\mathbf{H}_0, \mathbf{C}_0)).$$

这是带状态控制的隐变量自回归模型; 长程依赖仍使逐步展开代价随 T 增长。

9.2.4 小结

LSTM 用

$$\mathbf{C}_t = \mathbf{F}_t \odot \mathbf{C}_{t-1} + \mathbf{I}_t \odot \tilde{\mathbf{C}}_t, \quad \mathbf{H}_t = \mathbf{O}_t \odot \tanh(\mathbf{C}_t)$$

把长期记忆与对外隐状态分开, 从而缓解梯度消失。

9.2.5 练习

核验 $\tilde{C}_t$ 已经过 $\tanh$ 后输出为何再套 $\tanh$ ，以及在相同 h 下 GRU、LSTM 与普通 RNN 的逐步乘法次数。

9.3 深度循环神经网络

单层循环的 ϕ 可能不够灵活。把若干循环层堆叠，使不同层刻画不同时间尺度。

9.3.1 函数依赖关系

令 $\mathbf{H}_t^{(0)} = \mathbf{X}_t$ 。第 $\ell = 1, \dots, L$ 层

$$\mathbf{H}_t^{(\ell)} = \phi_\ell(\mathbf{H}_t^{(\ell-1)} \mathbf{W}_{xh}^{(\ell)} + \mathbf{H}_{t-1}^{(\ell)} \mathbf{W}_{hh}^{(\ell)} + \mathbf{b}_h^{(\ell)}).$$

顶层再接到输出

$$\mathbf{O}_t = \mathbf{H}_t^{(L)} \mathbf{W}_{hq} + \mathbf{b}_q.$$

于是信息同时沿时间传到 $\mathbf{H}_{t+1}^{(\ell)}$ ，并沿深度传到 $\mathbf{H}_t^{(\ell+1)}$ 。

9.3.2 简洁实现

以 LSTM 为例，仅增加层数 L 。词表大小决定输入输出维，隐藏宽度仍取 h ，层间按上式串联。GRU 或普通 RNN 只需更换 ϕ_ℓ 所对应的单元。

9.3.3 训练与预测

多层逐步展开使每步代价约为单层的 L 倍，收敛对学习率与裁剪更敏感。预测时逐层递推全部 $\mathbf{H}_t^{(\ell)}$ 。

9.3.4 小结

深度循环的依赖是

$$\mathbf{H}_t^{(\ell)} = \phi_\ell(\mathbf{H}_t^{(\ell-1)} \mathbf{W}_{xh}^{(\ell)} + \mathbf{H}_{t-1}^{(\ell)} \mathbf{W}_{hh}^{(\ell)} + \mathbf{b}_h^{(\ell)}),$$

同一套门控单元可以替换 ϕ_ℓ 。

9.3.5 练习

核验两层从零展开时 $\mathbf{H}_t^{(1)}$ 如何作为 $\mathbf{H}_t^{(2)}$ 的输入，以及用 GRU 替换 LSTM 后困惑度与逐步代价如何变化。

9.4 双向循环神经网络

单向循环只能看过去。填空、标注等任务需要未来上下文，因此再设一条反向循环。

9.4.1 隐马尔可夫模型中的动态规划

隐状态与观测的联合分布为

$$P(x_1, \dots, x_T, h_1, \dots, h_T) = \prod_{t=1}^T P(h_t | h_{t-1}) P(x_t | h_t),$$

其中 $P(h_1 | h_0) = P(h_1)$ 。边际化全部 h 时，前向量

$$\pi_{t+1}(h_{t+1}) = \sum_{h_t} \pi_t(h_t) P(x_t | h_t) P(h_{t+1} | h_t)$$

把过去的贡献压缩成关于 h_{t+1} 的函数，最终

$$P(x_1, \dots, x_T) = \sum_{h_T} \pi_T(h_T) P(x_T | h_T).$$

后向量

$$\rho_{t-1}(h_{t-1}) = \sum_{h_t} P(h_t | h_{t-1}) P(x_t | h_t) \rho_t(h_t)$$

压缩未来。两者合起来给出缺省位置的条件

$$P(x_j | x_{-j}) \propto \sum_{h_j} \pi_j(h_j) \rho_j(h_j) P(x_j | h_j).$$

9.4.2 双向模型

用一条前向 RNN 扮演 π ，一条后向 RNN 扮演 ρ ，在每个 t 拼接两边的隐状态。

9.4.2.1 定义

$$\vec{\mathbf{H}}_t = \phi(\mathbf{X}_t \mathbf{W}_{xh}^{(f)} + \overleftarrow{\mathbf{H}}_{t-1} \mathbf{W}_{hh}^{(f)} + \mathbf{b}_h^{(f)}),$$

$$\overleftarrow{\mathbf{H}}_t = \phi(\mathbf{X}_t \mathbf{W}_{xh}^{(b)} + \overleftarrow{\mathbf{H}}_{t+1} \mathbf{W}_{hh}^{(b)} + \mathbf{b}_h^{(b)}).$$

拼接 $\mathbf{H}_t = [\overrightarrow{\mathbf{H}}_t; \overleftarrow{\mathbf{H}}_t]$ 后

$$\mathbf{O}_t = \mathbf{H}_t \mathbf{W}_{hq} + \mathbf{b}_q.$$

9.4.2.2 模型的计算代价及其应用

每个 t 的输出同时依赖 $x_1, \dots, x_T$, 因此不能用于“只根据过去预测下一个词元”。前向需要跑完反向递归, 反向传播的链大约加倍, 训练代价高。适用场合是编码整句或双向上下文标注。

9.4.3 双向循环神经网络的错误应用

若把双向模型当作语言模型训练, 训练时未来词元可见, 测试时不可见, 条件分布 $P(x_t | x_{<t})$ 被错误地换成 $P(x_t | x_{\neq t})$, 续写会崩溃。

9.4.4 小结

双向更新

$$\mathbf{O}_t = [\overrightarrow{\mathbf{H}}_t; \overleftarrow{\mathbf{H}}_t] \mathbf{W}_{hq} + \mathbf{b}_q$$

对应 HMM 中的前向-后向量; 它编码双向上下文, 而不能当作因果语言模型。

9.4.5 练习

核验两个方向隐藏宽度不同时 $\mathbf{H}_t$ 的拼接维, 以及如何用双向上下文表示多义词在句中的向量。

9.5 机器翻译与数据集

机器翻译把源语言序列映到目标语言序列, 是变长到变长的序列转换。后续编码器-解码器都在这对平行语料上训练。

9.5.1 下载和预处理数据集

英-法句对经制表符分隔。预处理把不间断空格换成普通空格、字母小写, 并在单词与标点之间插入空格, 得到可切分的字符串对 (s, t) 。

9.5.2 词元化

与字符级语言模型不同，这里按单词（及标点）切分。第 i 对变成

$$\text{source}^{(i)} = (x_1, \dots, x_T), \quad \text{target}^{(i)} = (y_1, \dots, y_{T'}).$$

多数句对的词元数小于 20。

9.5.3 词表

源、目标各建一个词表。单词级使 $|\mathcal{V}|$ 远大于字符级，故把频次小于 2 的项并入 $\langle \text{unk} \rangle$ ，并加入 $\langle \text{pad} \rangle$ 、 $\langle \text{bos} \rangle$ 、 $\langle \text{eos} \rangle$ 。

9.5.4 加载数据集

句长不同。截断与填充把每条序列补/切到固定 n_{steps} ，同时记录有效长度 $T_{\text{valid}} \leq n_{\text{steps}}$ ，以便后面的损失与注意力忽略填充。

9.5.5 训练模型

数据迭代器每次返回一个小批量的源下标、源有效长度、目标下标与目标有效长度，以及两个词表。这给出监督学习对 $(\mathbf{X}, \mathbf{Y})$ 。

9.5.6 小结

平行语料经过词元化与填充后，每个样本是长度同为 n_{steps} 的整数对 $(\mathbf{x}, \mathbf{y})$ ，低频率词并入未知词元以控制 $|\mathcal{V}|$ 。

9.5.7 练习

核验句对数如何改变两个词表大小，以及无空格语言上单词切分是否仍有良定义的 $\mathcal{Z}$ 。

9.6 编码器-解码器架构

输入长度 T 与输出长度 T' 都可以变。编码器把变长输入压成固定形状的状态，解码器再把该状态展开成变长输出。

9.6.1 编码器

编码器是映射

$$\text{Enc} : \mathcal{X}^T \rightarrow \mathcal{C}, \quad \mathbf{X} \mapsto \mathbf{c}.$$

$\mathcal{C}$ 的形状不随 T 改变。任何具体网络只要实现这一接口即可。

9.6.2 解码器

解码器在每步消费已经生成的词元与编码状态：

$$\text{Dec} : \mathcal{C} \times \mathcal{Y}^{t'-1} \rightarrow \mathcal{Y}, \quad (\mathbf{c}, y_{1:t'-1}) \mapsto y_{t'}.$$

`init_state` 把编码器输出转换成解码器的初始状态，必要时使用源序列有效长度。

9.6.3 合并编码器和解码器

前向先算 $\mathbf{c} = \text{Enc}(\mathbf{X})$ ，再逐步

$$y_{t'} = \text{Dec}(\mathbf{c}, y_{1:t'-1}), \quad t' = 1, \dots, T'.$$

具有隐状态的网络自然适合携带 $\mathbf{c}$ 。下一节用两个循环网络实现该架构。

9.6.4 小结

编码器—解码器把变长到变长写成

$$\mathbf{c} = \text{Enc}(\mathbf{X}), \quad y_{1:T'} = \text{Dec}(\mathbf{c}),$$

因此适用于机器翻译这类序列转换。

9.6.5 练习

核验 `Enc` 与 `Dec` 不必同一类网络，以及哪些其他变长到变长任务可用同一接口。

9.7 序列到序列学习 (seq2seq)

用循环神经网络实现上一节的两个映射：编码器消费 $\mathbf{x}_{1:T}$ ，解码器生成 $\mathbf{y}_{1:T'}$ 。

9.7.1 编码器

逐步

$$\mathbf{h}_t = f(\mathbf{x}_t, \mathbf{h}_{t-1}).$$

上下文变量把全部隐状态汇总为固定向量

$$\mathbf{c} = q(\mathbf{h}_1, \dots, \mathbf{h}_T).$$

最常见的取法是 q 只保留 $\mathbf{h}_T$ ，或取多层最终隐状态的拼接。

9.7.2 解码器

解码器隐状态

$$\mathbf{s}_{t'} = g(y_{t'-1}, \mathbf{c}, \mathbf{s}_{t'-1}),$$

再由 $\mathbf{s}_{t'}$ 产生 $P(y_{t'} | y_{1:t'-1}, \mathbf{c})$ 。训练时 $y_{t'-1}$ 来自真实标签，预测时来自上一时刻的输出。

9.7.3 损失函数

每步在词表上做 softmax 交叉熵。填充位置不应计入损失。有效长度掩码把越界项置零，等价于

$$L = \frac{1}{\sum_{t'} \mathbb{I}\{t' \leq T'_{\text{valid}}\}} \sum_{t'=1}^{n_{\text{steps}}} \mathbb{I}\{t' \leq T'_{\text{valid}}\} \ell(y_{t'}, \hat{y}_{t'}).$$

9.7.4 训练

解码器输入是 `<bos>` 与去掉 `<eos>` 的真实目标序列拼接，即强制教学。也可以改用模型自己的 $\hat{y}_{t'-1}$ 。编码器可多层，解码器初始 $\mathbf{s}_0$ 由 $\mathbf{c}$ 给出。

9.7.5 预测

从 `<bos>` 开始，每步把 $\hat{y}_{t'}$ 送回解码器，直到输出 `<eos>` 或达到最大长度。这是贪心生成；下一节讨论束搜索。

9.7.6 预测序列的评估

BLEU 比较预测与标签的 n 元匹配率 p_n ，并惩罚过短输出：

$$\exp\left(\min\left(0, 1 - \frac{\text{len}_{\text{label}}}{\text{len}_{\text{pred}}}\right)\right) \prod_{n=1}^k p_n^{1/2^n}.$$

9.7.7 小结

seq2seq 用

$$\mathbf{c} = q(\mathbf{h}_{1:T}), \quad \mathbf{s}_{t'} = g(y_{t'-1}, \mathbf{c}, \mathbf{s}_{t'-1})$$

实现编码器-解码器；训练用强制教学，评价用 BLEU。

9.7.8 练习

核验去掉掩码后填充位置如何污染 L ，以及编码器与解码器隐藏宽度不同时 $\mathbf{s}_0$ 应如何由 $\mathbf{c}$ 初始化。

9.8 束搜索

解码要在词表 $\mathcal{Y}$ 上找一条序列最大化 $P(y_1, \dots, y_{T'} | \mathbf{c})$ 。逐步取最大是贪心；枚举全部序列是穷举；束搜索在二者之间保留 k 条候选。

9.8.1 贪心搜索

每步

$$y_{t'} = \operatorname{argmax}_{y \in \mathcal{Y}} P(y | y_1, \dots, y_{t'-1}, \mathbf{c}).$$

计算量 $\mathcal{O}(|\mathcal{Y}|T')$ ，但不能改正早期错误，整体不必最优。

9.8.2 穷举搜索

枚举 $\mathcal{Y}^{T'}$ 中全部序列并取联合概率最大者。计算量 $\mathcal{O}(|\mathcal{Y}|^{T'})$ 。例如 $|\mathcal{Y}| = 10^4, T' = 10$ 时约 10^{40} 条，不可行。

9.8.3 束搜索

束宽为 k 时，每步只保留条件概率最高的 k 条前缀。例如 $k = 2$ 时，若第一步留下 A, C ，则第二步比较

$$P(A, y_2 | \mathbf{c}) = P(A | \mathbf{c})P(y_2 | A, \mathbf{c}),$$

$$P(C, y_2 | \mathbf{c}) = P(C | \mathbf{c})P(y_2 | C, \mathbf{c}),$$

再留下两条；第三步同理，

$$P(A, B, y_3 | \mathbf{c}) = P(A, B | \mathbf{c})P(y_3 | A, B, \mathbf{c}),$$

$$P(C, E, y_3 \mid \mathbf{c}) = P(C, E \mid \mathbf{c})P(y_3 \mid C, E, \mathbf{c}).$$

最终在候选完整序列上用长度归一化分数

$$\frac{1}{L^\alpha} \log P(y_1, \dots, y_L \mid \mathbf{c}) = \frac{1}{L^\alpha} \sum_{t'=1}^L \log P(y_{t'} \mid y_{1:t'-1}, \mathbf{c})$$

选出输出。 $k = 1$ 退化为贪心； $k = |\mathcal{Y}|$ 恢复穷举。

9.8.4 小结

束搜索用宽度 k 在

$$\max_{y_{1:L}} P(y_{1:L} \mid \mathbf{c})$$

的精度与 $\mathcal{O}(k|\mathcal{Y}|T')$ 的代价之间折中。

9.8.5 练习

核验穷举是否为 $k = |\mathcal{Y}|$ 的束搜索，以及 k 如何改变机器翻译的速度与 BLEU。

第十章 注意力机制

注意力把查询与键的相似度变成权，再对值加权求和。从 Nadaraya–Watson 核回归出发，经过加性与缩放点积评分、多头与自注意力，得到完全基于注意力的 Transformer：线性投射给出 Q, K, V ，再计算 $\text{softmax}(QK^\top/\sqrt{d})$ 。

10.1 注意力提示

注意力是有限资源。神经网络若只有汇聚或全连接，选择不依赖当前任务；加入查询后，权可以随任务改变。

10.1.1 生物学中的注意力提示

非自主性提示由刺激的突出性驱动，例如场景中唯一的红色物体会抓住注视。自主性提示由当前目标驱动，例如要读论文时目光转向纸张。二者共同决定焦点。

10.1.2 查询、键和值

查询 q 对应自主提示。键 k_i 对应非自主提示，值 v_i 对应感官输入。注意力汇聚是

$$f(q, (k_1, v_1), \dots, (k_m, v_m)) = \sum_{i=1}^m \alpha(q, k_i) v_i.$$

没有查询时，权与任务无关，退化为平均或最大汇聚。

10.1.3 注意力的可视化

权矩阵的条目是 $\alpha(q_i, k_j)$ 。对角情形下

$$\alpha(q_i, k_j) = [i = j],$$

即只把与查询相同的键所对应的值取出。后续各节都用同一张权矩阵观察汇聚偏向。

10.1.4 小结

注意力与汇聚的差别在于查询：权 $\alpha(\mathbf{q}, \mathbf{k}_i)$ 由查询-键交互决定，再对成对的值加权。

10.1.5 练习

核验翻译解码中查询、键、值分别对应哪些向量，以及行归一化后的随机矩阵是否构成合法注意力权。

10.2 注意力汇聚：Nadaraya-Watson 核回归

Nadaraya-Watson 核回归是完整的注意力实例：查询是自变量 x ，键是训练输入 x_i ，值是训练输出 y_i 。

10.2.1 生成数据集

$$y_i = 2 \sin(x_i) + x_i^{0.8} + \epsilon,$$

ϵ 为噪声。回归目标是估计 $f(x) = \mathbb{E}[y | x]$ 。

10.2.2 平均汇聚

忽略查询，对全部值取平均

$$f(x) = \frac{1}{n} \sum_{i=1}^n y_i.$$

这是与 x 无关的常数估计。

10.2.3 非参数注意力汇聚

核 K 把查询与键的距离变成权：

$$f(x) = \sum_{i=1}^n \frac{K(x - x_i)}{\sum_{j=1}^n K(x - x_j)} y_i = \sum_{i=1}^n \alpha(x, x_i) y_i.$$

高斯核

$$K(u) = \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{u^2}{2}\right)$$

给出

$$\begin{aligned} f(x) &= \sum_{i=1}^n \frac{\exp(-\frac{1}{2}(x-x_i)^2)}{\sum_{j=1}^n \exp(-\frac{1}{2}(x-x_j)^2)} y_i \\ &= \sum_{i=1}^n \text{softmax}(-\frac{1}{2}(x-x_i)^2) y_i. \end{aligned}$$

10.2.4 带参数注意力汇聚

引入可学习带宽 w ，距离改成 $(x-x_i)w$ ：

$$f(x) = \sum_{i=1}^n \text{softmax}(-\frac{1}{2}((x-x_i)w)^2) y_i.$$

$|w|$ 越大，权越集中在最近的键上。

10.2.4.1 批量矩阵乘法

形状 (n, a, b) 与 (n, b, c) 的批量乘满足

$$(XY)_n = \mathbf{X}_n \mathbf{Y}_n \in \mathbb{R}^{a \times c}.$$

注意力里常用它一次算完整批查询对全部键的评分。

10.2.4.2 定义模型

把训练对 $\{(x_i, y_i)\}$ 作为键-值，查询为 x 。评分向量经 softmax 后与值向量做批量乘，实现上一段的 $f(x)$ 。

10.2.4.3 训练

平方损失加随机梯度更新 w 。每个样本的查询不与自身键配对，以免平凡自匹配。学得的 $|w|$ 较大时，权热图在邻近区域更尖锐，拟合曲线也更不平滑。

10.2.5 小结

Nadaraya-Watson 把回归写成

$$f(x) = \sum_i \alpha(x, x_i) y_i,$$

非参数核与带参数核只差评分函数是否含 w 。

10.2.6 练习

核验增大 n 是否改进非参数估计，以及 w 如何改变 $\alpha(x, x_i)$ 的尖锐程度。

10.3 注意力评分函数

一般地，

$$f(\mathbf{q}, (\mathbf{k}_1, \mathbf{v}_1), \dots, (\mathbf{k}_m, \mathbf{v}_m)) = \sum_{i=1}^m \alpha(\mathbf{q}, \mathbf{k}_i) \mathbf{v}_i \in \mathbb{R}^v,$$

$$\alpha(\mathbf{q}, \mathbf{k}_i) = \text{softmax}(a(\mathbf{q}, \mathbf{k}_i)) = \frac{\exp(a(\mathbf{q}, \mathbf{k}_i))}{\sum_{j=1}^m \exp(a(\mathbf{q}, \mathbf{k}_j))}.$$

不同的评分 a 给出不同的汇聚。

10.3.1 掩蔽 softmax 操作

填充位置不应进入汇聚。有效长度之外的评分置为 $-\infty$ ，再做 softmax，于是对应权为 0，加权和只落在合法键上。

10.3.2 加性注意力

查询与键维数可以不同。令

$$a(\mathbf{q}, \mathbf{k}) = \mathbf{w}_v^\top \tanh(\mathbf{W}_q \mathbf{q} + \mathbf{W}_k \mathbf{k}) \in \mathbb{R}.$$

这是一层隐层宽度为 h 的加性评分，适用于编码器-解码器中两边维数不一致的情形。

10.3.3 缩放点积注意力

当 $\mathbf{q}, \mathbf{k} \in \mathbb{R}^d$ 时，

$$a(\mathbf{q}, \mathbf{k}) = \frac{\mathbf{q}^\top \mathbf{k}}{\sqrt{d}}.$$

$\sqrt{d}$ 抵消点积方差随 d 线性增长，避免 softmax 进入饱和。批量形式为

$$\text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d}}\right)\mathbf{V} \in \mathbb{R}^{n \times v},$$

其中 $\mathbf{Q} \in \mathbb{R}^{n \times d}$, $\mathbf{K} \in \mathbb{R}^{m \times d}$, $\mathbf{V} \in \mathbb{R}^{m \times v}$ 。

10.3.4 小结

汇聚总是值的加权平均；维数不同用加性评分，维数相同时缩放点积 $\mathbf{q}^\top \mathbf{k} / \sqrt{d}$ 更省计算。

10.3.5 练习

核验改键之后加性与点积权是否仍重合，以及同维时求和评分与点积的几何差异。

10.4 Bahdanau 注意力

seq2seq 里每个解码步都使用同一个 $\mathbf{c}$ 。Bahdanau 注意力让上下文随解码状态变化，在源隐状态上做加性汇聚。

10.4.1 模型

解码时刻 t' 的上下文为

$$\mathbf{c}_{t'} = \sum_{t=1}^T \alpha(\mathbf{s}_{t'-1}, \mathbf{h}_t) \mathbf{h}_t,$$

其中 α 由加性评分给出，查询是上一解码状态 $\mathbf{s}_{t'-1}$ ，键和值都是编码器隐状态 $\mathbf{h}_t$ 。随后

$$\mathbf{s}_{t'} = g(y_{t'-1}, \mathbf{c}_{t'}, \mathbf{s}_{t'-1}).$$

10.4.2 定义注意力解码器

解码器状态由三部分初始化：全部时间步的顶层 $\mathbf{h}_{1:T}$ （作键和值）、编码器各层最终隐状态（作 $\mathbf{s}_0$ ）、源有效长度（供掩蔽 softmax）。每步用当前 $\mathbf{s}_{t'-1}$ 作查询，把注意力输出与输入嵌入拼接后送入循环单元。

10.4.3 训练

编码器仍是循环网络，解码器换成上一小节。训练目标与 BLEU 与 seq2seq 相同。学成后，每个查询在源位置上的权 $\alpha(\mathbf{s}_{t'-1}, \mathbf{h}_t)$ 随 t' 改变，即不同输出词对齐到不同的输入片段。

10.4.4 小结

Bahdanau 把固定 $\mathbf{c}$ 换成

$$\mathbf{c}_{t'} = \sum_t \alpha(\mathbf{s}_{t'-1}, \mathbf{h}_t) \mathbf{h}_t,$$

查询来自解码器，键值来自编码器，评分取加性注意力。

10.4.5 练习

核验把 GRU 换成 LSTM、把加性评分换成缩放点积后，对齐权与训练代价如何变化。

10.5 多头注意力

同一组 $(\mathbf{q}, \mathbf{k}, \mathbf{v})$ 经 h 组不同线性投射，得到 h 种汇聚，再拼起来。

10.5.1 模型

第 i 头

$$\mathbf{h}_i = f(\mathbf{W}_i^{(q)} \mathbf{q}, \mathbf{W}_i^{(k)} \mathbf{k}, \mathbf{W}_i^{(v)} \mathbf{v}) \in \mathbb{R}^{p_v},$$

f 为注意力汇聚。输出再经

$$\mathbf{W}_o \begin{bmatrix} \mathbf{h}_1 \\ \vdots \\ \mathbf{h}_h \end{bmatrix} \in \mathbb{R}^{p_o}.$$

各头落在不同子空间，可分别捕获短程与长程依赖。

10.5.2 实现

每个头用缩放点积注意力。取 $p_q = p_k = p_v = p_o/h$ ，并把各头的投射输出宽度设为 $p_q h = p_o$ ，即可把 h 个头排在批量维上一次算完。 p_o 对应隐藏宽度。映射在数学上仍是上一小节，只是张量轴被重排以便并行。

10.5.3 小结

多头把 h 组 $(\mathbf{W}_i^{(q)}, \mathbf{W}_i^{(k)}, \mathbf{W}_i^{(v)})$ 的汇聚拼起来再乘 $\mathbf{W}_o$ ，用同一查询-键-值的不同子空间表示。

10.5.4 练习

核验各头权矩阵是否学到不同对齐，以及去掉某头对预测的影响如何度量该头的重要性。

10.6 自注意力和位置编码

若查询、键、值来自同一组输入，就得到自注意力。它本身置换等变，必须另加位置编码才能感知次序。

10.6.1 自注意力

对 $\mathbf{x}_1, \dots, \mathbf{x}_n \in \mathbb{R}^d$,

$$\mathbf{y}_i = f(\mathbf{x}_i, (\mathbf{x}_1, \mathbf{x}_1), \dots, (\mathbf{x}_n, \mathbf{x}_n)) \in \mathbb{R}^d,$$

即第 i 个查询关注整组键-值。用线性投射写出矩阵形式：

$$\begin{aligned} \mathbf{Q} &= \mathbf{XW}_Q, & \mathbf{K} &= \mathbf{XW}_K, & \mathbf{V} &= \mathbf{XW}_V, \\ \text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) &= \text{softmax}\left(\frac{\mathbf{QK}^\top}{\sqrt{d}}\right)\mathbf{V}. \end{aligned}$$

10.6.2 比较卷积神经网络、循环神经网络和自注意力

把长 n 、宽 d 的序列映到同形序列。卷积核宽 k 时，计算复杂度 $\mathcal{O}(knd^2)$ ，顺序操作 $\mathcal{O}(1)$ ，最长路径 $\mathcal{O}(n/k)$ 。循环网络复杂度 $\mathcal{O}(nd^2)$ ，顺序操作 $\mathcal{O}(n)$ ，最长路径 $\mathcal{O}(n)$ 。自注意力复杂度 $\mathcal{O}(n^2d)$ ，顺序操作 $\mathcal{O}(1)$ ，最长路径 $\mathcal{O}(1)$ 。因此自注意力最利于并行与长程依赖，但序列很长时二次代价占优。

10.6.3 位置编码

绝对位置编码 $\mathbf{P} \in \mathbb{R}^{n \times d}$ 与输入相加： $\mathbf{X} \leftarrow \mathbf{X} + \mathbf{P}$ ，其中

$$\begin{aligned} p_{i,2j} &= \sin\left(\frac{i}{10000^{2j/d}}\right), \\ p_{i,2j+1} &= \cos\left(\frac{i}{10000^{2j/d}}\right). \end{aligned}$$

10.6.3.1 绝对位置信息

第 j 维的角频率随 j 下降，类似二进制中高比特翻转更慢。连续三角函数比比特编码更省空间，并使相邻位置的表示平滑变化。

10.6.3.2 相对位置信息

令 $\omega_j = 10000^{-2j/d}$ 。位移 δ 对应二维旋转：

$$\begin{aligned} \begin{bmatrix} \cos(\delta\omega_j) & \sin(\delta\omega_j) \\ -\sin(\delta\omega_j) & \cos(\delta\omega_j) \end{bmatrix} \begin{bmatrix} p_{i,2j} \\ p_{i,2j+1} \end{bmatrix} \\ = \begin{bmatrix} p_{i+\delta,2j} \\ p_{i+\delta,2j+1} \end{bmatrix}. \end{aligned}$$

因此相对位置可由绝对编码的线性变换得到。

10.6.4 小结

自注意力用同一 $\mathbf{X}$ 生成 $(\mathbf{Q}, \mathbf{K}, \mathbf{V})$ ；位置编码 $\mathbf{P}$ 把次序注入表示，否则模型对置换不变。

10.6.5 练习

核验只堆叠带位置编码的自注意力层可能出现的秩与长度外推问题，以及如何把 $\mathbf{P}$ 改成可学习矩阵。

10.7 Transformer

Transformer 是编码器–解码器，但子层只有多头注意力与位置前馈网络，不含卷积或循环。

10.7.1 模型

源序列与目标序列先做词嵌入，乘 $\sqrt{d}$ 后加上位置编码，再分别送入 N 层编码器与解码器。编码器每层是多头自注意力加前馈；解码器每层是掩蔽自注意力、编码器–解码器注意力与前馈。子层都包在残差与层规范化中。注意力内部仍是

$$\mathbf{Q} = \mathbf{X}\mathbf{W}_Q, \mathbf{K} = \mathbf{X}\mathbf{W}_K, \mathbf{V} = \mathbf{X}\mathbf{W}_V, \quad \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d}}\right)\mathbf{V}.$$

10.7.2 基于位置的前馈网络

同一两层感知机作用于每个位置：

$$\text{FFN}(\mathbf{x}) = \phi(\mathbf{x}\mathbf{W}_1 + \mathbf{b}_1)\mathbf{W}_2 + \mathbf{b}_2,$$

ϕ 取 ReLU。形状 (n_{batch}, n, d) 的张量只在最后一维上被映射到 n_{ffn} 再映回 d ；位置之间不混合。相同输入给出相同输出。

10.7.3 残差连接和层规范化

层规范化沿特征维标准化。对 $\mathbf{x} \in \mathbb{R}^d$,

$$\mu = \frac{1}{d} \sum_{i=1}^d x_i, \quad \sigma^2 = \frac{1}{d} \sum_{i=1}^d (x_i - \mu)^2,$$

$$\text{LN}(\mathbf{x}) = \gamma \odot \frac{\mathbf{x} - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta.$$

与批量规范化不同，它不依赖批量大小，适合变长序列。子层包装为

$$\text{AddNorm}(\mathbf{x}, \text{Sublayer}) = \text{LN}(\mathbf{x} + \text{Dropout}(\text{Sublayer}(\mathbf{x}))).$$

残差要求 Sublayer 输入输出同形状。

10.7.4 编码器

第 ℓ 个编码器块不改变形状：

$$\mathbf{X} \leftarrow \text{AddNorm}(\mathbf{X}, \text{MHAtt}(\mathbf{X}, \mathbf{X}, \mathbf{X})),$$

$$\mathbf{X} \leftarrow \text{AddNorm}(\mathbf{X}, \text{FFN}).$$

嵌入先乘 $\sqrt{d}$ 再加 $\mathbf{P}$ ，然后堆叠 N 个上述块。自注意力的 $\mathbf{Q}, \mathbf{K}, \mathbf{V}$ 都来自同一层输入。

10.7.5 解码器

每个解码器块三步：

$$\mathbf{Y} \leftarrow \text{AddNorm}(\mathbf{Y}, \text{MaskedMHAtt}(\mathbf{Y}, \mathbf{Y}, \mathbf{Y})),$$

$$\mathbf{Y} \leftarrow \text{AddNorm}(\mathbf{Y}, \text{MHAtt}(\mathbf{Y}, \mathbf{X}, \mathbf{X})),$$

$$\mathbf{Y} \leftarrow \text{AddNorm}(\mathbf{Y}, \text{FFN}).$$

第一项是掩蔽自注意力：时刻 t' 只能看见 $y_{\leq t'}$ ，以保持自回归。第二项是编码器–解码器注意力：查询来自解码器，键值来自编码器输出 $\mathbf{X}$ 。预测阶段词元逐个生成，训练阶段可用后续掩蔽一次算完全序列。

10.7.6 训练

编码器与解码器各 N 层、 h 头，在英–法数据上最小化带掩码的交叉熵。编码器自注意力权形状为 (层数, 头数, $n_{\text{steps}}, n_{\text{steps}}$)，查询与键都来自源序列；解码器交叉注意力的键则来自源、查询来自目标。

10.7.7 小结

Transformer 用

$$\text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right)V, \quad \text{FFN}, \quad \text{LN}(\boldsymbol{x} + \text{Sublayer}(\boldsymbol{x}))$$

堆成编码器-解码器；位置编码提供次序，掩蔽自注意力保持自回归。

10.7.8 练习

核验加深 N 对速度与 BLEU 的影响、长序列下 n^2 注意力的代价，以及语言模型应使用编码器、掩蔽解码器或二者的组合。

第十一章 优化算法

训练把经验风险写成可微标量 $f(\mathbf{x})$ ，再用迭代 $\mathbf{x} \leftarrow \mathbf{x} - \eta \widehat{\nabla} f(\mathbf{x})$ 更新参数。深度学习要的是泛化，优化只直接压训练误差；二者目标不同。下面先写凸性与梯度下降，再写随机、动量与自适应方法。

11.1 优化和深度学习

记训练目标为 $f(\mathbf{x})$ 。最大化等价于最小化 $-f$ 。优化求 f 的数值极小；深度学习还要求该极小对应的模型在未见数据上风险低。

11.1.1 优化的目标

经验风险与风险分别为

$$R_{\text{emp}}(\mathbf{x}) = \frac{1}{n} \sum_{i=1}^n \ell(f_{\mathbf{x}}(\mathbf{z}^{(i)}), y^{(i)}), \quad R(\mathbf{x}) = \mathbb{E}_{(\mathbf{z}, y)}[\ell(f_{\mathbf{x}}(\mathbf{z}), y)].$$

优化算法直接减小 R_{emp} ；泛化要求 R 小。过拟合时 $R_{\text{emp}} \ll R$ ，故仅压训练误差不够。

11.1.2 深度学习中的优化挑战

多数深度目标无解析解，只能用数值迭代。梯度为零的点可以是局部极小、鞍点，或因梯度消失而停滞。

11.1.2.1 局部最小值

在 $[-1, 2]$ 上

$$f(x) = x \cos(\pi x)$$

有多个局部极小。梯度下降停在哪一个，取决于初值与步长。

11.1.2.2 鞍点

$f(x) = x^3$ 在 $x = 0$ 处 $f'(0) = f''(0) = 0$ ，但不是极小。 $f(x, y) = x^2 - y^2$ 在 $(0, 0)$ 是鞍点：沿 x 极小、沿 y 极大。Hessian 的特征值全正为局部极小，全负为局部极大，有正有负则为鞍点。

11.1.2.3 梯度消失

若 $f(x) = \tanh(x)$ ，则

$$f'(x) = 1 - \tanh^2(x), \quad f'(4) \approx 0.0013.$$

从 $x = 4$ 出发，步长 $\eta f'(x)$ 接近零，迭代长时间几乎不动。

11.1.3 小结

优化压 R_{emp} ，不自动压 R 。局部极小、鞍点与 $f' \approx 0$ 都会使 $x \leftarrow x - \eta f'(x)$ 停滞。

11.1.4 练习

核验：隐藏层宽度为 d 时局部极小的置换对称、对称随机矩阵特征值分布是否关于零对称。

11.2 凸性

若算法在凸问题上失败，非凸上更难指望。深度目标整体非凸，但局部常近似凸。

11.2.1 定义

先固定凸集，再固定凸函数。

11.2.1.1 凸集

集合 $\mathcal{X}$ 凸，当且仅当对任意 $a, b \in \mathcal{X}$ 与 $\lambda \in [0, 1]$,

$$\lambda a + (1 - \lambda)b \in \mathcal{X}.$$

11.2.1.2 凸函数

f 在凸集上凸，当且仅当

$$\lambda f(x) + (1 - \lambda)f(x') \geq f(\lambda x + (1 - \lambda)x').$$

11.2.1.3 詹森不等式

对 $\alpha_i \geq 0$ 、 $\sum_i \alpha_i = 1$,

$$\begin{aligned} \sum_i \alpha_i f(x_i) &\geq f\left(\sum_i \alpha_i x_i\right), \\ \mathbb{E}_X[f(X)] &\geq f(\mathbb{E}_X[X]). \end{aligned}$$

取 $f = -\log$ 得

$$\mathbb{E}_{Y \sim P(Y)}[-\log P(X | Y)] \geq -\log P(X).$$

11.2.2 性质

凸性把局部信息变成全局信息。

11.2.2.1 局部极小值是全局极小值

设 x^* 局部极小而 $f(x') < f(x^*)$ 。对充分小的 $\lambda \in (0, 1)$,

$$\begin{aligned} f(\lambda x^* + (1 - \lambda)x') &\leq \lambda f(x^*) + (1 - \lambda)f(x') \\ &< \lambda f(x^*) + (1 - \lambda)f(x^*) = f(x^*), \end{aligned}$$

与局部极小矛盾。故凸函数的局部极小即全局极小。

11.2.2.2 凸函数的下水平集是凸的

下水平集

$$\mathcal{S}_b := \{x \in \mathcal{X} : f(x) \leq b\}$$

对凸 f 是凸的：若 $x, x' \in \mathcal{S}_b$ ，则

$$f(\lambda x + (1 - \lambda)x') \leq \lambda f(x) + (1 - \lambda)f(x') \leq b.$$

11.2.2.3 凸性和二阶导数

一维时，凸性蕴含

$$\frac{1}{2}f(x+\epsilon) + \frac{1}{2}f(x-\epsilon) \geq f(x),$$

从而

$$f''(x) = \lim_{\epsilon \rightarrow 0} \frac{f(x+\epsilon) + f(x-\epsilon) - 2f(x)}{\epsilon^2} \geq 0.$$

若 f' 单调，由中值定理

$$f'(\alpha) = \frac{f(x) - f(a)}{x - a}, \quad f'(\beta) = \frac{f(b) - f(x)}{b - x},$$

再令 $\lambda = (x - a)/(b - a)$ 得

$$\lambda f(b) + (1 - \lambda)f(a) \geq f((1 - \lambda)a + \lambda b).$$

高维时定义

$$g(z) \stackrel{\text{def}}{=} f(z\mathbf{x} + (1 - z)\mathbf{y}), \quad z \in [0, 1].$$

f 凸当且仅当每个这样的 g 凸，当且仅当 Hessian $\nabla^2 f$ 半正定。

11.2.3 约束

约束问题写成

$$\begin{aligned} & \underset{\mathbf{x}}{\text{minimize}} && f(\mathbf{x}) \\ & \text{subject to} && c_i(\mathbf{x}) \leq 0, \quad i = 1, \dots, N. \end{aligned}$$

11.2.3.1 拉格朗日函数

乘子 $\alpha_i \geq 0$,

$$L(\mathbf{x}, \alpha_1, \dots, \alpha_n) = f(\mathbf{x}) + \sum_{i=1}^n \alpha_i c_i(\mathbf{x}).$$

11.2.3.2 惩罚

把 $\alpha_i c_i(\mathbf{x})$ 加进目标，近似强制 $c_i(\mathbf{x}) \leq 0$ 。权重衰减 $\frac{\lambda}{2} \|\mathbf{w}\|^2$ 对应约束 $\|\mathbf{w}\|^2 - r^2 \leq 0$ 。

11.2.3.3 投影

梯度裁剪

$$\mathbf{g} \leftarrow \mathbf{g} \cdot \min(1, \theta / \|\mathbf{g}\|)$$

把更新限制在半径 θ 的球上。一般投影为

$$\text{Proj}_{\mathcal{X}}(\mathbf{x}) = \underset{\mathbf{x}' \in \mathcal{X}}{\text{argmin}} \|\mathbf{x} - \mathbf{x}'\|.$$

11.2.4 小结

凸函数满足 $\lambda f(x) + (1 - \lambda)f(x') \geq f(\lambda x + (1 - \lambda)x')$ ，局部极小即全局极小，Hessian 半正定。约束经 L 或惩罚进入目标，可行点由 $\text{Proj}_{\mathcal{X}}$ 拉回。

11.2.5 练习

核验： p 范数球的凸性、 $\max(f, g)$ 保持凸而 $\min(f, g)$ 不必、Softmax 规范化的凸性。

11.3 梯度下降

全梯度下降很少直接用于深度模型，但它给出步长、曲率与预处理的原型。

11.3.1 一维梯度下降

泰勒展开

$$f(x + \epsilon) = f(x) + \epsilon f'(x) + \mathcal{O}(\epsilon^2).$$

取 $\epsilon = -\eta f'(x)$,

$$f(x - \eta f'(x)) = f(x) - \eta f'(x)^2 + \mathcal{O}(\eta^2 f'(x)^2).$$

η 充分小时 f 下降，更新为

$$x \leftarrow x - \eta f'(x).$$

11.3.1.1 学习率

η 过小则每步 $|\eta f'(x)|$ 太小； $|\eta f'(x)|$ 过大则二阶余项不可忽略， f 不必下降。

11.3.1.2 局部最小值

$f(x) = x \cos(cx)$ 有无穷多个局部极小。过大的 η 可把迭代送进较差的谷。

11.3.2 多元梯度下降

$$\nabla f(\mathbf{x}) = \left[\frac{\partial f(\mathbf{x})}{\partial x_1}, \dots, \frac{\partial f(\mathbf{x})}{\partial x_d} \right]^\top,$$

$$f(\mathbf{x} + \boldsymbol{\epsilon}) = f(\mathbf{x}) + \boldsymbol{\epsilon}^\top \nabla f(\mathbf{x}) + \mathcal{O}(\|\boldsymbol{\epsilon}\|^2),$$

$$\mathbf{x} \leftarrow \mathbf{x} - \eta \nabla f(\mathbf{x}).$$

11.3.3 自适应方法

合适的 η 难选。二阶方法用曲率自动定步，计算贵，但给出预处理的直觉。

11.3.3.1 牛顿法

$$f(\mathbf{x} + \boldsymbol{\epsilon}) = f(\mathbf{x}) + \boldsymbol{\epsilon}^\top \nabla f(\mathbf{x}) + \frac{1}{2} \boldsymbol{\epsilon}^\top \nabla^2 f(\mathbf{x}) \boldsymbol{\epsilon} + \mathcal{O}(\|\boldsymbol{\epsilon}\|^3).$$

令 $\mathbf{H} = \nabla^2 f(\mathbf{x})$ ，驻点条件

$$\nabla f(\mathbf{x}) + \mathbf{H}\boldsymbol{\epsilon} = 0 \quad \Rightarrow \quad \boldsymbol{\epsilon} = -\mathbf{H}^{-1} \nabla f(\mathbf{x}).$$

11.3.3.2 收敛性分析

一维牛顿误差 $e^{(k)} = x^{(k)} - x^*$ 满足

$$\begin{aligned} 0 &= f'(x^{(k)} - e^{(k)}) \\ &= f'(x^{(k)}) - e^{(k)} f''(x^{(k)}) + \frac{1}{2} (e^{(k)})^2 f'''(\xi^{(k)}), \end{aligned}$$

从而

$$|e^{(k+1)}| = \frac{1}{2} (e^{(k)})^2 \frac{|f'''(\xi^{(k)})|}{f''(x^{(k)})} \leq c (e^{(k)})^2.$$

在凸且 f'' 有下界时为二次收敛。

11.3.3.3 预处理

对角近似

$$\mathbf{x} \leftarrow \mathbf{x} - \eta \operatorname{diag}(\mathbf{H})^{-1} \nabla f(\mathbf{x})$$

按坐标缩放步长，避免直接求 $\mathbf{H}^{-1}$ 。

11.3.3.4 梯度下降和线搜索

沿 $-\nabla f(\mathbf{x})$ 对 η 做一维搜索，使 $f(\mathbf{x} - \eta \nabla f(\mathbf{x}))$ 最小。每步需在全数据上求值，深度模型上通常不可行。

11.3.4 小结

更新 $\mathbf{x} \leftarrow \mathbf{x} - \eta \nabla f(\mathbf{x})$ 中 η 过大发散、过小停滞。牛顿步 $-\mathbf{H}^{-1} \nabla f$ 在凸问题上快，非凸上 Hessian 不定时不能直接用。

11.3.5 练习

核验：不同 η 下 f 是否单调下降、线搜索是否需要导数、对角 Hessian 预处理对坐标尺度不均问题的作用。

11.4 随机梯度下降

全梯度每次用全部 n 个样本；随机梯度每次用一个样本。

11.4.1 随机梯度更新

$$f(\mathbf{x}) = \frac{1}{n} \sum_{i=1}^n f_i(\mathbf{x}), \quad \nabla f(\mathbf{x}) = \frac{1}{n} \sum_{i=1}^n \nabla f_i(\mathbf{x}).$$

随机更新

$$\mathbf{x} \leftarrow \mathbf{x} - \eta \nabla f_i(\mathbf{x})$$

无偏：

$$\mathbb{E}_i[\nabla f_i(\mathbf{x})] = \nabla f(\mathbf{x}).$$

11.4.2 动态学习率

$\eta(t) = \eta_i$	$t_i \leq t \leq t_{i+1}$	分段常数,
$\eta(t) = \eta_0 e^{-\lambda t}$		指数衰减,
$\eta(t) = \eta_0 (\beta t + 1)^{-\alpha}$		多项式衰减.

11.4.3 凸目标的收敛性分析

$$\mathbf{x}_{t+1} = \mathbf{x}_t - \eta_t \partial_{\mathbf{x}} f(\boldsymbol{\xi}_t, \mathbf{x}), \quad R(\mathbf{x}) = \mathbb{E}_{\boldsymbol{\xi}}[f(\boldsymbol{\xi}, \mathbf{x})].$$

展开距离

$$\begin{aligned} \|\mathbf{x}_{t+1} - \mathbf{x}^*\|^2 &= \|\mathbf{x}_t - \mathbf{x}^*\|^2 + \eta_t^2 \|\partial_{\mathbf{x}} f(\boldsymbol{\xi}_t, \mathbf{x})\|^2 \\ &\quad - 2\eta_t \langle \mathbf{x}_t - \mathbf{x}^*, \partial_{\mathbf{x}} f(\boldsymbol{\xi}_t, \mathbf{x}) \rangle. \end{aligned}$$

若梯度有界 $\|\partial_{\mathbf{x}} f\| \leq L$ ，则 $\eta_t^2 \|\partial_{\mathbf{x}} f\|^2 \leq \eta_t^2 L^2$ 。凸性给出

$$f(\boldsymbol{\xi}_t, \mathbf{x}^*) \geq f(\boldsymbol{\xi}_t, \mathbf{x}_t) + \langle \mathbf{x}^* - \mathbf{x}_t, \partial_{\mathbf{x}} f(\boldsymbol{\xi}_t, \mathbf{x}_t) \rangle,$$

于是

$$\|\mathbf{x}_t - \mathbf{x}^*\|^2 - \|\mathbf{x}_{t+1} - \mathbf{x}^*\|^2 \geq 2\eta_t(f(\boldsymbol{\xi}_t, \mathbf{x}_t) - f(\boldsymbol{\xi}_t, \mathbf{x}^*)) - \eta_t^2 L^2.$$

取期望并累加，加权平均

$$\bar{\mathbf{x}} \stackrel{\text{def}}{=} \frac{\sum_{t=1}^T \eta_t \mathbf{x}_t}{\sum_{t=1}^T \eta_t}$$

满足

$$\mathbb{E}[R(\bar{\mathbf{x}})] - R^* \leq \frac{r^2 + L^2 \sum_{t=1}^T \eta_t^2}{2 \sum_{t=1}^T \eta_t}.$$

11.4.4 随机梯度和有限样本

有放回时，一轮中某样本被抽到的概率

$$1 - (1 - 1/n)^n \approx 1 - e^{-1} \approx 0.63.$$

恰好抽一次的概率约为 $e^{-1} \approx 0.37$ 。无放回遍历则每个样本每轮恰好一次。

11.4.5 小结

凸时 SGD 对广泛的 $\{\eta_t\}$ 收敛到最优；深度目标非凸，无同样保证。 n 大时全梯度每步代价高，故用 ∇f_i 。

11.4.6 练习

核验： η_t 衰减过快或过慢时 $\|\mathbf{x}_t - \mathbf{x}^*\|$ 的轨迹，以及有放回与无放回采样对无偏性的影响。

11.5 小批量随机梯度下降

全梯度数据效率低，单样本计算效率低。小批量在二者之间。

11.5.1 向量化和缓存

一次处理 $|\mathcal{B}|$ 个样本，使矩阵乘与缓存命中摊薄解释器与内存延迟。多 GPU 时每卡至少一张图，有效批量随设备数放大。

11.5.2 小批量

单样本

$$\mathbf{g}_t = \partial_{\mathbf{w}} f(\mathbf{x}_t, \mathbf{w}),$$

小批量

$$\mathbf{g}_t = \partial_{\mathbf{w}} \frac{1}{|\mathcal{B}_t|} \sum_{i \in \mathcal{B}_t} f(\mathbf{x}_i, \mathbf{w}).$$

$\mathcal{B}_t$ 来自数据的随机排列，每轮每样本一次。

11.5.3 读取数据集

对已标准化的设计矩阵 $\mathbf{X} \in \mathbb{R}^{n \times d}$ 、响应 $\mathbf{y}$ ，迭代器输出 $(\mathbf{X}_{\mathcal{B}}, \mathbf{y}_{\mathcal{B}})$ ， $|\mathcal{B}|$ 为批量。

11.5.4 从零开始实现

线性模型 $\hat{y} = \mathbf{w}^\top \mathbf{x} + b$ ，损失在批量上先平均再反传，故更新中 $\mathbf{g}_t$ 不再除 $|\mathcal{B}|$ 。 $|\mathcal{B}| = n$ 退化为批量梯度下降。

11.5.5 简洁实现

框架优化器实现同一映射 $\mathbf{w} \leftarrow \mathbf{w} - \eta \mathbf{g}_t$ ，状态为空。

11.5.6 小结

小批量梯度 $\mathbf{g}_t = \partial_{\mathbf{w}} \frac{1}{|\mathcal{B}_t|} \sum_{i \in \mathcal{B}_t} f(\mathbf{x}_i, \mathbf{w})$ 同时利用向量化与噪声梯度。训练中降低 η 有助于后期收敛。

11.5.7 练习

核验： $|\mathcal{B}|$ 与 η 如何改变目标下降速度，以及数据被复制一倍时三种梯度估计的偏差。

11.6 动量法

噪声梯度上 η 难选。动量用过去梯度的泄漏平均代替当前梯度。

11.6.1 基础

先写平均，再写更新。

11.6.1.1 泄漏平均值

$$\mathbf{g}_{t,t-1} = \partial_w \frac{1}{|\mathcal{B}_t|} \sum_{i \in \mathcal{B}_t} f(\mathbf{x}_i, \mathbf{w}_{t-1}).$$

$$\mathbf{v}_t = \beta \mathbf{v}_{t-1} + \mathbf{g}_{t,t-1} = \sum_{\tau=0}^{t-1} \beta^\tau \mathbf{g}_{t-\tau,t-\tau-1}.$$

11.6.1.2 条件不佳的问题

$$f(\mathbf{x}) = 0.1 x_1^2 + 2 x_2^2$$

两个方向曲率相差 20 倍，固定 η 的梯度下降在窄谷中振荡。

11.6.1.3 动量法

$$\mathbf{v}_t \leftarrow \beta \mathbf{v}_{t-1} + \mathbf{g}_{t,t-1},$$

$$\mathbf{x}_t \leftarrow \mathbf{x}_{t-1} - \eta_t \mathbf{v}_t.$$

11.6.1.4 有效样本权重

$\sum_{\tau=0}^{\infty} \beta^\tau = 1/(1-\beta)$ ，有效步长约为 $\eta/(1-\beta)$ 。 $\beta = 0.9$ 相当于约 10 个梯度的加权平均。

11.6.2 实际实验

速度 $\mathbf{v}$ 与 $\mathbf{g}$ 同形，需额外状态。

11.6.2.1 从零开始实现

维护 $\mathbf{v}$ ，按上式更新。增大 β 时应略降 η ，以免有效步长过大。

11.6.2.2 简洁实现

框架求解器在同一 (β, η) 下实现同一 $\mathbf{v}, \mathbf{x}$ 递推。

11.6.3 理论分析

二次凸目标把动量写成线性递推。

11.6.3.1 二次凸函数

$$h(\mathbf{x}) = \frac{1}{2} \mathbf{x}^\top \mathbf{Q} \mathbf{x} + \mathbf{x}^\top \mathbf{c} + b.$$

配平方后经 $\mathbf{Q} = \mathbf{U} \mathbf{\Lambda} \mathbf{U}^\top$ 变成

$$h(\mathbf{z}) = \frac{1}{2} \mathbf{z}^\top \mathbf{\Lambda} \mathbf{z} + b'.$$

无动量时

$$\mathbf{z}_t = (\mathbf{I} - \mathbf{\Lambda}) \mathbf{z}_{t-1}.$$

有动量时

$$\begin{aligned} \mathbf{v}_t &= \beta \mathbf{v}_{t-1} + \mathbf{\Lambda} \mathbf{z}_{t-1}, \\ \mathbf{z}_t &= (\mathbf{I} - \eta \mathbf{\Lambda}) \mathbf{z}_{t-1} - \eta \beta \mathbf{v}_{t-1}. \end{aligned}$$

11.6.3.2 标量函数

无动量 $x_{t+1} = (1 - \eta\lambda)x_t$ 。有动量

$$\begin{bmatrix} v_{t+1} \\ x_{t+1} \end{bmatrix} = \begin{bmatrix} \beta & \lambda \\ -\eta\beta & 1 - \eta\lambda \end{bmatrix} \begin{bmatrix} v_t \\ x_t \end{bmatrix} = \mathbf{R}(\beta, \eta, \lambda) \begin{bmatrix} v_t \\ x_t \end{bmatrix}.$$

谱半径 $\rho(\mathbf{R}) < 1$ 时该坐标收敛。

11.6.4 小结

动量用 $\mathbf{v}_t = \sum_{\tau} \beta^\tau \mathbf{g}_{t-\tau}$ 替代当前梯度，有效梯度个数为 $1/(1 - \beta)$ ，并多存一个状态 $\mathbf{v}$ 。

11.6.5 练习

核验： $h(\mathbf{x}) = \frac{1}{2} \mathbf{x}^\top \mathbf{Q} \mathbf{x} + \mathbf{x}^\top \mathbf{c} + b$ 的最小点，以及 β, η 对 $\rho(\mathbf{R})$ 的影响。

11.7 AdaGrad 算法

稀疏特征上，全局递减的 η 对常见坐标过慢、对罕见坐标过快。

11.7.1 稀疏特征和学习率

语言模型中罕见词的参数只在该词出现时更新。若 $\eta_t = \mathcal{O}(t^{-1/2})$ 已很小，罕见坐标尚未被充分观测。

11.7.2 预处理

二次型经坐标缩放后

$$f(\mathbf{x}) = \bar{f}(\bar{\mathbf{x}}) = \frac{1}{2} \bar{\mathbf{x}}^\top \mathbf{\Lambda} \bar{\mathbf{x}} + \bar{\mathbf{c}}^\top \bar{\mathbf{x}} + b,$$

条件数 $\kappa = \Lambda_1/\Lambda_d$ 。对角预处理

$$\tilde{\mathbf{Q}} = \text{diag}^{-1/2}(\mathbf{Q}) \mathbf{Q} \text{diag}^{-1/2}(\mathbf{Q})$$

把各坐标曲率拉近。梯度

$$\partial_{\bar{\mathbf{x}}} \bar{f}(\bar{\mathbf{x}}) = \mathbf{\Lambda}(\bar{\mathbf{x}} - \bar{\mathbf{x}}_0)$$

幅度随坐标变化，可用 $|\mathbf{g}|$ 代理曲率。

11.7.3 算法

$$\begin{aligned} \mathbf{g}_t &= \partial_{\mathbf{w}} \ell(y_t, f(\mathbf{x}_t, \mathbf{w})), \\ \mathbf{s}_t &= \mathbf{s}_{t-1} + \mathbf{g}_t^2, \\ \mathbf{w}_t &= \mathbf{w}_{t-1} - \frac{\eta}{\sqrt{\mathbf{s}_t} + \epsilon} \cdot \mathbf{g}_t. \end{aligned}$$

平方与除法按坐标逐元素。测试函数仍用 $f(\mathbf{x}) = 0.1 x_1^2 + 2 x_2^2$ 。

11.7.4 从零开始实现

状态 $\mathbf{s}$ 与 $\mathbf{w}$ 同形，按上式更新。因 $\sqrt{\mathbf{s}_t}$ 单调增，后期步长自动变小，故可用比 SGD 更大的初始 η 。

11.7.5 简洁实现

框架 AdaGrad 实现同一 $(\mathbf{s}, \mathbf{w})$ 递推。

11.7.6 小结

AdaGrad 用 $\mathbf{s}_t = \sum_{\tau=1}^t \mathbf{g}_\tau^2$ 按坐标缩小步长：大梯度坐标走得慢，稀疏坐标保持相对大的有效学习率。深度非凸上 $\mathbf{s}_t$ 可能累积过快。

11.7.7 练习

核验：正交变换下 $\|\mathbf{U}\mathbf{c} - \mathbf{U}\boldsymbol{\delta}\|_2 = \|\mathbf{c} - \boldsymbol{\delta}\|_2$ ，以及旋转 $f(\mathbf{x}) = 0.1(x_1 + x_2)^2 + 2(x_1 - x_2)^2$ 后轨迹是否改变。

11.8 RMSProp 算法

AdaGrad 的 $\mathbf{s}_t$ 无界累积，等价于 η 按 $\mathcal{O}(t^{-1/2})$ 下降，非凸上往往过早停滞。

11.8.1 算法

$$\begin{aligned}\mathbf{s}_t &\leftarrow \gamma \mathbf{s}_{t-1} + (1 - \gamma) \mathbf{g}_t^2, \\ \mathbf{x}_t &\leftarrow \mathbf{x}_{t-1} - \frac{\eta}{\sqrt{\mathbf{s}_t + \epsilon}} \odot \mathbf{g}_t.\end{aligned}$$

展开

$$\mathbf{s}_t = (1 - \gamma)(\mathbf{g}_t^2 + \gamma \mathbf{g}_{t-1}^2 + \gamma^2 \mathbf{g}_{t-2}^2 + \cdots).$$

η 由实验者单独调度； γ 控制历史窗口，有效长度约 $1/(1 - \gamma)$ 。

11.8.2 从零开始实现

同一二次函数上，AdaGrad 在 $\eta = 0.4$ 后期几乎不动；RMSProp 因泄漏平均不让 $\mathbf{s}_t$ 无限增长。深度网络常用 $\eta = 0.01$ 、 $\gamma = 0.9$ 。

11.8.3 简洁实现

框架 RMSProp 实现同一更新。

11.8.4 小结

RMSProp 用泄漏平均 $\mathbf{s}_t \leftarrow \gamma \mathbf{s}_{t-1} + (1 - \gamma) \mathbf{g}_t^2$ 做按坐标预处理，并把全局 η 从累积平方和中分离。

11.8.5 练习

核验： $\gamma = 1$ 时 $\mathbf{s}_t$ 不再更新的后果，以及旋转二次面上的收敛。

11.9 Adadelta

Adadelta 进一步取消显式 η ，用参数变化量校准更新幅度。

11.9.1 Adadelta 算法

梯度二阶矩

$$\mathbf{s}_t = \rho \mathbf{s}_{t-1} + (1 - \rho) \mathbf{g}_t^2.$$

重缩放梯度与参数更新

$$\begin{aligned} \mathbf{g}'_t &= \frac{\sqrt{\Delta \mathbf{x}_{t-1} + \epsilon}}{\sqrt{\mathbf{s}_t + \epsilon}} \odot \mathbf{g}_t, \\ \mathbf{x}_t &= \mathbf{x}_{t-1} - \mathbf{g}'_t, \\ \Delta \mathbf{x}_t &= \rho \Delta \mathbf{x}_{t-1} + (1 - \rho) \mathbf{g}'_t{}^2. \end{aligned}$$

需两个状态 $\mathbf{s}_t$ 、 $\Delta \mathbf{x}_t$ 。 $\rho = 0.9$ 对应约 10 个半衰期。

11.9.2 代码实现

每坐标维护 $(\mathbf{s}, \Delta \mathbf{x})$ ，按上式递推；框架 API 实现同一映射。

11.9.3 小结

Adadelta 用 $\sqrt{\Delta \mathbf{x}_{t-1}}/\sqrt{\mathbf{s}_t}$ 代替 η ，仍用泄漏平均估计二阶统计。

11.9.4 练习

核验：能否不显式形成 $\mathbf{g}'_t$ 而得到同一 $\mathbf{x}_t$ ，以及与 AdaGrad、RMSProp 轨迹的差别。

11.10 Adam 算法

Adam 把动量的一阶矩与 RMSProp 的二阶矩合在同一更新里，并做偏差校正。

11.10.1 算法

$$\begin{aligned} \mathbf{v}_t &\leftarrow \beta_1 \mathbf{v}_{t-1} + (1 - \beta_1) \mathbf{g}_t, \\ \mathbf{s}_t &\leftarrow \beta_2 \mathbf{s}_{t-1} + (1 - \beta_2) \mathbf{g}_t^2. \end{aligned}$$

因 $\mathbf{v}_0 = \mathbf{s}_0 = \mathbf{0}$,

$$\begin{aligned} \hat{\mathbf{v}}_t &= \frac{\mathbf{v}_t}{1 - \beta_1^t}, & \hat{\mathbf{s}}_t &= \frac{\mathbf{s}_t}{1 - \beta_2^t}. \\ \mathbf{g}'_t &= \frac{\eta \hat{\mathbf{v}}_t}{\sqrt{\hat{\mathbf{s}}_t} + \epsilon}, & \mathbf{x}_t &\leftarrow \mathbf{x}_{t-1} - \mathbf{g}'_t. \end{aligned}$$

11.10.2 实现

状态为 $(\mathbf{v}, \mathbf{s}, t)$ 。常用 $\eta = 0.01$, $\beta_1 = 0.9$, $\beta_2 = 0.999$ 。

11.10.3 Yogi

Adam 中 $\mathbf{s}_t$ 在梯度突变时可能被大正项拉动。把

$$\mathbf{s}_t \leftarrow \mathbf{s}_{t-1} + (1 - \beta_2)(\mathbf{g}_t^2 - \mathbf{s}_{t-1})$$

改成

$$\mathbf{s}_t \leftarrow \mathbf{s}_{t-1} + (1 - \beta_2) \mathbf{g}_t^2 \odot \text{sgn}(\mathbf{g}_t^2 - \mathbf{s}_{t-1})$$

使增量幅度受当前 $\mathbf{s}$ 限制。

11.10.4 小结

Adam 的更新是偏差校正后的 $\mathbf{x}_t \leftarrow \mathbf{x}_{t-1} - \eta \hat{\mathbf{v}}_t / (\sqrt{\hat{\mathbf{s}}_t} + \epsilon)$ 。梯度尺度差异大时可用更大批量或 Yogi 的 $\mathbf{s}_t$ 。

11.10.5 练习

核验：去掉偏差校正后早期步长、以及何种 $\mathbf{g}_t$ 序列使 Adam 发散而 Yogi 收敛。

11.11 学习率调度器

更新规则确定方向， η_t 确定步长。过大发散，过小停滞或次优；条件数大时更敏感。

11.11.1 一个问题

在 Fashion-MNIST 上用固定 $\eta = 0.3$ 训练修改后的 LeNet。训练精度可继续升而测试精度停滞，间隙对应过拟合。

11.11.2 学习率调度器

调度器是映射 $t \mapsto \eta_t$ 。例如

$$\eta_t = \eta_0(t+1)^{-1/2}.$$

比固定 η 更平滑，过拟合间隙往往更小。

11.11.3 策略

常用多项式、分段常数、余弦与预热。

11.11.3.1 单因子调度器

乘法衰减

$$\eta_{t+1} \leftarrow \eta_t \cdot \alpha, \quad \alpha \in (0, 1),$$

并加下限

$$\eta_{t+1} \leftarrow \max(\eta_{\min}, \eta_t \cdot \alpha).$$

11.11.3.2 多因子调度器

在预定时刻 $s = \{5, 10, 20\}$ 上 $\eta_{t+1} \leftarrow \eta_t \cdot \alpha$ 。直觉是：先用较大 η 走到驻点附近，再减小 η 降低参数噪声。

11.11.3.3 余弦调度器

$$\eta_t = \eta_T + \frac{\eta_0 - \eta_T}{2} (1 + \cos(\pi t/T)).$$

11.11.3.4 预热

先把 η 从接近 0 线性升到 η_0 ，再接入余弦或其他衰减，避免深度网络初期过大步长导致发散。

11.11.4 小结

η_t 应随训练下降。余弦调度为 $\eta_t = \eta_T + \frac{\eta_0 - \eta_T}{2}(1 + \cos(\pi t/T))$ ；预热限制初期 $\|\eta \mathbf{g}\|$ 。

11.11.5 练习

核验：固定 η 能达到的最优训练误差、多项式指数如何改变收敛，以及预热长度对初期损失的影响。

第十二章 计算性能

前向与反向都是计算图上的算子序列。性能取决于图如何编译、设备如何重叠计算与通信、以及参数如何在多卡多机上同步。命令式便于写控制流，符号式便于整图优化。下面把二者写成同一套映射。

12.1 编译器和解释器

命令式编程按语句改状态。Python 解释执行，每步都经过前端。

12.1.1 符号式编程

先定义计算流程，再编译成可执行程序，最后喂入输入。记命令式求值为

$$(a, b, c, d) \mapsto e = a + b, f = c + d, g = e + f,$$

符号式先构造图 G ，再

$$G \mapsto P = \text{compile}(G), \quad P(a, b, c, d) = g.$$

编译器可见全图，可消去中间量，甚至把常量折叠成 $P \equiv 10$ 。

12.1.2 混合式编程

PyTorch 默认动态图。TorchScript 把已追踪的命令式模型编译成静态图，开发仍用 Python，部署用 $P = \text{script}(f)$ 。

12.1.3 Sequential 的混合式编程

L 层序列 $f = f_L \circ \dots \circ f_1$ 。解释器对每层单独发指令；多 GPU 时单线程前端成为瓶颈。把 Sequential 脚本化后，整条复合映射一次下发。

12.1.3.1 通过混合式编程加速

同一输入 $\mathbf{x}$ ，比较 $f(\mathbf{x})$ 与 $\text{script}(f)(\mathbf{x})$ 的墙钟时间。编译后减少 Python 往返，输出值不变。

12.1.3.2 序列化

$$(f, \theta) \mapsto \text{save}(P, \theta) \mapsto \text{其他设备或语言上的 } P.$$

序列化的是编译后的图与参数，不是 Python 源。

12.1.4 小结

命令式写 f ；符号式把计算图 G 编译成 P 。混合式在 $f \mapsto P$ 后加速，且 $P(\mathbf{x}) = f(\mathbf{x})$ 。

12.1.5 练习

核验：对已有模型做 $f \mapsto \text{script}(f)$ 后，前向时延是否下降、数值是否一致。

12.2 异步计算

GPU 核函数默认异步：前端插入任务后立即返回，后端在设备上执行。

12.2.1 通过后端异步处理

前端发出 $\mathbf{C} = \mathbf{A}\mathbf{B}$ 后，Python 已继续；实际 GEMM 在队列中。同步 S 强制等待队列清空。测时若不调用 S ，测到的是入队时间而非计算时间。

12.2.2 障碍物与阻塞器

障碍 S 使前端与指定设备上所有流对齐：

`sync(device)`：等待该设备队列空。

把结果拷回 CPU 或打印标量也会隐式阻塞。

12.2.3 改进计算

设前端入队、后端计算、回传分别为 t_1, t_2, t_3 。同步执行 N 次总时间为 $N(t_1 + t_2 + t_3)$ ；异步且 Nt_2 主导时约为 $t_1 + Nt_2 + t_3$ 。队列过长会堆积中间张量，故常按小批量同步一次。

12.2.4 小结

前端与后端解耦：命令快速入队，计算并行执行。过度填充队列增加显存；每个小批量一次 sync 使两侧大致对齐。

12.2.5 练习

核验：同一矩阵乘在 CPU 上是否仍能观察到入队与完成之间的时间差。

12.3 自动并行

框架根据计算图上的依赖决定哪些算子可并行。无依赖的子图可同时占用不同设备。

12.3.1 基于 GPU 的并行计算

在 GPU1、GPU2 上各做 10 次矩阵乘。两次 sync 串行测时约为两段之和；去掉中间同步后，墙钟时间接近 $\max(T_1, T_2)$ ，即自动重叠。

12.3.2 并行计算与通信

计算 $y = f(x)$ 于 GPU，再 $y \mapsto \text{CPU}$ 。copy(non_blocking) 允许尚未完成的后续核与总线传输重叠。反传中先就绪的梯度可提前走 PCI-Express。

12.3.3 小结

计算图给出依赖；无依赖的计算与通信可并行。设备间搬移与算子执行重叠，才能吃满总线与计算单元。

12.3.4 练习

核验：无依赖的多个小算子是否被自动并行，以及 CPU-GPU 同时计算加拷贝的总时延。

12.4 硬件

算法决定统计效率；硬件决定一次 ∇f 要多久。延迟差几个数量级就会使可训练与不可训练对换。

12.4.1 计算机

一台训练机构成映射

(CPU, RAM, GPU, PCIe, NIC, 存储) $\mapsto$ 每秒可完成的迭代.

CPU 跑调度与数据管道；参数与激活放 RAM/显存；PCIe 接加速卡。

12.4.2 内存

带宽与延迟同时约束。DDR 通道峰值约 40-100 GB/s，但先发地址再突发读；小块随机访问远低于峰值。深度学习应尽量连续、大批量访问。

12.4.3 存储器

持久存储同样由带宽与 IOPS 刻画，器件间差几个数量级。

12.4.3.1 硬盘驱动器

约 7200 RPM，最坏旋转等待约 8 ms，约 10^2 IOPS。顺序带宽远高于随机。

12.4.3.2 固态驱动器

10^5 - 5×10^5 量级 IOPS，带宽 1-3 GB/s。按块擦写，随机小写远慢于读。

12.4.3.3 云存储

IOPS 与容量可配置。训练若大量小记录，延迟高时应提高 IOPS 配额。

12.4.4 CPU

核心执行指令，缓存降低访存延迟，向量单元做 SIMD。

12.4.4.1 微体系结构

前端取指、预测、解码；执行端口可同时发射多条独立微操作。整数端口与浮点端口不同，吞吐取决于指令能否并行。

12.4.4.2 矢量化

SIMD 在一个周期内对向量寄存器做同一运算。FMA 计算 $a \leftarrow a + b \cdot c$ 。寄存器可达 512 位。

12.4.4.3 缓存

4 核、2 GHz、每周期 256 位 AVX 约需

$$4 \times 256 \text{ bit} \times 2 \times 10^9 / \text{s} = 256 \text{ GB/s},$$

而内存接口只有数十 GB/s。差额必须由 L1/L2/L3 命中补上。

12.4.5 GPU 和其他加速卡

训练要存中间激活，常用 FP16/FP32；推断可 INT8 且不必存反传状态。GPU 以吞吐换取相对 CPU 高一个数量级的密集线性代数。

12.4.6 网络和总线

PCIe 单链路高带宽、微秒级延迟，通道数有限。NVLink 卡间带宽高于 PCIe；以太网跨机约 10 GB/s 量级，比 NVLink 慢约一个数量级。

12.4.7 更多延迟

缓存命中约纳秒，内存约几十纳秒，PCIe/网络微秒到毫秒，磁盘毫秒。同一迭代里应避免把计算拆成大量高延迟小传输。

12.4.8 小结

量少次多的传输浪费延迟。向量化与对齐决定能否达到峰值。训练精度过低会下溢；算法应让热参数留在缓存带宽内。

12.4.9 练习

核验：对齐与非对齐访问、固定步幅访问的带宽，以及 FP16 累积是否溢出。

12.5 多 GPU 训练

k 张 GPU 增加显存与算力。需把一次小批量上的 ∇f 拆开再聚合。

12.5.1 问题拆分

层并行按层切网络，模型并行按层内切参数，数据并行按样本切批量。前两者要编排激活与梯度的传递；数据并行只同步梯度，实现最直接。

12.5.2 数据并行性

k 张卡各持一份完整 θ 。一次迭代：

$$\begin{aligned}\mathcal{B} &= \mathcal{B}_1 \cup \dots \cup \mathcal{B}_k, \\ \mathbf{g}^{(j)} &= \nabla_{\theta} \frac{1}{|\mathcal{B}_j|} \sum_{i \in \mathcal{B}_j} \ell_i, \\ \mathbf{g} &= \text{allreduce}(\mathbf{g}^{(1)}, \dots, \mathbf{g}^{(k)}), \\ \theta &\leftarrow \theta - \eta \mathbf{g} \quad \text{在每张卡上相同.}\end{aligned}$$

12.5.3 简单网络

用修改后的 LeNet 作为 f_{θ} ，以便显式写出参数拷贝与 allreduce。

12.5.4 数据同步

`get_params` 把 θ 复制到设备 j 并附加梯度缓冲。allreduce 实现

$$\mathbf{g} \leftarrow \sum_{j=1}^k \mathbf{g}^{(j)},$$

再广播回每张卡。

12.5.5 数据分发

$$(\mathbf{X}, \mathbf{y}) \mapsto \{(\mathbf{X}^{(j)}, \mathbf{y}^{(j)})\}_{j=1}^k, \quad \sum_j |\mathcal{B}_j| = |\mathcal{B}|.$$

12.5.6 训练

每个小批量：分发数据、各卡前向反向、allreduce、本地更新。有效批量变为 $k|\mathcal{B}_j|$ 时， η 通常需略增大。单卡精度评估可只在一张卡上进行。

12.5.7 小结

数据并行把 $\mathcal{B}$ 切到 k 张卡，聚合梯度后再广播。更大有效批量需要相应调整 η 。

12.5.8 练习

核验：批量从 b 扩到 kb 时精度与 η 应如何同扩，以及分块 allreduce 的通信量。

12.6 多 GPU 的简洁实现

高级 API 把上一节的分发、反传与聚合收成同一训练循环。至少两张 GPU。

12.6.1 简单网络

用修改后的 ResNet-18：更小卷积核与步幅，去掉开头最大汇聚，以适应小图。

12.6.2 网络初始化

必须在目标设备上初始化 θ ，再访问该设备上的参数，否则引用落在错误设备。

12.6.3 训练

循环执行：全设备初始化；把 $\mathcal{B}$ 切到各设备；并行求损失与梯度；聚合后更新；并行算精度。当计算时间明显长于同步时间，加速比接近线性。

12.6.4 小结

单卡可自动求 $f_{\theta}(\mathbf{x})$ 。多卡时每设备先初始化，优化器自动 allreduce 梯度。

12.6.5 练习

核验：更多卡、更大 b 与 η 的组合，以及 CPU 与 GPU 算力不均时如何划分前后向。

12.7 参数服务器

从单机多卡到多机时，互连带宽从 NVLink、PCIe 降到以太网，同步算法必须适配拓扑。

12.7.1 数据并行训练

各 GPU 算 $\mathbf{g}_{ijk}$ ，先机内聚合，再跨机聚合，最后广播新 $\boldsymbol{\theta}$ 。聚合位置可以是某张 GPU、CPU，或按参数块分散到不同设备。

12.7.2 环同步（Ring Synchronization）

k 张卡组成环，每步向邻居发送 $1/k$ 的梯度块并接收一块，共 $2(k-1)$ 步后每卡得到全和。在 NVLink 带宽远高于 PCIe 的机器上，环同步走 NVLink，避免单点经 PCIe 汇聚。

12.7.3 多机训练

每台机器：读不同批量、卡间切分、本地前向反向、机内聚合，再把梯度发给参数服务器（或对等节点），取回更新后的 $\boldsymbol{\theta}$ 。机器间时钟差要求显式同步，否则同步 SGD 会等待最慢的那台。

12.7.4 键值存储

参数服务器把切片 i 上的梯度写成

$$\mathbf{g}_i = \sum_{k \in \text{workers}} \sum_{j \in \text{GPUs}} \mathbf{g}_{ijk}.$$

push 写入 $\mathbf{g}_{ijk}$, pull 读回聚合后的 $\boldsymbol{\theta}_i$ 。增加服务器个数提高聚合带宽；分层同步先机架内再机架间。

12.7.5 小结

同步时间由拓扑决定。环同步适合 NVLink 稠密连接；多机用参数服务器聚合 $\mathbf{g}_i = \sum_k \sum_j \mathbf{g}_{ijk}$ ，带宽不够就分层。

12.7.6 练习

核验：双向环能否减半延迟、计算尚未结束时异步发送对 f 收敛的影响，以及单机失效时的容错。

第十三章 计算机视觉

图像分类把整图映到类别；检测还要位置，分割还要逐像素类别。增广扩大训练分布，锚框把空间离散成候选框，转置卷积把特征图拉回像素网格。

13.1 图像增广

增广是随机映射 T ，使 $T(\mathbf{X})$ 与 $\mathbf{X}$ 同类但统计不同，从而减小对位置、颜色的依赖。

13.1.1 常用的图像增广方法

多数 T 含随机数。下面在固定示例图上重复采样 T 。

13.1.1.1 翻转和裁剪

水平翻转 T_{flip} 以 $1/2$ 概率作用。随机裁剪先采区域再缩放到固定 $H \times W$ ，降低对物体绝对位置的依赖。

13.1.1.2 改变颜色

亮度、对比度、饱和度、色调各乘以接近 1 的随机因子，例如亮度在 $[1 - \delta, 1 + \delta]$ 。

13.1.1.3 结合多种图像增广方法

复合 $T = T_k \circ \dots \circ T_1$ ，对每张训练图独立采样一次。

13.1.2 使用图像增广进行训练

训练用随机 T ，预测用确定性预处理 T_{det} （如 ToTensor）。输入形状为（批量, C, H, W ），像素在 $[0, 1]$ 。

13.1.2.1 多 GPU 训练

在 CIFAR-10 上训练 ResNet-18。优化器为 Adam。增广只作用于训练集；多卡按数据并行切批量。

13.1.3 小结

训练分布是 $\{T(\mathbf{X})\}$ ，测试分布仍是原始 $\mathbf{X}$ 。随机 T 只出现在训练。

13.1.4 练习

核验：去掉 T 后训练-测试间隙是否增大，以及复合多种 T 是否降低测试误差。

13.2 微调

源数据集大、目标数据集小。微调把源模型的特征层参数当目标模型的初值。

13.2.1 步骤

源模型 $f_{\theta_{\text{src}}}$ 在源数据上预训练。目标模型复制除输出层外的结构与参数，换上类别数为 K_{tgt} 的新输出层并随机初始化，再在目标数据上以较小 η 更新全部（或大部分）参数。

13.2.2 热狗识别

在热狗/非热狗二分类上微调 ImageNet 预训练的 ResNet-18。

13.2.2.1 获取数据集

正负类各约 1400 张，每类 1000 张训练。训练时随机裁剪并缩放到 224×224 ；测试用确定性中心裁剪。

13.2.2.2 定义和初始化模型

源模型最后线性层 $\mathbb{R}^d \rightarrow \mathbb{R}^{1000}$ 。目标模型改为 $\mathbb{R}^d \rightarrow \mathbb{R}^2$ ，特征层参数 $\theta_{\text{feat}} \leftarrow \theta_{\text{src}}$ ，输出层随机。

13.2.2.3 微调模型

特征层用小 η ，输出层可用更大 η 。对照实验把全部参数随机初始化并用更大 η 从头训练。

13.2.3 小结

除输出层外 $\theta_{\text{tgt}} \leftarrow \theta_{\text{src}}$ ，输出层从头学。微调 η 小于从头训练。

13.2.4 练习

核验：只训练输出层（冻结 θ_{feat} ）时精度，以及提高微调 η 是否破坏预训练特征。

13.3 目标检测和边界框

检测输出类别与位置。位置用矩形边界框。

13.3.1 边界框

两角表示 (x_1, y_1, x_2, y_2) 与中心表示 (x, y, w, h) 互转：

$$\begin{aligned}x &= \frac{x_1 + x_2}{2}, & y &= \frac{y_1 + y_2}{2}, & w &= x_2 - x_1, & h &= y_2 - y_1, \\x_1 &= x - w/2, & x_2 &= x + w/2, & y_1 &= y - h/2, & y_2 &= y + h/2.\end{aligned}$$

图像坐标原点在左上，向右为 x 正、向下为 y 正。

13.3.2 小结

检测 = 分类 + 框回归。框在两角与中心宽高两种坐标之间是双射。

13.3.3 练习

核验：最内维为 4 才能同时存两个角或 (x, y, w, h) 。

13.4 锚框

以像素为中心放置多尺度、多宽高比的候选框，再预测其类别与偏移。

13.4.1 生成多个锚框

给定尺度 $s_1 > \dots > s_n$ 与宽高比 $r_1, \dots, r_m$ ，每个中心生成

$$(s_1, r_1), \dots, (s_1, r_m), (s_2, r_1), \dots, (s_n, r_1)$$

共 $n + m - 1$ 个锚框。宽高由 $s\sqrt{r}$ 与 $s/\sqrt{r}$ 确定。

13.4.2 交并比 (IoU)

$$J(\mathcal{A}, \mathcal{B}) = \frac{|\mathcal{A} \cap \mathcal{B}|}{|\mathcal{A} \cup \mathcal{B}|} \in [0, 1].$$

13.4.3 在训练数据中标注锚框

每个锚框需要类别与相对真实框的偏移。预测时对所有锚框回归后再筛选。

13.4.3.1 将真实边界框分配给锚框

锚框 $A_1, \dots, A_{n_a}$ ，真实框 $B_1, \dots, B_{n_b}$ ， $n_a \geq n_b$ 。矩阵 $\mathbf{X} \in \mathbb{R}^{n_a \times n_b}$ ， $x_{ij} = J(A_i, B_j)$ 。反复取剩余子阵的最大元 (i^*, j^*) ，把 B_{j^*} 分给 A_{i^*} 并删去该行该列；然后对未分配锚框，若 $\max_j x_{ij}$ 超过阈值则赋予对应真实框，否则标为背景。

13.4.3.2 标记类别和偏移量

正锚框类别为所赋真实框类别。偏移（标准化后）为

$$\left(\frac{\frac{x_b - x_a}{w_a} - \mu_x}{\sigma_x}, \frac{\frac{y_b - y_a}{h_a} - \mu_y}{\sigma_y}, \frac{\log \frac{w_b}{w_a} - \mu_w}{\sigma_w}, \frac{\log \frac{h_b}{h_a} - \mu_h}{\sigma_h} \right).$$

13.4.3.3 一个例子

给定若干 A_i 与 B_j ，上述分配输出类别张量（0 为背景）与偏移张量。

13.4.4 使用非极大值抑制预测边界框

由锚框与预测偏移经逆变换得预测框 B ，并带类别概率 p 。NMS：按 p 排序，保留当前最高，删去与其 IoU 超过阈值的其余框，重复直到空。

13.4.5 小结

锚框是候选；IoU 是 $J(\mathcal{A}, \mathcal{B})$ ；训练标签是类别加偏移；预测用 NMS 去掉重叠框。

13.4.6 练习

核验：IoU = 0.5 的两框如何重叠，以及 NMS 阈值如何改变保留集合。

13.5 多尺度目标检测

对每个像素生成锚框数量为 $H \times W \times a$ ，输入稍大就会上百万。需在特征图网格上采样中心。

13.5.1 多尺度锚框

小目标可能位置多，用密而小的锚；大目标用疏而大的锚。特征图尺寸 (h, w) 决定中心网格。

13.5.2 多尺度检测

某尺度特征图 $Y \in \mathbb{R}^{c \times h \times w}$ 生成 hwa 个锚框。该单元的感受野内信息用于预测对应锚的类别与偏移。深层特征图感受野更大，对应更大锚。

13.5.3 小结

尺度 s 上锚框数由特征图 $h \times w$ 与每单元 a 决定。类别与偏移从该感受野的特征预测。

13.5.4 练习

核验：形状 $1 \times c \times h \times w$ 的特征如何映到类别与偏移，输出形状是什么。

13.6 目标检测数据集

用小型香蕉检测集演示：图像加框标签，而非仅类别。

13.6.1 下载数据集

图像与 CSV 标签（类别、左上、右下）成对出现。

13.6.2 读取数据集

小批量图像形状 (B, C, H, W) ；标签形状 $(B, m, 5)$ ， m 为每图最大目标数，每条为（类, x_1, y_1, x_2, y_2 ）。

13.6.3 演示

香蕉的旋转、尺度与位置在图间变化；真实框随目标走。

13.6.4 小结

检测加载器多返回真实框。类别分类没有这第五维几何。

13.6.5 练习

核验：随机裁剪若只留下目标的一小块，框标签应如何同步变换。

13.7 单发多框检测（SSD）

SSD 用基础网络加若干多尺度特征块，一次前向预测所有锚的类与偏移。

13.7.1 模型

基础网输出较大 $h \times w$ 以检小目标；后续块反复减半空间尺寸、扩大感受野以检大目标。

13.7.1.1 类别预测层

q 个物体类加背景共 $q + 1$ 。特征图 $h \times w$ 、每单元 a 个锚，需 hwa 个分类。用保持空间尺寸的卷积，输出通道 $a(q + 1)$ ，使位置 (x, y) 的通道对应该中心全部锚的类别。

13.7.1.2 边界框预测层

同样卷积，输出通道 $4a$ ，每锚 4 个偏移。

13.7.1.3 连结多尺度的预测

不同尺度的 (C, H, W) 不同。把批量维保留，将其余维展平后在锚维上拼接，得到整图全部预测。

13.7.1.4 高和宽减半块

两个填充 1 的 3×3 卷积加步幅 2 的 2×2 最大汇聚，空间减半。输出单元相对输入有 6×6 感受野。

13.7.1.5 基本网络块

串联三个减半块并倍增通道。 256×256 输入得到 32×32 特征图。

13.7.1.6 完整的模型

五块：基础网、三个减半块、全局最大汇聚到 1×1 。每块输出特征图、该尺度锚框、类别与偏移。较深层生成更大锚。

13.7.2 训练模型

前向生成多尺度锚并预测；用真实框标注；类损失加偏移损失。

13.7.2.1 读取数据集和初始化

香蕉数据集 $q = 1$ 。随机初始化后选优化算法。

13.7.2.2 定义损失函数和评价函数

类别用交叉熵。正锚偏移用 L_1 ：

$$\ell_{\text{bbox}} = \sum_{k=1}^4 |\hat{o}_k - o_k|,$$

负锚与填充锚由掩码不计入。总损失为两类之和。分类看准确率，框看平均绝对误差。

13.7.2.3 训练模型

每个批量：生成 anchors，预测 cls, bbox，由 Y 标出标签，再求损失。

13.7.3 预测目标

图像缩放到训练尺寸，得预测框后 NMS，再保留置信度不低于阈值（如 0.9）的框。

13.7.4 小结

SSD 在多尺度特征上生成锚并预测类与偏移。损失是交叉熵与正锚 L_1 之和。

13.7.5 练习

练习中的 Smooth L_1 与焦点损失分别为

$$f(x) = \begin{cases} (\sigma x)^2/2, & |x| < 1/\sigma^2, \\ |x| - 0.5/\sigma^2, & \text{否则}, \end{cases}$$

$$\ell = -\alpha(1 - p_j)^\gamma \log p_j.$$

核验二者相对 L_1 与交叉熵如何改变难例权重。

13.8 区域卷积神经网络（R-CNN）系列

R-CNN 先提区域再分类；后续工作共享卷积并学习提议。

13.8.1 R-CNN

选择性搜索得到约 2000 个提议；对每个提议独立卷积抽特征；再用分类器与回归器预测类和框。重复卷积是主要瓶颈。

13.8.2 Fast R-CNN

整图只卷积一次，得 $Y \in \mathbb{R}^{c \times h_1 \times w_1}$ 。兴趣区域汇聚把每个提议映到固定 $h \times w$ 特征，再经全连接预测类与框。

13.8.3 Faster R-CNN

用区域提议网络代替选择性搜索：在特征图上 3×3 卷积后分出物体性分数与框回归，生成较少且可学习的提议，其余与 Fast R-CNN 相同。

13.8.4 Mask R-CNN

兴趣区域对齐用双线性插值保留空间。对齐特征除预测类与框外，再经全卷积支路输出该区域的像素掩码。

13.8.5 小结

R-CNN：每区域独立卷积。Fast：整图卷积加 RoI 汇聚。Faster：RPN 学提议。Mask：对齐加像素掩码。

13.8.6 练习

核验：把检测写成一次回归（如 YOLO）与多阶段提议在输出形式上的差别。

13.9 语义分割和数据集

语义分割给每个像素一个类别，边界是像素级的，不是框。

13.9.1 图像分割和实例分割

图像分割按像素相似性分区，不必有语义标签。实例分割还要区分同类的不同物体；语义分割把同类像素标成同一类。

13.9.2 Pascal VOC2012 语义分割数据集

输入 JPEG 与同尺寸的彩色标签图，同色像素同类。

13.9.2.1 预处理数据

缩放会错位像素类别。改为对图像与标签做同一随机裁剪到固定 $H \times W$ 。

13.9.2.2 自定义语义分割数据集类

索引 i 返回 $(X^{(i)}, Y^{(i)})$ ， Y 是像素类别。过小图过滤；RGB 按通道标准化。

13.9.2.3 读取数据集

裁剪 320×480 后统计保留样本。标签小批量是三维数组 (B, H, W) 。

13.9.2.4 整合所有组件

加载器返回训练与测试迭代器。

13.9.3 小结

分割标签与输入像素一一对应，故用同步裁剪而非缩放。

13.9.4 练习

核验：分类里常用的颜色抖动若不同步改标签，分割监督是否失效。

13.10 转置卷积

普通卷积缩小空间。转置卷积把特征图放大，便于像素级输出。

13.10.1 基本操作

输入 $n_h \times n_w$ 、核 $k_h \times k_w$ 、步幅 1、无填充。每个输入标量乘核，放到与该输入位置对应的输出块，所有块相加。输出空间为 $(n_h + k_h - 1) \times (n_w + k_w - 1)$ 。

13.10.2 填充、步幅和多通道

填充作用在输出上：两侧填充 1 会删掉输出最外一行一列。步幅作用在中间结果的间距上，步幅 2 使输出更稀、更大。多通道与普通卷积相同： c_i 入、 c_o 出。

13.10.3 与矩阵变换的联系

卷积 $Y = f(X)$ 可写成 $\text{vec}(Y) = \mathbf{W} \text{vec}(X)$ 。转置卷积对应 $\text{vec}(X') = \mathbf{W}^T \text{vec}(Y)$ ，故正向与反向在卷积与转置卷积之间对换。若 g 与 f 超参数对应且通道对齐，则 $g(f(X))$ 与 X 同空间形状。

13.10.4 小结

转置卷积经核广播输入，增大空间。它是卷积矩阵的转置作用在向量化特征上。

13.10.5 练习

核验： X 与 $g(f(X))$ 形状相同是否蕴含数值相同。

13.11 全卷积网络

FCN 把分类网改成像素分类：特征、 1×1 卷积改通道、转置卷积恢复 $H \times W$ 。

13.11.1 构造模型

预训练 ResNet-18 去掉全局平均汇聚与全连接。 1×1 卷积把通道变为类别数 K ，再转置卷积使空间回到输入尺寸。输出 $\hat{Y} \in \mathbb{R}^{K \times H \times W}$ ，通道维是逐像素 logits。

13.11.2 初始化转置卷积层

双线性上采样：把输出坐标 (x, y) 映到输入实坐标 (x', y') ，用最近四像素按距离加权。该核可作为转置卷积的初值。

13.11.3 读取数据集

VOC 随机裁剪 320×480 ，使高宽可被 32 整除（与步幅一致）。

13.11.4 训练

损失是逐像素交叉熵，在通道维上做 Softmax。准确率是像素类别是否正确的平均。

13.11.5 预测

输入标准化为四维。类别映回调色板颜色。若 H 或 W 不能被 32 整除，切多块可整除区域，重叠像素对 logits 取平均再分类。

13.11.6 小结

FCN: $\hat{Y} = g_{H,W}(h_{1 \times 1}(f(X)))$ ， g 为转置卷积，可用双线性核初始化。

13.11.7 练习

核验：转置卷积改 Xavier 随机初始化对分割精度的影响。

13.12 风格迁移

内容图 X_c 与风格图 X_s 。合成图 X 是唯一优化变量；预训练 CNN 只抽特征。

13.12.1 方法

初始化 $X \leftarrow X_c$ 。选若干层作内容层、若干层作风格层。最小化内容损失、风格损失与全变分损失的加权和，只更新 X 。

13.12.2 阅读内容和风格图像

两图空间尺寸一般不同，后续会缩放到同一训练尺寸。

13.12.3 预处理和后处理

预处理：RGB 标准化并变成卷积输入格式。后处理：反标准化并把像素裁到 $[0, 1]$ 。

13.12.4 抽取图像特征

VGG-19。内容层取较深卷积以去细节；风格层取各块较浅卷积以同时匹配局部与全局纹理。

13.12.5 定义损失函数

三项加权。

13.12.5.1 内容损失

内容层输出 $\phi_c(X)$ 与 $\phi_c(X_c)$ 的平方误差。

13.12.5.2 风格损失

风格层输出整形为 $X \in \mathbb{R}^{c \times hw}$ ，格拉姆矩阵

$$G = XX^T \in \mathbb{R}^{c \times c}.$$

风格损失为 $\|G(X) - G(X_s)\|_F^2$ （对各风格层求和）。

13.12.5.3 全变分损失

$$\sum_{i,j} (|x_{i,j} - x_{i+1,j}| + |x_{i,j} - x_{i,j+1}|)$$

鼓励相邻像素接近，减少噪点。

13.12.5.4 损失函数

$$\ell = \alpha \ell_{\text{content}} + \beta \ell_{\text{style}} + \gamma \ell_{\text{TV}}.$$

权重控制保内容、迁风格与去噪的折中。

13.12.6 初始化合成图像

把 $\mathbf{X}$ 当作模型参数，前向即返回该参数。风格图的格拉姆在训练前算一次。

13.12.7 训练模型

反复抽 $\mathbf{X}$ 的内容/风格特征，求 ℓ ，对 $\mathbf{X}$ 求梯度并更新。

13.12.8 小结

$\ell = \alpha \ell_{\text{content}} + \beta \ell_{\text{style}} + \gamma \ell_{\text{TV}}$ ，风格由 $\mathbf{G} = \mathbf{X} \mathbf{X}^\top$ 表达，只更新合成图像。

13.12.9 练习

核验：改内容/风格层集合或 (α, β, γ) 时， $\mathbf{X}$ 更贴近哪一张输入。

13.13 实战 Kaggle 比赛：图像分类 (CIFAR-10)

从原始 PNG 组织数据，再用卷积与增广做十分类。

13.13.1 获取并组织数据集

训练 5×10^4 张，测试 3×10^5 张（其中 10^4 用于评分）。图为 32×32 RGB，十类。

13.13.1.1 下载数据集

train/、test/ 与 trainLabels.csv。

13.13.1.2 整理数据集

按标签分文件夹。验证比例 r ，每类验证张数为 $\max(\lfloor nr \rfloor, 1)$ ， n 为最小类的训练张数。

13.13.2 图像增广

训练：随机水平翻转与 RGB 标准化。测试：仅标准化。

13.13.3 读取数据集

训练加载器施用随机 T ；验证与最终训练（训练 + 验证）用确定性变换。

13.13.4 定义模型

ResNet-18，输出 10 类。

13.13.5 定义训练函数

按验证集指标选超参。学习率可每 T 个周期乘衰减因子。

13.13.6 训练和验证模型

可调批量、周期、 η 、衰减周期与因子。演示可用较少周期。

13.13.7 在 Kaggle 上对测试集进行分类并提交结果

用全部有标签数据重训，对测试集写类别，生成提交表。

13.13.8 小结

文件数据集 $\rightarrow$ 按类目录 $\rightarrow$ 增广加载器 $\rightarrow f_{\theta}$ 的十分类。

13.13.9 练习

核验：完整数据与给定超参下的测试精度，以及去掉增广后的精度差。

13.14 实战 Kaggle 比赛：狗的品种识别（ImageNet Dogs）

120 类犬种，图像大于 CIFAR，且尺寸不一，来自 ImageNet 子集。

13.14.1 获取和整理数据集

训练约 10^4 张，测试约 10^4 张 JPEG。

13.14.1.1 下载数据集

`labels.csv` 与 `train/`、`test/`。

13.14.1.2 整理数据集

与 CIFAR 相同：读标签、拆验证、按类建目录。

13.14.2 图像增广

更大图上用随机裁剪缩放到网络输入尺寸，测试用确定性缩放与裁剪。

13.14.3 读取数据集

加载器构造与 CIFAR 一节相同，只是 T 不同。

13.14.4 微调预训练模型

ResNet-34 在 ImageNet 上预训练，冻结特征，只训练小型输出头（如两层全连接，120 类）。不反传主干以省计算与显存。

13.14.5 定义训练函数

`train` 只迭代输出头参数。按验证指标选头。

13.14.6 训练和验证模型

η 每若干周期乘衰减。周期与衰减可调。

13.14.7 对测试集分类并在 Kaggle 提交结果

用全部有标签数据训练输出头，对测试集写 120 维概率或类别。

13.14.8 小结

大图子集上，冻结 ImageNet 主干 f ，只学 $\mathbb{R}^d \rightarrow \mathbb{R}^{120}$ 的头，并配以更强的空间增广。

13.14.9 练习

核验：更大主干与不同 (η, T_{decay}) 是否降低验证误差。

第十四章 自然语言处理：预训练

词是离散符号。预训练把词或词元映到向量，使共现与上下文进入内积。先写 word2vec 与 GloVe，再写子词与 BERT 的掩蔽语言建模、下一句预测。

14.1 词嵌入 (word2vec)

词向量 $\mathbf{v} \in \mathbb{R}^d$ 是词的特征。词嵌入是映射 $w \mapsto \mathbf{v}_w$ 。

14.1.1 为何独热向量是一个糟糕的选择

词表 $\mathcal{V}$ 上的独热 $\mathbf{e}_w$ 两两正交，余弦

$$\frac{\mathbf{x}^\top \mathbf{y}}{\|\mathbf{x}\| \|\mathbf{y}\|} \in [-1, 1]$$

对任何不同词都是 0，无法表达相似。

14.1.2 自监督的 word2vec

跳元 (skip-gram) 用中心词预测上下文；连续词袋 (CBOW) 用上下文预测中心词。二者都用语料中的共现，不另需标签。

14.1.3 跳元模型 (Skip-Gram)

由 “loves” 生成两侧词的联合，并假定条件独立：

$$\begin{aligned} &P(\text{the, man, his, son} \mid \text{loves}) \\ &\approx P(\text{the} \mid \text{loves}) P(\text{man} \mid \text{loves}) P(\text{his} \mid \text{loves}) P(\text{son} \mid \text{loves}). \end{aligned}$$

一般地，中心词向量 $\mathbf{v}_c$ 、上下文向量 $\mathbf{u}_o$ ，

$$P(w_o \mid w_c) = \frac{\exp(\mathbf{u}_o^\top \mathbf{v}_c)}{\sum_{i \in \mathcal{V}} \exp(\mathbf{u}_i^\top \mathbf{v}_c)}.$$

长度为 T 、窗口 m 的语料似然

$$\prod_{t=1}^T \prod_{\substack{-m \leq j \leq m \\ j \neq 0}} P(w^{(t+j)} | w^{(t)}).$$

14.1.3.1 训练

最小化

$$-\sum_{t=1}^T \sum_{\substack{-m \leq j \leq m \\ j \neq 0}} \log P(w^{(t+j)} | w^{(t)}).$$

对数条件概率

$$\log P(w_o | w_c) = \mathbf{u}_o^\top \mathbf{v}_c - \log \sum_{i \in \mathcal{V}} \exp(\mathbf{u}_i^\top \mathbf{v}_c).$$

对 $\mathbf{v}_c$ 的梯度

$$\frac{\partial \log P(w_o | w_c)}{\partial \mathbf{v}_c} = \mathbf{u}_o - \sum_{j \in \mathcal{V}} P(w_j | w_c) \mathbf{u}_j.$$

每步与 $|\mathcal{V}|$ 成正比。

14.1.4 连续词袋 (CBOW) 模型

$$P(\text{loves} | \text{the, man, his, son}).$$

上下文平均 $\bar{\mathbf{v}}_o = \frac{1}{2m}(\mathbf{v}_{o_1} + \dots + \mathbf{v}_{o_{2m}})$,

$$P(w_c | \mathcal{W}_o) = \frac{\exp(\mathbf{u}_c^\top \bar{\mathbf{v}}_o)}{\sum_{i \in \mathcal{V}} \exp(\mathbf{u}_i^\top \bar{\mathbf{v}}_o)}.$$

序列似然

$$\prod_{t=1}^T P(w^{(t)} | w^{(t-m)}, \dots, w^{(t-1)}, w^{(t+1)}, \dots, w^{(t+m)}).$$

14.1.4.1 训练

$$-\sum_{t=1}^T \log P(w^{(t)} | w^{(t-m)}, \dots, w^{(t+m)}).$$

$$\log P(w_c | \mathcal{W}_o) = \mathbf{u}_c^\top \bar{\mathbf{v}}_o - \log \sum_{i \in \mathcal{V}} \exp(\mathbf{u}_i^\top \bar{\mathbf{v}}_o).$$

$$\frac{\partial \log P(w_c | \mathcal{W}_o)}{\partial \mathbf{v}_{o_i}} = \frac{1}{2m} \left(\mathbf{u}_c - \sum_{j \in \mathcal{V}} P(w_j | \mathcal{W}_o) \mathbf{u}_j \right).$$

14.1.5 小结

跳元: $P(w_o | w_c) \propto \exp(\mathbf{u}_o^\top \mathbf{v}_c)$ 。CBOW: $P(w_c | \mathcal{W}_o) \propto \exp(\mathbf{u}_c^\top \bar{\mathbf{v}}_o)$ 。Softmax 分母扫过整个 $\mathcal{V}$ 。

14.1.6 练习

核验: 梯度对 $|\mathcal{V}|$ 的复杂度, 以及 $\mathbf{u}^\top \mathbf{v}$ 与余弦的关系。

14.2 近似训练

精确 Softmax 每步求和 $|\mathcal{V}|$ 项。负采样与层序 Softmax 把代价降到常数或对数。

14.2.1 负采样

把“中心-上下文是否共现”当成二分类

$$P(D = 1 | w_c, w_o) = \sigma(\mathbf{u}_o^\top \mathbf{v}_c), \quad \sigma(x) = \frac{1}{1 + \exp(-x)}.$$

正对似然再乘 K 个噪声词的负例

$$P(w^{(t+j)} | w^{(t)}) = P(D = 1 | w^{(t)}, w^{(t+j)}) \prod_{k=1}^K P(D = 0 | w^{(t)}, w_k).$$

损失

$$-\log P(w^{(t+j)} | w^{(t)}) = -\log \sigma(\mathbf{u}_{i_{t+j}}^\top \mathbf{v}_{i_t}) - \sum_{k=1}^K \log \sigma(-\mathbf{u}_{h_k}^\top \mathbf{v}_{i_t}).$$

每步与 K 线性, 与 $|\mathcal{V}|$ 无关。

14.2.2 层序 Softmax

词表做成二叉树。从根到叶 w_o 的路径长度为 $L(w_o)$,

$$P(w_o | w_c) = \prod_{j=1}^{L(w_o)-1} \sigma(\mathbb{I}[n(w_o, j+1) = \text{leftChild}(n(w_o, j))] \mathbf{u}_{n(w_o, j)}^\top \mathbf{v}_c).$$

路径上向左与向右对应 $+\mathbf{u}^\top \mathbf{v}$ 与 $-\mathbf{u}^\top \mathbf{v}$ 。对任意 w_c 有 $\sum_{w \in \mathcal{V}} P(w | w_c) = 1$ 。代价为 $O(\log |\mathcal{V}|)$ 。

14.2.3 小结

负采样用 $1 + K$ 次 $\sigma(\mathbf{u}^\top \mathbf{v})$ ；层序 Softmax 用根到叶的 $O(\log |\mathcal{V}|)$ 次。

14.2.4 练习

核验： $\sum_w P(w | w_c) = 1$ 在二叉树上是否成立，以及噪声分布 $P(w)$ 如何取。

14.3 用于预训练词嵌入的数据集

把 PTB 文本变成跳元 + 负采样所需的小批量。

14.3.1 读取数据集

一行一句、空格分词。频次低于阈值的词并入未知词元。

14.3.2 下采样

词 w_i 被丢弃的概率

$$P(w_i) = \max\left(1 - \sqrt{\frac{t}{f(w_i)}}, 0\right),$$

f 为相对频率， t 为阈值。高频词更常被丢掉。

14.3.3 中心词和上下文词的提取

对每个中心位置均匀抽窗口半径 1 到 m ，距离不超过该半径的词为上下文。

14.3.4 负采样

噪声 w 的抽样概率正比于 $f(w)^{0.75}$ 。每对正例抽 K 个噪声词。

14.3.5 小批量加载训练实例

第 i 个样本有 n_i 个上下文与 m_i 个噪声。拼接到长度 $\max_i(n_i + m_i)$ 并补零；掩码标出非填充；标签区分正负例。

14.3.6 整合代码

迭代器一次产出中心词、上下文加噪声、掩码与标签。

14.3.7 小结

下采样降高频；批量用掩码对齐变长的正负例。

14.3.8 练习

核验：关闭下采样对每个 epoch 词元数与墙钟时间的影响。

14.4 预训练 word2vec

在 PTB 上用负采样训练跳元。

14.4.1 跳元模型

两个嵌入：中心与上下文（含噪声）。

14.4.1.1 嵌入层

权重 $\mathbf{E} \in \mathbb{R}^{|\mathcal{V}| \times d}$ 。索引 i 取第 i 行。批量索引形状 (B, L) 时输出 (B, L, d) 。

14.4.1.2 定义前向传播

中心 $(B, 1, d)$ 与上下文-噪声 $(B, \text{max_len}, d)$ 做批量点积，得 $(B, 1, \text{max_len})$ ，每个元素是 $\mathbf{v}_c^\top \mathbf{u}$ 。

14.4.2 训练

损失只在非填充位置上平均。

14.4.2.1 二元交叉熵损失

对每个正负位置用 $-\left[y \log \sigma(s) + (1 - y) \log(1 - \sigma(s))\right]$ ，再按掩码平均，与负采样公式一致。

14.4.2.2 初始化模型参数

两张嵌入表，行数为 $|\mathcal{V}|$ ，例如 $d = 100$ 。

14.4.2.3 定义训练阶段代码

每个批量：前向点积、掩码交叉熵、反传更新两张表。

14.4.3 应用词嵌入

查询词 v ，在词表中取余弦最大的 k 个 v' 。

14.4.4 小结

跳元前向是 $\mathbf{v}_c^T \mathbf{u}$ ；训练是掩码二元交叉熵。下游用余弦找近邻。

14.4.5 练习

核验：同一中心词在不同轮次重抽上下文与噪声，对更新方差的影响。

14.5 全局向量的词嵌入（GloVe）

用全局共现计数 x_{ij} 解释跳元，再用平方损失拟合 $\log x_{ij}$ 。

14.5.1 带全局语料统计的跳元模型

$$q_{ij} = \frac{\exp(\mathbf{u}_j^T \mathbf{v}_i)}{\sum_{k \in \mathcal{V}} \exp(\mathbf{u}_k^T \mathbf{v}_i)},$$

共现加权交叉熵

$$-\sum_{i \in \mathcal{V}} \sum_{j \in \mathcal{V}} x_{ij} \log q_{ij} = -\sum_{i \in \mathcal{V}} x_i \sum_{j \in \mathcal{V}} p_{ij} \log q_{ij},$$

其中 $x_i = \sum_j x_{ij}$, $p_{ij} = x_{ij}/x_i$ 。

14.5.2 GloVe 模型

$$\sum_{i \in \mathcal{V}} \sum_{j \in \mathcal{V}} h(x_{ij}) (\mathbf{u}_j^T \mathbf{v}_i + b_i + c_j - \log x_{ij})^2.$$

h 对大计数截断，避免极高频对支配损失。中心向量与上下文向量在此目标下地位对称。

14.5.3 从条件概率比值理解 GloVe 模型

希望

$$f(\mathbf{u}_j, \mathbf{u}_k, \mathbf{v}_i) \approx \frac{p_{ij}}{p_{ik}}.$$

取指数内积比

$$\frac{\exp(\mathbf{u}_j^\top \mathbf{v}_i)}{\exp(\mathbf{u}_k^\top \mathbf{v}_i)} \approx \frac{p_{ij}}{p_{ik}},$$

从而

$$\mathbf{u}_j^\top \mathbf{v}_i + b_i + c_j \approx \log x_{ij}.$$

14.5.4 小结

GloVe 用加权平方损失拟合 $\log x_{ij}$ ，而不是对整个 $\mathcal{V}$ 做 Softmax。比值 p_{ij}/p_{ik} 由内积差解释。

14.5.5 练习

核验：用窗口内距离重加权 p_{ij} 后目标如何变，以及 b_i 与 c_i 是否可对换。

14.6 子词嵌入

变形、复合词共享字符片段。把词向量写成子词向量之和。

14.6.1 fastText 模型

词 w 的字符 n -gram 集合 $\mathcal{G}_w$ (含词本身),

$$\mathbf{v}_w = \sum_{g \in \mathcal{G}_w} \mathbf{z}_g.$$

跳元仍用 $\mathbf{u}_o^\top \mathbf{v}_c$ ，但 $\mathbf{v}_c$ 可加性分解。未见词只要子词在词表中即可表示。

14.6.2 字节对编码 (Byte Pair Encoding)

从单字符开始，反复合并当前语料中最频的相邻符号对，得到可变长子词，词表大小预先固定。不跨词边界合并。

14.6.3 小结

fastText: $\mathbf{v}_w = \sum_{g \in \mathcal{G}_w} \mathbf{z}_g$ 。BPE 贪心合并最频对，得到固定词表上的可变长子词。

14.6.4 练习

核验：从 n 个初始符号得到词表大小 m 需要多少次合并。

14.7 词的相似性和类比任务

大语料上预训练的向量可直接用于近邻与类比。

14.7.1 加载预训练词向量

GloVe 有 50/100/300 维；fastText 有 300 维英文向量。词表含四十万词加未知词元。

14.7.2 应用预训练词向量

相似度用余弦； k 近邻在词表上穷举。

14.7.2.1 词相似度

对查询 w ，返回 $\arg \max_{w' \neq w} \cos(\mathbf{v}_w, \mathbf{v}_{w'})$ 的前 k 个。

14.7.2.2 词类比

$a : b :: c : d$ 中求

$$d = \underset{w}{\operatorname{argmin}} \|\mathbf{v}_w - (\mathbf{v}_c + \mathbf{v}_b - \mathbf{v}_a)\|$$

(或最大余弦)。例如性别与首都-国家。

14.7.3 小结

近邻是 $\cos(\mathbf{v}, \mathbf{v}')$ ；类比是 $\mathbf{v}_c + \mathbf{v}_b - \mathbf{v}_a$ 的最近邻。

14.7.4 练习

核验：词表很大时如何用近似近邻代替全表扫描。

14.8 来自 Transformers 的双向编码器表示（BERT）

word2vec/GloVe 的 v_w 与上下文无关。BERT 用双向 Transformer 编码器，使表示依赖整句。

14.8.1 从上下文无关到上下文敏感

上下文无关： $f(x)$ 只摄入词元 x 。多义词在不同句中应不同； $f(x, \text{上下文})$ 才能区分。

14.8.2 从特定于任务到不可知任务

ELMo 双向但下游仍要定制结构；GPT 任务无关但从左到右。微调时 GPT 更新全部解码器参数，ELMo 常冻结。

14.8.3 BERT：把两个最好的结合起来

双向编码 + 下游只加一个浅输出层，并微调全部编码器参数。

14.8.4 输入表示

单句： $\langle \text{cls} \rangle + \text{词元} + \langle \text{sep} \rangle$ 。句对： $\langle \text{cls} \rangle + A + \langle \text{sep} \rangle + B + \langle \text{sep} \rangle$ 。输入嵌入为词元嵌入、片段嵌入与位置嵌入之和。

14.8.5 预训练任务

编码器输出每个词元（含特殊词元）的向量，供两个自监督头使用。

14.8.5.1 掩蔽语言模型（Masked Language Modeling）

随机选 15% 词元做预测。对被选词元：以 80% 换成 $\langle \text{mask} \rangle$ ，10% 换成随机词，10% 保持原词，再用双向上下文预测原词。避免微调阶段从未见过 $\langle \text{mask} \rangle$ 导致的不一致。

14.8.5.2 下一句预测（Next Sentence Prediction）

句对一半为真实邻句（正），一半第二句随机（负）。 $\langle \text{cls} \rangle$ 的表示经单隐层 MLP 做二分类。

14.8.6 整合代码

总损失为 MLM 与 NSP 损失的和。前向返回编码、MLM logits 与 NSP logits。

14.8.7 小结

BERT 输入是三类嵌入之和。预训练

$$\ell = \ell_{\text{MLM}} + \ell_{\text{NSP}},$$

MLM 给双向上下文，NSP 给句对关系。

14.8.8 练习

核验：MLM 相对从左到右 LM 需要更多还是更少步才能收敛，以及 GELU 与 ReLU 的差别。

14.9 用于预训练 BERT 的数据集

在 WikiText-2 上生成 MLM 与 NSP 样本。保留标点、大小写与数字。

14.9.1 为预训练任务定义辅助函数

从段落造句对，再对 BERT 序列做掩蔽。

14.9.1.1 生成下一句预测任务的数据

以 1/2 概率取下句为真邻句，否则从语料随机抽一句。截断使总长不超过 `max_len`。

14.9.1.2 生成遮蔽语言模型任务的数据

在非特殊词元中抽约 15% 位置；按 80/10/10 替换；返回改写后的词元、预测位置与原词标签。

14.9.2 将文本转换为预训练数据集

填充到固定长，附片段下标、有效长度掩码、MLM 标签与 NSP 标签。一条样本对应一对句子上的两个任务。

14.9.3 小结

WikiText-2 样本是填充后的 (词元, 片段, MLM 位置与标签, NSP 标签)。

14.9.4 练习

核验：不用句号切句、不过滤稀有词元时词表有多大。

14.10 预训练 BERT

用上一节样本训练小型 BERT，再用编码器表示文本。

14.10.1 预训练 BERT

原论文 BERT_{BASE}：12 层、768 隐、12 头，约 1.1×10^8 参数；BERT_{LARGE}：24 层、1024、16 头，约 3.4×10^8 。演示用 2 层、128 隐、2 头。批量损失为该批量 MLM 与 NSP 损失之和。

14.10.2 用 BERT 表示文本

单句或句对加上特殊词元后编码。 $\langle \text{cls} \rangle$ 位是整段表示；同一词元（如 crane）在不同句中的向量不同。

14.10.3 小结

预训练后 $f_{\text{BERT}}(x, \text{上下文})$ 随上下文变。 $\langle \text{cls} \rangle$ 与各词元向量都可作下游输入。

14.10.4 练习

核验：为何实验中 MLM 损失高于 NSP，以及 $\text{max_len} = 512$ 配大模型时显存是否溢出。

第十五章 自然语言处理：应用

预训练向量或 BERT 编码器提供 $w \mapsto v$ 或序列 $\mapsto H$ 。下游把可变长文本映到固定标签：情感、句对关系、词元标签或问答跨度。

15.1 情感分析及数据集

情感分析是文本分类：序列 $w_{1:T}$ 映到极性 $y \in \{0, 1\}$ 。数据为 IMDb 影评，正负标签。

15.1.1 读取数据集

每个样本是评论文本与 $y \in \{0, 1\}$ 。

15.1.2 预处理数据集

词元为词，低频过滤后建词表。长度直方图显示 T 变化大；截断或填充到固定 $L = 500$ 。

15.1.3 创建数据迭代器

每个批量返回 (X, y) ， $X \in \mathbb{Z}^{B \times L}$ 为词元下标。

15.1.4 整合代码

加载函数返回训练/测试迭代器与词表。

15.1.5 小结

情感是 $\{w_t\}_{t=1}^T \mapsto y \in \{0, 1\}$ 。变长 T 经截断填充变成固定 L 。

15.1.6 练习

核验：哪些预处理超参缩短每个 epoch，以及如何把另一评论语料映到同一迭代器接口。

15.2 情感分析：使用循环神经网络

小数据集上用冻结的预训练词向量，再用双向 RNN 把序列压成一个向量。

15.2.1 使用循环神经网络表示单个文本

嵌入 $\mathbf{e}_t = \mathbf{E}[w_t]$ (GloVe)。双向 LSTM 最后一层的初始与最终时刻隐状态拼接为 $\mathbf{h}$ ，再

$$\hat{\mathbf{y}} = \text{softmax}(\mathbf{W}\mathbf{h} + \mathbf{b}) \in \mathbb{R}^2.$$

15.2.2 加载预训练的词向量

$\mathbf{E}$ 用 100 维 GloVe 初始化，训练中不更新，使 $\mathbf{e}_t$ 固定。

15.2.3 训练和评估模型

最小化交叉熵。推理时把新句词元化、填充后取 $\hat{\mathbf{y}} = \arg \max \hat{\mathbf{y}}$ 。

15.2.4 小结

$$\mathbf{h} = [\vec{\mathbf{h}}_1; \overleftarrow{\mathbf{h}}_T], \quad \hat{\mathbf{y}} = \text{softmax}(\mathbf{W}\mathbf{h} + \mathbf{b}),$$

其中 $\mathbf{e}_t$ 来自冻结 GloVe。

15.2.5 练习

核验：增大周期或改用 300 维 GloVe 是否降低测试误差。

15.3 情感分析：使用卷积神经网络

把序列看成一维“图像”，用卷积抽 n 元语法，再用时间最大汇聚得到定长向量。

15.3.1 一维卷积

核 k 在序列上滑动，输出

$$y_i = \sum_{j=0}^{k-1} x_{i+j} k_j.$$

多通道时等价于把嵌入维当作二维卷积的通道。

15.3.2 最大时间汇聚层

每个通道在时间维取 $\max$ ，得到与通道数相同的向量，允许各通道有效长度不同。

15.3.3 textCNN 模型

输入形状视为宽 n 、高 1、通道 d 。若干不同宽度的一维核抽局部特征；各通道时间最大汇聚后拼接；全连接加 Dropout 得到两类。

15.3.3.1 定义模型

两套嵌入：一套可训练，一套固定 GloVe；通道上拼接后再卷积。例如核宽 $\{3, 4, 5\}$ ，每类 100 个输出通道。

15.3.3.2 加载预训练词向量

可训练表与固定表都用 100 维 GloVe 初始化，后者不更新。

15.3.3.3 训练和评估模型

交叉熵训练。推理同 RNN 节。

15.3.4 小结

一维卷积抽 n 元，时间最大汇聚得 $z = \text{concat}_c \max_t(Y_{c,t})$ ，再线性分类。

15.3.5 练习

核验：与双向 LSTM 在精度和吞吐上的差别，以及加上位置编码是否改变 $\hat{y}$ 。

15.4 自然语言推断与数据集

NLI 判断假设能否由前提推出。标签为蕴涵、矛盾、中性。

15.4.1 自然语言推断

前提 A 、假设 B 都是词元序列。输出

$$y \in \{\text{蕴涵}, \text{矛盾}, \text{中性}\}.$$

15.4.2 斯坦福自然语言推断 (SNLI) 数据集

约 5×10^5 带标签英文句对。

15.4.2.1 读取数据集

抽取 (A, B, y) 。打印可见三类在训练与测试上大致均衡。

15.4.2.2 定义用于加载数据集的类

长度统一为 `num_steps`：超长截断，不足填填充词元。下标 i 返回一对前提、假设与标签。

15.4.2.3 整合代码

词表只由训练集构建。批量里有两个输入张量，分别是前提与假设。

15.4.3 小结

NLI 是句对分类 $y = f(A, B)$ 。SNLI 提供三类均衡的监督。

15.4.4 练习

核验：用 NLI 模型给机器翻译候选打“是否蕴涵参考”的分数是否可行。

15.5 自然语言推断：使用注意力

可分解注意力：对齐、比较、聚合，无循环、无卷积。

15.5.1 模型

前提词向量 $\mathbf{a}_1, \dots, \mathbf{a}_m$, 假设 $\mathbf{b}_1, \dots, \mathbf{b}_n$ 。

15.5.1.1 注意 (Attending)

$$e_{ij} = f(\mathbf{a}_i)^\top f(\mathbf{b}_j).$$

软对齐

$$\begin{aligned}\beta_i &= \sum_{j=1}^n \frac{\exp(e_{ij})}{\sum_{k=1}^n \exp(e_{ik})} \mathbf{b}_j, \\ \alpha_j &= \sum_{i=1}^m \frac{\exp(e_{ij})}{\sum_{k=1}^m \exp(e_{kj})} \mathbf{a}_i.\end{aligned}$$

f 先把向量映到较低维再做内积, 复杂度对 m, n 是线性而非二次深层交互。

15.5.1.2 比较

$$\begin{aligned}\mathbf{v}_{A,i} &= g([\mathbf{a}_i, \beta_i]), \quad i = 1, \dots, m, \\ \mathbf{v}_{B,j} &= g([\mathbf{b}_j, \alpha_j]), \quad j = 1, \dots, n.\end{aligned}$$

15.5.1.3 聚合

$$\begin{aligned}\mathbf{v}_A &= \sum_{i=1}^m \mathbf{v}_{A,i}, & \mathbf{v}_B &= \sum_{j=1}^n \mathbf{v}_{B,j}, \\ \hat{\mathbf{y}} &= h([\mathbf{v}_A, \mathbf{v}_B]).\end{aligned}$$

h 输出三类 logits。

15.5.1.4 整合代码

注意、比较、聚合联合训练, 参数即 f, g, h 中的 MLP。

15.5.2 训练和评估模型

在 SNLI 上优化交叉熵。

15.5.2.1 读取数据集

批量 256，序列长 50。

15.5.2.2 创建模型

$f(\mathbf{a}_i)$ 与词向量 100 维； f, g 输出 200 维。GloVe 初始化词表。

15.5.2.3 训练和评估模型

句对作为两个输入分到多卡。记录训练与测试准确率。

15.5.2.4 使用模型

新的 (A, B) 词元化后取 $\arg \max \hat{\mathbf{y}}$ 。

15.5.3 小结

对齐 $e_{ij} = f(\mathbf{a}_i)^\top f(\mathbf{b}_j)$ ，比较 $g([\mathbf{a}_i, \mathbf{b}_i])$ ，聚合后 $h([\mathbf{v}_A, \mathbf{v}_B])$ 得三类。

15.5.4 练习

核验：该分解无法对高阶跨句交互建模时误差如何表现，以及如何改成输出 $[0, 1]$ 相似度。

15.6 针对序列级和词元级应用微调 BERT

BERT 编码后只加浅层。序列级用 $\langle \text{cls} \rangle$ ；词元级用每个位置的向量。额外层从头学，编码器全部微调。

15.6.1 单文本分类

输入一条序列。 $\langle \text{cls} \rangle$ 向量经线性层得 K 类，如情感或语法可接受性。

15.6.2 文本对分类或回归

句对打包进一条 BERT 序列，片段嵌入区分两句。分类：NLI 三类。回归：语义相似度分数，例如 $[0, 5]$ 上的实数，损失用平方误差。

15.6.3 文本标注

每个词元的 BERT 向量送到共享（或位置共享）的分类头，如词性。 $\langle \text{cls} \rangle$ 、 $\langle \text{sep} \rangle$ 通常不预测标签。

15.6.4 问答

段落与问题拼成句对。在段落词元上预测答案起点与终点的两个分类器，答案跨度为 $\arg \max_{s \leq e} P_{\text{start}}(s)P_{\text{end}}(e)$ 。

15.6.5 小结

序列级： $\hat{y} = h(\mathbf{h}_{\langle \text{cls} \rangle})$ 。词元级： $\hat{y}_t = h(\mathbf{h}_t)$ 。问答再加起点终点两个头。

15.6.6 练习

核验：检索中如何用负采样与 BERT 打分，以及 BERT 作编码器时 MLM 头如何改回标准 LM。

15.7 自然语言推断：微调 BERT

把 SNLI 当作文本对分类，在预训练 BERT 上加 MLP。

15.7.1 加载预训练的 BERT

小模型便于演示；base 与原 BERT 基础规模相当。加载词表与预训练参数。

15.7.2 微调 BERT 的数据集

前提与假设打包为 BERT 输入，片段下标区分两句。超长时轮流删较长一句的末词元直到不超过 `max_len`。

15.7.3 微调 BERT

$\langle \text{cls} \rangle$ 表示经两层全连接得三类。编码器与嵌入参与微调；仅与预训练 MLM/NSP 头相关、且未接入该分类器的参数不更新。

15.7.4 小结

$$\hat{\mathbf{y}} = \text{MLP}(\mathbf{h}_{\langle \text{cls} \rangle}(A, B)) \in \mathbb{R}^3.$$

BERT 成为下游模型的子图，预训练任务头不再前向。

15.7.5 练习

核验：按两句长度比截断与轮流删末词元哪种更少切断前提或假设的关键词。

附录 A 深度学习工具

附录把运行环境写成可重复的映射：笔记本、云实例、硬件选择与贡献流程。计算仍落在同一套张量运算上；差别在于谁提供 CPU/GPU 以及代码如何进入仓库。

A.1 使用 Jupyter Notebook

笔记本是交错的叙述单元与代码单元。本地或远程内核执行单元，返回张量与打印。

A.1.1 在本地编辑和运行代码

工作目录指向本书代码后启动笔记本服务，浏览器连到本地端口。打开 `.ipynb` 即可按单元求值。

A.1.2 高级选项

原生笔记本含大量运行期元数据，不利于版本合并。可用 Markdown 源编辑；远程机器则用端口转发。

A.1.2.1 Jupyter 中的 Markdown 文件

贡献应改 Markdown 源而非 `.ipynb`。插件把 `.md` 当作笔记本打开，使公式与代码单元可原地运行。

A.1.2.2 在远程服务器上运行 Jupyter Notebook

SSH 把远程内核端口映到本机同一或另一端口，浏览器仍访问 `localhost`。

A.1.2.3 执行时间

对每个代码单元记录墙钟时间，便于比较 $A^T B$ 与 AB 等运算。

A.1.3 小结

本地或经端口转发远程编辑单元。源文件用 Markdown 时，版本记录的是文本而不是输出缓存。

A.1.4 练习

核验：在 $\mathbb{R}^{1024 \times 1024}$ 上 $A^T B$ 与 AB 哪一个更快。

A.2 使用 Amazon SageMaker

当本地算力不够时，用托管笔记本实例跑 GPU 密集型单元。

A.2.1 注册

创建云账户并打开 SageMaker 控制台。建议打开计费警报，避免实例忘记停止。

A.2.2 创建 SageMaker 实例

选定实例类型即选定 (CPU 核数, GPU)。例如单卡 V100 加多核 CPU，足以覆盖本书大部分训练。

A.2.3 运行和停止实例

实例就绪后在浏览器打开笔记本。停止实例停止按计算计费；忘记停止会持续产生费用。

A.2.4 更新 Notebook

在实例终端向远程仓库取更新。若有本地改动，先提交或丢弃再拉取，以免合并冲突。

A.2.5 小结

SageMaker 把“开 GPU 笔记本”变成创建-打开-停止。更新走实例上的 Git。

A.2.6 练习

核验：任选一节 GPU 代码在实例上能否跑通，以及终端是否指向笔记本目录。

A.3 使用 Amazon EC2 实例

相对托管笔记本，自建虚拟机通常更便宜。步骤：申请 GPU 机、装 CUDA、装框架、端口转发。

A.3.1 创建和运行 EC2 实例

在 EC2 控制台启动实例。下面按位置、配额、映像与登录展开。

A.3.1.1 预置位置

选邻近区域以降低控制面延迟。需确认该区域提供所需 GPU 机型。

A.3.1.2 增加限制

新账户对 GPU 机型常有配额 0。先申请提高限额，再启动。

A.3.1.3 启动实例

选 Ubuntu 类映像与 GPU 机型。不同代号对应不同 GPU 个数与显存，按任务选。

A.3.1.4 连接到实例

用密钥文件 SSH。密钥权限必须仅本人可读，否则握手失败。

A.3.2 安装 CUDA

先更新驱动，再按发行说明安装与框架匹配的 CUDA 工具包。安装包 URL 与文件名随版本变，以厂商当前仓库为准。

A.3.3 安装库以运行代码

按本书安装章在远程 Linux 上建隔离环境并安装 PyTorch 与辅助包。无图形界面时，下载器用命令行取安装脚本，初始化后再加载 shell 配置。

A.3.4 远程运行 Jupyter 笔记本

云主机无显示器。从本地 SSH 并转发端口，使浏览器连本机端口即连上远程内核。端口转发必须在本地命令行发起：本地端口映到远程笔记本端口。激活环境后启动笔记本，把给出的 URL 中的端口改成本地转发端口再打开。

A.3.5 关闭未使用的实例

停止：盘仍在，可再开机，收少量存储费。终止：盘与数据删除，不可恢复。需要模板时先做映像再终止。

A.3.6 小结

按需启动 GPU 机；框架前先装 CUDA；用本地发起的端口转发跑笔记本。

A.3.7 练习

核验：竞价实例相对按需的价格，以及多卡机上数据并行能扩到几张卡。

A.4 选择服务器和 GPU

训练受 GPU 小时约束。单机可到数张卡；再多通常用云，因供电与散热。

A.4.1 选择服务器

算力在 GPU 上，CPU 核数不必最多，但单线程频率在多卡加 Python 时仍重要。电源按每卡峰值（可到数百瓦）留余量；机箱与风道必须容下长卡。PCIe 通道数限制满速 GPU 个数。

A.4.2 选择 GPU

加速库对其中一家的通用计算接口支持更成熟，故训练卡多选该路线。消费级与企业级算力接近，企业级多为被动散热、更大显存与 ECC，单价高一个数量级。实验室规模常用消费级；上百台则考虑企业级或云。新一代卡通常能效更好。低精度（如 FP16）提高吞吐，但训练需防止下溢。

A.4.3 小结

电源、通道、单线程 CPU 与散热决定能插几张卡。大部署用云；核对机械尺寸与功耗是否匹配机箱。

A.5 为本书做贡献

改正笔误、链接与表述通过向源仓库提交拉取请求完成。持续集成会跑改动过的代码。

A.5.1 提交微小更改

在网页上打开对应 Markdown，改一句，写说明并开拉取请求。

A.5.2 大量文本或代码修改

源为 Markdown 加书籍扩展（公式、图、交叉引用）。改代码时用笔记本跑通，提交前清空输出，由 CI 重算。多框架章节用标记区分代码块所属框架。

A.5.3 提交主要更改

标准 Git：派生、克隆、本地改、推送、开拉取请求。

A.5.3.1 安装 Git

在本机安装 Git 并注册托管账户。

A.5.3.2 登录 GitHub

在浏览器打开本书仓库，派生到自己的命名空间。

A.5.3.3 克隆存储库

用派生仓库的地址在本地做副本，再按安装章建环境。克隆地址中的用户名必须换成派生后的账户名，否则指向的不是可写副本。

A.5.3.4 编辑和推送

改文件后查看差异，提交到自己的分支并推送。

A.5.3.5 提交 Pull 请求

在派生仓库上对新改动开拉取请求，写清改了什么。可能被直接合并、要求修改或拒绝。请求应小，便于审阅。

A.5.4 小结

小改动可网页编辑；大改动走派生-本地-拉取请求。避免巨型请求。

A.5.5 练习

核验：完成一次派生，并尝试用新分支提交最小改动。

A.6 d2l API 文档

辅助包把模型、数据、训练循环与公用函数收成可复用映射。实现见源文件。

A.6.1 模型

网络模块： f_θ 的层、块与常见骨干，前向为张量到张量。

A.6.2 数据

数据集与加载器： $(\mathcal{D}) \mapsto$ 小批量迭代器，含词表、增广与填充。

A.6.3 训练

训练循环：给定 f_θ 、迭代器与优化器，执行前向、损失、反传与评估。

A.6.4 公用

作图、计时、设备选择与累加器等与任务无关的工具函数。